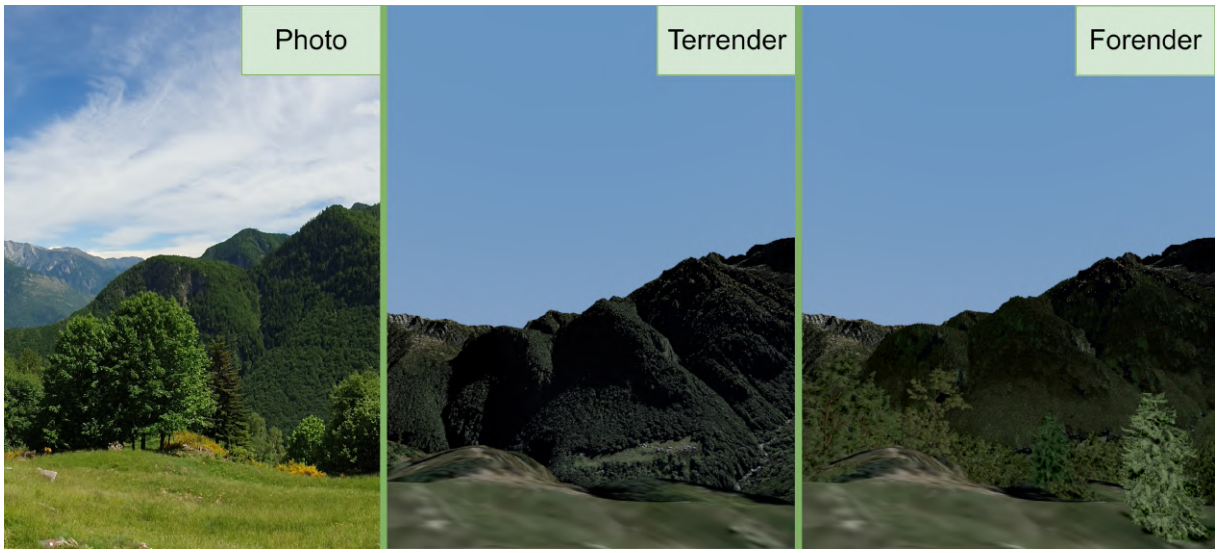
Forender: A Web-Based Real-Time Forest Renderer



Jan Grunder

Master's Thesis
August 2026

Julian A. Croci
Prof. Dr. Renato Pajarola, Prof. Dr. Robert W. Sumner

ETH
Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

University of Zurich[UZH]

Game Technology Center

VISUALIZATIONANDMULTIMEDIALAB

Abstract

Forest visualization is important for tasks such as urban planning, forestry, or hike planning. However, visualizing forests at large viewing distances remains challenging due to the visual complexity and large numbers of trees, especially for web applications. Existing web-based visualizers often struggle with interactive frame rates without reducing visual quality. This thesis introduces Forender, a web-based forest visualizer designed to maintain interactive frame rates and visual quality at large viewing distances. Forender is implemented in two graphics APIs, WebGL and WebGPU. Both implementations use a quadtree for asynchronous streaming of forest data and for hierarchical culling. Forender WebGPU uses GPU-driven rendering to perform additional per-instance culling, density reduction, and LOD selection. The finer granularity can reduce unnecessary GPU work compared to Forender WebGL's coarser per leaf node operations. Forender WebGL achieves interactive performance in smaller scenes, however it scales poorly to large scenes. In contrast, Forender WebGPU scales efficiently to larger scenes containing up to 18 million trees, while supporting additional visual features such as multiple tree species, leaf tinting, and screen-space ambient occlusion. These results demonstrate that GPU-driven rendering, enabled by the capabilities provided by WebGPU, is a practical and efficient approach for scaling web-based forest visualization to large scenes and viewing distances.

Acknowledgements

I want to express my gratitude to my supervisor Julian Croci, who provided both technical feedback as well as a place to rant about my latest issues with the Web as a platform. I also want to express my thanks to all the staff and students at Game Technology Center for both their valuable feedback on many of my diagrams as well as the social gatherings and lunches I spent with them. Especially, I want to thank Henry Raymond, Prof. Dr. Renato Pajarola and Prof. Dr. Robert W. Sumner for overseeing this thesis and making the work across the two groups possible. I would like to thank the Swiss Federal Office of Topography (swisstopo) both for their large and open collection of geographical data and for answering my question about their forest visualizer.

I am immensely grateful to my family and friends for their support throughout my studies and this thesis. The stressful times were better, as the suffering was at least shared with friends.

Contents

List of Figures ix

List of Tables xi

1. Introduction 1
- 1.1. Contributions . . . 2
- 1.2. Outline . . . 3

2. Related Work 5
- 2.1. Tree Extraction . . . 5
- 2.2. Vegetation Rendering . . . 6
  - 2.2.1. Texture-Based Methods . . . 7
  - 2.2.2. Geometry-Based Methods . . . 7
  - 2.2.3. Culling . . . 9
  - 2.2.4. Industry Usage . . . 10
- 2.3. Existing Visualizers . . . 10

3. Implementation 13
- 3.1. Datasets . . . 14
  - 3.1.1. Vegetation Height Model NFI . . . 14
  - 3.1.2. swissALTI3D . . . 15
  - 3.1.3. swissTLM3D . . . 15
  - 3.1.4. Forest Mix Rate NFI . . . 15
  - 3.1.5. swissIMAGE . . . 15
  - 3.1.6. 3D Models . . . 16
- 3.2. Data Preprocessing . . . 16
  - 3.2.1. Position and Height Computation . . . 16
  - 3.2.2. Optional Data . . . 18

3.2.3. Quadtree Building . . . 19
3.3. Server Backend . . . 21
3.4. Frontend Overview . . . 22
3.5. WebGL Frontend . . . 23
3.5.1. Startup . . . 24
3.5.2. Culling . . . 25
3.5.3. Density . . . 27
3.5.4. Level of Detail . . . 28
3.5.5. Instanced Rendering . . . 29
3.5.6. Shading . . . 30
3.5.7. Blending . . . 31
3.5.8. Z Position Updates . . . 32
3.6. WebGPU Frontend . . . 33
3.6.1. Density . . . 34
3.6.2. GPU-Driven Rendering . . . 35
3.6.3. Occlusion Culling . . . 37
3.6.4. Leaf Tinting . . . 38
3.6.5. Alpha to Coverage . . . 39
3.6.6. SSAO . . . 39

4. Results **43**
4.1. Evaluation Setup . . . 43
4.2. Quantitative . . . 45
4.2.1. WebGL Implementations . . . 47
4.2.2. WebGL Z Updates . . . 47
4.2.3. Forender WebGL and Forender WebGPU . . . 49
4.2.4. WebGPU Quadtree Depth . . . 51
4.2.5. WebGPU Optimizations . . . 51
4.2.6. WebGPU Startup Times . . . 53
4.2.7. WebGPU Long-Term Stability . . . 54
4.2.8. iGPU Performance . . . 55
4.3. Qualitative . . . 56
4.3.1. Tree Position Extraction . . . 56
4.3.2. LOD Decision . . . 57
4.3.3. Density Downsampling . . . 58
4.3.4. Tree species . . . 62
4.3.5. Leaf Tinting . . . 63
4.3.6. Alpha to Coverage . . . 65
4.3.7. SSAO . . . 65

5. Conclusion **67**
5.1. Limitations and Future Work . . . 68

A. Appendix **69**

B. Declaration of Usage of AI **75**

Bibliography **76**

List of Figures

3.1. Implementation overview 14
3.2. Misclassifications in Vegetation height data 17
3.3. Offline z computation 18
3.4. Processing of Vegetation mix rate 19
3.5. WebGL frontend 23
3.6. Quadtree frustum culling 26
3.7. Culling visualization 27
3.8. Instanced drawing implementations 29
3.9. Alpha modes 30
3.10. LOD Blending 31
3.11. WebGPU frontend 33
3.12. Per instance decisions 35
3.13. GPU-driven rendering 37
3.14. Occlusion culling test 38
3.15. Terrain normal discontinuity 40
3.16. SSAO noise 41

4.1. Extent of evaluation scenes 44
4.2. Comparing WebGL implementations 48
4.3. Cost of runtime z position update 49
4.4. Comparing Forender WebGL & Forender WebGPU 50
4.5. Empirical evaluation of quadtree depth 52
4.6. Ablation study on optimizations 53
4.7. Startup times in Forender WebGPU 54
4.8. Long-term trajectory 55
4.9. Tree position extraction 57
4.10. False positive tree 58

4.11. LOD decision visuals 59
4.12. Density levels with different methods 60
4.13. Per instance density at various distances 61
4.14. Tree species computation 62
4.15. Leaf tinting masks 63
4.16. Leaf color under- and over-blurring: 64
4.17. Various tint strengths from afar 64
4.18. Alpha to Coverage visuals 65
4.19. SSAO visuals 66

A.1. FLV visuals 69
A.2. Visual Forester visuals 70
A.3. swisstopo3D visuals 70
A.4. Stahl visuals 70
A.5. Remaining density levels with different methods 71
A.6. Mipmap issues in Alpha Masked material 72
A.7. Instance density with Terrain background 72
A.8. Large viewing distance screenshot 73
A.9. Real world comparison 73

List of Tables

2.1. Comparison between different forest visualizers 10

3.1. Frontend Overview 22
3.2. Overview of material properties 25

4.1. Forender parameters 45
4.2. iGPU frame time 56

A.1. Geometric complexity of tree LODs 71

1

Introduction

Humans have long been using maps to visualize the world around us and to improve their understanding of their environment. The process of how we visualize has changed a lot throughout time, from maps to 3D point cloud captures of buildings. Early manual map-making approaches[1] have largely been replaced with more modern approaches to capture our world such as cameras and LiDAR. With advancements in computer graphics, it became possible to visualize the world in 3D, enriching the maps with additional landmarks such as buildings. Geographic information systems (GIS) provide tools to visualize and analyze the captured geographical data.

GIS software is used in many areas ranging from geography to urban planning or forestry. GIS applications are used not only in a work environment, but also for hobby activities such as planning hikes or other trips [Croci et al., 2025]. The increasing capabilities and availability of Web technologies have made it possible to run GIS software directly in the browser, reducing the need to install specific software.

The 3D visualization of terrain is well-supported in many GIS applications using digital elevation data. A *Digital Elevation Model* (DEM) represents the elevation of the terrain, while a *Digital Surface Model* (DSM) represents the elevation of the terrain including above-ground features such as buildings and trees. However, in many applications, the objects present in the DSM are not visualized with quality similar to the terrain. Among these various objects, trees play an important role in the visualization of natural and urban environments, particularly for forestry and urban planning. In these applications, tree visualization is important to visualize the spatial tree distribution as well as to visualize sightlines. However, detailed visualization of trees and forests at large viewing distances presents significant performance challenges, particularly in web-based applications.

Visualizing forests over large viewing distances is challenging due to their visual complexity

[1] `https://www.swisstopo.admin.ch/en/dufour-map` (Accessed: 17.08.2026)

and the large number of trees that need to be represented. Trees are complex both from a geometry perspective with many fine details and from a shading perspective due to their translucent appearance. Although simplified representations can reduce the rendering cost, they reduce the visual quality when viewed up close. Consequently, maintaining sufficient visual quality while rendering a large number of trees at large viewing distances can become computationally expensive.

Interactive forest visualization at large viewing distances has been demonstrated by existing visualizers. However, these visualizers are often implemented as native applications rather than for the Web. Existing browser visualizers, however, either limit the viewing distance drastically or exhibit notable frame rate drops.

The thesis investigates how large forest scenes can be visualized in a web-based 3D environment at large viewing distances while maintaining interactive frame rates. To address this question, a web-based 3D forest visualizer is developed. The visualizer adopts techniques commonly used in video games and other 3D desktop applications to the Web environment. Special attention is given to the trade-off between performance and visual quality.

1.1. Contributions

This thesis makes the following contributions:

- **Offline pipeline for extracting tree information from public geographical data.** Beyond position and height, the pipeline extracts per-tree species and averaged leaf color, enabling rendering trees with a more accurate appearance.
- **Adaptation of established real-time rendering techniques to the constraints of the Web.** Web-based forest visualizers do not consistently apply well-established graphics techniques such as frustum culling and discrete LODs. This thesis integrates such techniques into a web-based renderer and introduces the use of multiple density levels, allowing a gradual reduction in the number of trees at a distance.
- **GPU-driven rendering approach built using WebGPU.** To address performance and appearance limitations imposed by WebGL limitations, this thesis explores GPU-driven rendering, enabling fine-grained culling and LOD decisions. To our knowledge, this is one of the first application of GPU-driven rendering to large-scale forest visualization on the Web.
- **Performance and visual quality evaluation.** The proposed approaches are evaluated on scenes containing up to 18 million extracted trees. Multiple trajectories were chosen to stress test different viewing conditions. The results show that interactive frame times are maintained with a mean frame time of at most 20 ms and a 99th percentile of at most 32 ms across all tested scenes, demonstrating the feasibility of interactive web-based forest visualization at large viewing distances.

1.2. Outline

Chapter 2 provides an overview of the existing research in tree extraction and rendering, as well as a brief comparison with existing visualizers. Chapter 3 describes the implementation of the web-based visualizer and the optimizations used to achieve interactive frame rates. The benchmarks and visual results are presented and discussed in Chapter 4. The thesis is concluded in Chapter 5 by discussing its limitations and possible future work.

2

Related Work

The goal of this thesis is to render large forests in real-time on the Web, with trees being accurately represented. This raises two problems, which the thesis addresses. The first being how to go from geographical data to a representation of forests that can be rendered, which is discussed in Section 2.1. The methods should remain accurate to the geographical data with regard to tree position, height, and species. The second part focuses on how to efficiently render large forest scenes that are represented by the data gained in the first part. This topic is the focus of Section 2.2. Section 2.3 focuses on existing visualizers that already attempt to solve this problem, not just one of the two parts.

2.1. Tree Extraction

This section covers various preexisting approaches for extracting trees from real-world data. The extracted output is intended for real-time rendering, as discussed in Section 2.2.

Although 3D reconstruction of scenes such as [Mildenhall et al., 2021] and [Kerbl et al., 2023] has improved greatly over the last few years, with some methods specifically made to support vegetation such as [Ojo et al., 2024], [Yang et al., 2025] and [Grammatikaki et al., 2026], they still struggle with larger scenes and cannot currently be used to reconstruct an entire forest at our desired visual fidelity. As this is still an active area of research, this assessment may change. Different methods are needed to position and scale the vegetation and use the correct type of vegetation with respect to geographical data. Given the extracted data, computer graphics ideas from section 2.2 can be used to render either a preexisting or a procedural mesh. We first discuss some noise-based approaches, then we mention ecosystem simulation approaches before discussing swisstopo 3D, on which our extraction is based.

A simple approach to positioning would be to use a random noise distribution within a forest region. Stahl [Stahl, 2021] uses a tileable Poisson pattern to place trees where the DSM contains

vegetation that is higher than the ground by some threshold. The tree height can be extracted using the same DSM. The implementation by Stahl also distinguishes between coniferous and deciduous trees given a map with the ratio between coniferous and deciduous trees. Although the Poisson pattern gives a nice spatial distribution, it might not match the underlying vegetation. Noise-based methods often do not match the density of the forest nor do they match the accurate positions of trees, and are thus not used in Forender.

Instead of using noise to imitate the spread of plants, Deussen [Deussen et al., 1998] places plants by simulating an ecosystem given terrain data (elevation, orientation, water content, etc.) and properties of the different plant species. The plants are represented as circles that will compete with each other if they overlap. Pałubicki [Pałubicki et al., 2022] also simulates an ecosystem, modeling tree growth as a response to water, temperature, and light to capture the response of vegetation to climate change. The paper includes a comparative analysis with other findings in ecology. Niese [Niese et al., 2022] focuses on plant placement and growth in urban environments. The paper introduces multiple procedural placement models (PPM) for plants, using different models for different zoning within a city. The parameters of those PPMs can either be user defined or learned from satellite data. Although simulation-based methods are better at modelling realistic placement of trees when compared to noise-based approaches, they do not precisely correspond to measured geographic data. They are more useful in procedural placement for synthetic regions than for visualizing the real world.

Unlike ecosystem approaches, swisstopo 3D [swi, 2026] visualizes existing forests extracted from geographical data. swisstopo 3D uses swissSURFACE3D[1] (DSM) to accurately place vegetation in its 3D map. swissSURFACE3D is a point cloud with different classifications such as trees, ground, buildings, and water. From the vegetation class, swisstopo extracts locally maximal points that are within forest polygons given by swissTLM3D. swissTLM3D is described in Section 3.1.3. In addition to these trees, it also makes use of the data in swissTLM3D feature class "TLM_EINZELBAUM" to add trees back that are outside of forest polygons. To obtain the z position of the trees, it uses swissALTI3D (DEM). Swisstopo does not distinguish between different tree types.

Forender implements an approach similar to swisstopo 3D but aims to support distinguishing between deciduous and coniferous trees, each with individual leaf colors for greater variety. Other tree extraction methods from DSM data exist, as discussed in [Saeed et al., 2024], however, the extraction method itself is not the focus of this thesis and could be replaced by other methods if needed.

2.2. Vegetation Rendering

Rendering vegetation, especially at large scales, is difficult for multiple reasons. One reason is that the large geometric complexity necessary to represent foliage up close is too expensive to be used to render each tree in view. This issue is made even worse by the fact that large parts of the geometry are alpha tested, i.e., discarding fragments based on the alpha values, which disables some GPU optimizations such as Early Z testing. Another reason is the shading complexity due

[1] `https://www.swisstopo.admin.ch/en/height-model-swisssurface3d` (Accessed: 03.08.2026)

to translucency. Although there are methods to accurately render leaves in real-time, the focus of this thesis lies on the geometric complexity of foliage and how to deal with it.

We distinguish between two popular groups of approaches. While some methods lie outside these two categories, most can be split into either texture-based approaches that try to represent the geometric complexity using textures and geometry-based approaches that focus on simplifying said complexity.

2.2.1. Texture-Based Methods

A conceptually simple texture-based representation of vegetation is billboards, i.e., flat, textured quads. Their implementation complexity can vary a lot, from simple representations such as camera-aligned billboards to crossed billboards (2-3 billboards) and billboard clouds, which find the optimal placement of n billboards. Behrendt [Behrendt et al., 2005] uses billboard clouds to represent trees, combining them with spherical harmonics to capture self-shadowing. Using spherical harmonics for self-shadowing can lead to smoothed out shadows, as spherical harmonics can struggle to represent high frequency signals if only a few bands are used. Their method also suffers from parallax problems when viewed at very close distances due to the planar nature of the billboards.

Another texture-based method is impostors [Schaufler and Stuerzlinger, 1996], where the mesh is rendered from multiple views as a preprocessing step. At runtime, the closest views to the camera are interpolated to get the correct color given the current view. Meyer [Meyer et al., 2001] extends this idea and looks up views based on both the camera and the light direction, enabling more complex lighting effects at the cost of requiring more memory.

Bruneton [Bruneton and Neyret, 2012] introduces z-field trees which are similar to impostors but store depth maps and ambient occlusion instead of color in their precomputed textures. The depth maps are used to approximate an intersection between a camera ray and the z-field tree, which makes a more accurate depth estimate possible. The accurate depth enables effects such as self-shadowing, overlap between multiple z-field trees, and receiving shadows cast by other trees. Z-field trees have no mechanism for proper albedo texture in their original paper. Another issue of z-field trees that it shares with impostor methods is their high memory cost per tree species.

Texture-based approaches are often combined with geometry-based ones. This combination is due to texture-based approaches suffering from artifacts when viewed up close.

2.2.2. Geometry-Based Methods

The idea of geometry-based methods or level of detail (LOD) methods is to simplify the complex mesh. The switch to a simplified model is possible without (much) perceptual difference if the object is either far away or small. We differentiate between two categories: discrete level of detail and continuous level of detail. In the discrete methods, there are a few models of the same object all with varying details, whereas in the continuous ones, there are (nearly) continuous ranges of how detailed the model is, often with different detail levels present within the

same model instance.

Discrete level of detail can either use artist-created or algorithmically simplified models. Although there are many general-purpose simplification methods (see [Heckbert and Garland, 1997] and [Khan et al., 2022]), they often struggle with foliage, which has led to research into foliage-specific methods. Remolar [Remolar et al., 2002] simplifies geometry by merging leaf pairs given their relative distance and planarity. Gasch [Gasch et al., 2018] removes leaves completely depending on their visual impact on multiple views. In order not to lose the leafy appearance of the tree, the method scales another leaf to compensate for the loss. Neither of these methods considers the effect of geometric modifications on the texture, stretching it to cover the new area, leading to visible distortion. Both methods focus on trees with many small and similar leaves. The underlying leaf merging or removal might not generalize to trees with larger but fewer leaves such as palms.

Texture-based methods can also either generate multiple LODs or be used as a specific LOD. For example, the method by Kratt [Kratt et al., 2014] uses billboard clouds to generate multiple levels of detail.

Changing the discrete level of detail from one frame to the other can lead to sudden changes in the image, i.e., popping. To avoid that issue, Giegl [Giegl and Wimmer, 2007] uses transparent rendering on LOD change. First, the method fades the new LOD in, keeping the old opaque. Once both are opaque, the old one gets faded out. CryEngine 2 [Mittring, 2007] similarly uses transparency for LOD transitions, but relies on screen door transparency instead of blending hardware.

Discrete methods, by definition, use one level of detail for the whole model, even if parts of it could be rendered at lower detail. They also have a limited number of LODs which require additional methods to make the transition more seamless as described above. Continuous LOD methods are designed to avoid these limitations. Although general-purpose, continuous LOD solutions exist (such as [Karis et al., 2021]), they can suffer from similar issues as generic discrete methods, thus, specific methods are needed. Remolar [Remolar et al., 2003] extends [Remolar et al., 2002] to a continuous LOD method using leaf merging. Similarly, Gasch [Gasch et al., 2020] extends [Gasch et al., 2018] with leaf removal depending on the point of view. Both continuous LOD solutions [Remolar et al., 2003] and [Gasch et al., 2020] are implemented on the CPU and have some of the same limitations as their discrete counterparts. Gumbau [Gumbau et al., 2011] groups leaves into cells and precomputes visibility of the cells from different views. At runtime, the method removes leaves based on the interpolated cell visibility from the three closest viewpoints and the distance from the camera. This method is implemented in CUDA (GPU), unlike the previous methods, thus reducing CPU-GPU communications. This continuous LOD method is limited to static trees as the precomputation may no longer be accurate on the deformed tree due to wind or other forces.

Hu [Hu, 2023] also supports merging leaves similar to [Gasch et al., 2020] but is implemented on the GPU. By baking the merged textures beforehand, the method reduces the visual error caused by merges. This solves the previously discussed issue of texture distortion which was not addressed by either Gasch or Remolar. However, like their methods, this approach still focuses on trees with many small leaves. Nanite Foliage [Epic Games, 2025] switches to a voxel representation at a distance to avoid the shortcomings of the general Nanite implementation.

Although GPU-driven continuous LOD methods seem promising, they are often implemented and tested with a focus on the newest graphics cards and use features such as mesh shaders not available in any Web graphics API. In [Hu, 2023], mesh shaders could be replaced with compute shaders. However, this change is likely slower due to requiring writing the results to a buffer. To reduce hardware requirements and being based on the Web, Forender uses discrete LOD, with the lowest quality LOD consisting of billboards.

2.2.3. Culling

The methods discussed above reduce the scene's geometric complexity but still incur some unnecessary cost when used on their own. One approach to reduce that cost is to perform culling, i.e., not drawing objects that do not contribute to the final image. Without frustum culling, objects outside the frustum still incur cost as the vertex shader is executed before clipping. Similarly, without occlusion culling, objects that become fully occluded after being visible execute both their vertex and fragment shaders unnecessarily.

Frustum culling checks if objects are within the camera's view frustum. Clark [Clark, 1976] introduces hierarchical culling, using a tree data structure to change the frustum culling cost to logarithmic in the number of objects. Hierarchical culling is not always used in practice, for example, Battlefield [Collin, 2011] performs culling linearly to benefit better from caching, SIMD and multithreading. Note that at the time of writing, Battlefield supported a maximum of 15'000 objects.

Screen-space culling removes objects that are below a certain threshold in size on the screen. Unlike frustum or occlusion culling, there is a difference in the final image, since the objects would have been partially visible. Bruneton [Bruneton and Neyret, 2012] uses screen-space culling to stop drawing trees and instead shade the terrain the corresponding tree color. To ensure a seamless transition, it fades out trees at a distance and fades in the new terrain color. Similarly, Whatley [Whatley, 2005] uses screen door transparency to blend out vegetation further away.

Occlusion culling avoids drawing objects that are completely hidden by others. A classical approach for occlusion culling is introduced by [Greene et al., 1993] and [Greene, 1995]. The method builds a depth pyramid given the rasterized depth of some occluders and checks objects against the pyramid. The method was originally designed for the CPU but is now the basis of many GPU-driven occlusion culling systems such as [Haar and Aaltonen, 2015] and [Jansson, 2024]. The choice of which occluder to render and check against has a great impact on the performance of the implementation. Haar [Haar and Aaltonen, 2015] uses the closest objects, and Alan Wake 2 [Jansson, 2024] uses objects that were visible in the previous frame. CryEngine [Kaplanyan, 2010] uses the z buffer of the previous frame to build its depth pyramid, which might lead to false negatives for one frame. Haar [Haar and Aaltonen, 2015] combines the z buffer of the best occluders with a reprojection of the previous frame's z buffer.

Forender targets scenes containing millions of trees, thus motivating the use of hierarchical culling. Forender's WebGPU implementation uses compute shaders to perform additional frustum, distance and occlusion culling per tree. Forender introduces the notion of density in Section 3.5.3, a mechanism similar to screen-space culling but applied more gradually.

2.2.4. Industry Usage

Although the research in the above section introduced many possible approaches for rendering vegetation, it is not always clear how well they would integrate with other systems. By looking into the rendering techniques used in modern, real-time computer graphics software such as video games, we can see some indications of what is production ready and what requires further research.

Most video games, such as Horizon Zero Dawn [Sanders, 2018] or God of War [Feeley, 2019], make heavy use of discrete level of detail for vegetation. The latter uses billboard clouds for low detail LODs and for shadow proxies, meaning only billboard clouds, rather than any higher LODs, are rendered during a shadow pass. In their presentation, they explicitly mention that they did not use impostors, due to them being unable to support interactive and animated foliage. Fortnite [Brucks, 2018] used impostors for trees at a distance. Whether impostors are still used today is unknown. Microsoft Flight Simulator [Fuentes, 2022] used impostors for their trees (without LOD) due to the high number of trees visible from their camera perspective, compared to other games.

Current productions rely on discrete LOD with some of the low-quality levels being represented using texture-based approaches, reinforcing the choice of using discrete LOD in Forender.

2.3. Existing Visualizers

As forest visualization is an important topic for many tasks such as architecture, forestry, climate research and entertainment, there exist many different visualizers (see Table 2.1) that handle both the extraction of trees from geographical data and the rendering of them.

	Commercial Visualizers			**Research**					
	Visual Forester [Alpine LIS, 2026]	ArcGIS Tree Management [ESRI, 2026]	SwissTopo [swi, 2026]	[Wang et al., 2017]	[Stahl, 2021]	Virtual Forest [Stora Enso Metsä, 2026]	EcoVis [3] [Rammer et al., 2026]	FLV [Yu et al., 2022]	**Forender**
Platform	Desktop (NA)	Web (WebGL)	Web (NA)	Web (X3DOM)	Desktop (OpenGL)	Desktop (NA)	Desktop (OpenGL)	Desktop (Unreal Engine)	Web (WebGL & WebGPU)
Multiple species	✓	NA	X	✓	✓[2]	✓	✓	✓	✓[2]
Accurate height	✓	✓	✓	✓	✓	✓	✓	✓	✓
Far view distance	✓[1]	X	X	X	✓	X	✓	✓	✓

[1] only in Visual Forester Pro [2] only deciduous / coniferous distinction [3] only symbolic real-time rendering

Table 2.1.: Comparison between different forest visualizers: *Overview of different visualizers from both the commercial and research sector. Forender is included as a reference, not prior work, with this thesis aiming to achieve these goals. Not all categories are known publicly with such cases being indicated as NA. Most entries stem from published papers and website material rather than testing or actual code.*

Alpine LIS develops two visualizers, Stand Designer and Visual Forester Pro. Their capabilities are reported to be quite similar with Visual Forester Pro supporting farther view distances but lacking some simulation tools and visual effects. Stand Designer and Visual Forester Pro [Alpine LIS, 2026] use DEM, vector data, or GIS data derived from LiDAR to visualize forests. Both support up to 32 different tree species, as well as shrubbery below the trees. Only Stand Designer integrates with simulators that model forest growth and supports different weather effects such as fog (see Figure A.2b). ArcGIS Tree Management [ESRI, 2026] comes with a tree viewer. Their solution can extract tree locations from a point cloud, load them from GIS

data, or allow manual input into an inventory. Swisstopo [swi, 2026] uses DSM, vector data and DEM to extract points and render them as mentioned in Section 2.1. The 3D renderer is based on Cesium [Cesium GS, 2026] and does not use LOD nor visualize different species. The view distance is quite limited and is cut off on a per-tile basis (see Figure A.3b).

Wang [Wang et al., 2017] uses DEM, vector data, and density maps to position trees. Trees are randomly placed inside forest polygons with a minimum and maximum distance to each other. The implementation supports multiple LODs and is written in X3DOM, which is based on WebGL. Stahl [Stahl, 2021] uses DEM and DSM to extract vegetation as explained in Section 2.1. The renderer is written using OpenGL. Nearby trees are rendered as z-field trees [Bruneton and Neyret, 2012]. Further away, the method tints the terrain to match the tree color, where no actual tree geometry is used anymore. The z-field representation enables the implementation of volumetric effects such as the silver lining of the leaves (see Figure A.4b). Metsabiotalous [Stora Enso Metsä, 2026] uses data from DEM and DSM to extract tree position and height but suffers from low-quality models and very noticeable LOD popping. EcoVis [Rammer et al., 2026] is a real-time visualizer using density maps and Poisson sampling similar to [Wang et al., 2017] to place their trees. While their real-time renderer focuses more on symbolic visualization, they support exporting to Mitsuba for photorealistic, non-realtime rendering. Similarly, FLV [Yu et al., 2022] also uses a per-species density map to place vegetation but uses uniform sampling instead. Their large scenes are handled by Unreal Engine and they additionally scatter shrubs and rocks randomly on the forest floor (see Figure A.1).

Forender aims to provide a web-based visualizer with features previously only available in desktop applications. By using different techniques introduced throughout the chapter such as hierarchical culling, occlusion culling and discrete LOD, Forender visualizes accurate geographic data at large viewing distances in real-time.

3

Implementation

To achieve real-time performance in scenes containing millions of trees, Forender shifts computationally expensive operations such as tree extraction and spatial acceleration structure building into an offline preprocessing step. This is reflected in Forender's architecture, which is split into three parts: the preprocessor, the frontend, and the backend (see Figure 3.1). The preprocessor takes in the vegetation height (Section 3.1.1) and a height pyramid, extracting tree position p and height h and grouping them into a quadtree for efficient rendering in the frontend.

Since all processing is performed offline, the backend merely serves the static files, such as the website, the processed files and the terrain tiles.

There are two frontend implementations in Terrender, one implemented in WebGL and one in WebGPU. Both frontends are built on top of their corresponding Terrender implementation. As rendering full quality tree models is infeasible for large scenes, the Forender frontend uses a discrete level of detail system to reduce geometric complexity. To avoid unnecessary overhead of rendering models not visible on screen, Forender performs culling. With scenes containing millions of trees, Forender cannot perform per tree computation in real-time on the CPU due to them scaling linearly with tree count.

To avoid per-tree computation, Forender uses hierarchical culling with both frustum and distance checks using the precomputed quadtree. In Forender WebGL, LOD selection is computed per leaf node to avoid similar scaling issues as are present in culling. The forest density is further reduced at a distance, similarly to screen space culling, to lower the number of trees rendered without significant visual impact.

The chapter follows the data flow through the system. We first introduce the different data sets used in Section 3.1. Next, we describe how the preprocessor converts those datasets into an efficient runtime representation in Section 3.2. We then describe the backend responsible for serving these assets in Section 3.3. Finally, we provide an overview of both frontends in Section 3.4 before going into detail on the WebGL (Section 3.5) and WebGPU (Section 3.6)

frontends.

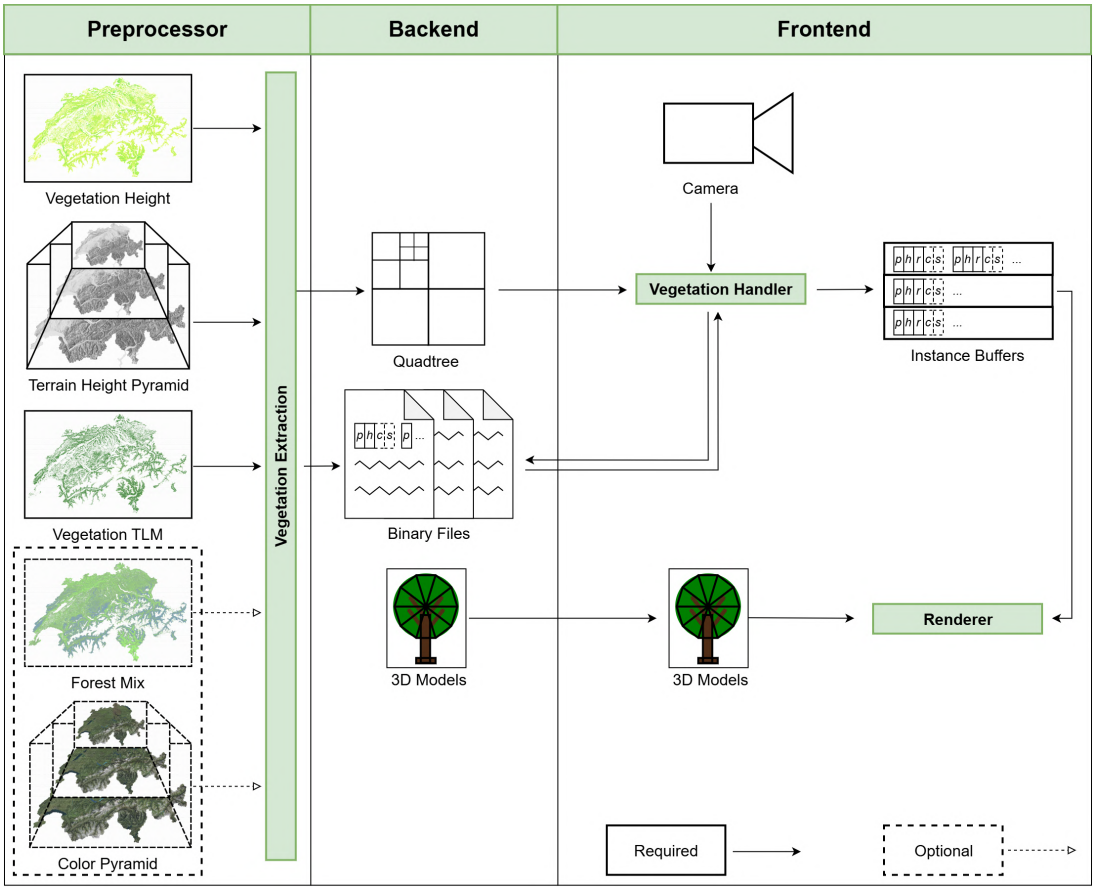


Figure 3.1.: Implementation overview: *Diagram showing the three core parts of Forender: the preprocessor, the backend and the frontend.*

3.1. Datasets

This section describes the datasets used in Forender. The datasets cover the whole of Switzerland, allowing for large-scale scenes, and are publicly available. Although this thesis focuses on Switzerland, comparable datasets exist in other regions or can be derived from other data as an additional preprocessing step.

3.1.1. Vegetation Height Model NFI

The Vegetation height model [Marty and Ginzler, 2025] is a raster image representing the height of vegetation in Switzerland. It is based on swissSurface3D [Federal office of Topography swisstopo, 2024c], a digital surface model (DSM) created using LiDAR. The swissSurface3D point cloud with the category Vegetation is rasterized to a resolution of 0.5 meters. The raster values are normalized such that the value of the pixel corresponds to the distance from the ground.

Each year, 1/6 of Switzerland is updated. We merge the per year data using GDAL[1]. Vegetation height is used to place trees using local maxima extraction as explained in Section 3.2.1.

3.1.2. swissALTI3D

swissALTI3D [Federal office of Topography swisstopo, 2024a] is a digital elevation model (DEM) of Switzerland with a spatial resolution of 0.5 meters. DEM do not contain vegetation or buildings. We do not directly rely on swissALTI3D but instead make use of the height pyramid created as a preprocessing step in Terrender. Using the height pyramid from Terrender ensures that the tree's z position matches the height of the rendered terrain more closely.

3.1.3. swissTLM3D

swissTLM3D [Federal office of Topography swisstopo, 2026] is a topographic landscape model (TLM) of Switzerland containing artificial objects, such as roads and buildings, as well as natural objects, both represented as vector data. The dataset is created by manual measurement on aerial images. Important for us are the forest polygons (”Wald”, ”Wald offen” and ”Gebueschwald”) as well as isolated trees (”TLM_EINZELBAUM”). The data can be downloaded as a single file. The forest polygons are used for filtering, while isolated tree markers are used to preserve trees outside of forest regions.

3.1.4. Forest Mix Rate NFI

Forest Mix rate NFI [Waser et al., 2025] is a raster map representing the ratio between deciduous and coniferous trees in Switzerland. It has a resolution of 10 meters and is created using machine learning on satellite images. The data can be downloaded as a single file. The raster is used in preprocessing to determine the optional species data.

3.1.5. swissIMAGE

swissImage [Federal office of Topography swisstopo, 2024b] are orthoimages of Switzerland with a 0.1 meter resolution. Orthoimages are based on aerial images that are modified to remove the impact of terrain inclination and camera distortions. The process relies on a DEM, in their case, swissALTI3D. As is the case with swissALTI3D, we do not use swissImage directly, but instead the color pyramid created in Terrender's preprocessing. The color pyramid is used to tint leaves to increase visual fidelity and more closely match the underlying terrain.

[1] `https://gdal.org/en/stable/` (Accessed: 13.07.2026)

3.1.6. 3D Models

The 3D models used in this thesis are of a maple [LOLIPOP, 2025] and a fir tree [LOLIPOP, 2024]. Although the provided model already contains multiple LOD variants, we decimated them with gltfpack[2] to more closely match the triangle counts of the models used in 3D games. This decimation makes the evaluation data more representative, matching the amount of detail that would be used in real products. The models are stored as `glb` files. The vertex and triangle counts are noted in Table A.1.

3.2. Data Preprocessing

The preprocessor converts geographic data into a format that can be efficiently loaded and rendered by the frontend. Given the vegetation height texture and the height pyramid, it extracts realistic tree position p and height h. Optionally, the preprocessor can also use a color pyramid and a forest mix rate image to compute a per tree color c and tree species s, adding greater visual variety. The preprocessor creates a quadtree, later used for hierarchical culling (see Section 3.5.2). To enable streaming of the tree data only when necessary, the preprocessor splits tree data into multiple binary files.

The preprocessor is implemented in Python and uses rasterio[3] for `TIF` loading, geopandas[4] for loading TLM data, and skimage[5] for extracting local maxima.

3.2.1. Position and Height Computation

The extraction of tree position p and tree height h is implemented similarly to swisstopo 3D [swi, 2026]. The pseudocode for it is shown in Algorithm 1. Instead of using the point cloud data of swissSurface3D, as done in swisstopo 3D, we use the rasterized Vegetation height model NFI and extract the local maxima from that.

Algorithm 1 Tree position and height extraction

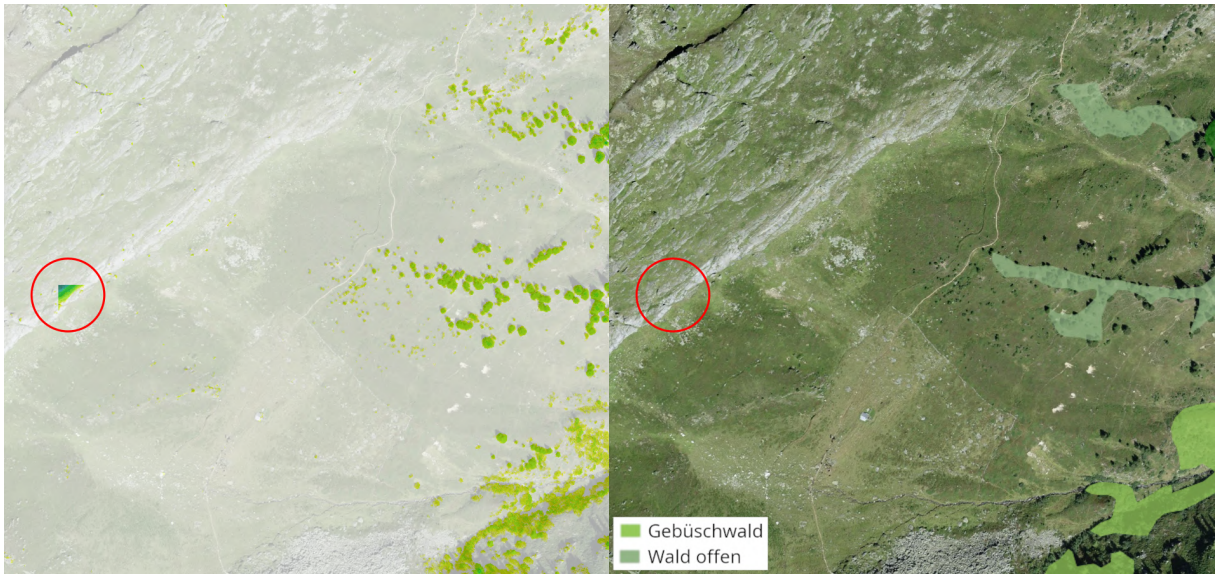
for each patch $\in$ vegetation height image **do**
  $p_{xy} \leftarrow p_{xy} \cup$ extract local maxima(patch, min distance, min height)
end for
$p_{xy} \leftarrow$ filter forest polygon(p_{xy}) $\cup$ single trees
$p_z \leftarrow$ sample terrain height pyramid(p_{xy})
$h \leftarrow$ sample vegetation height(p_{xy})

The runtime cost of local maxima computation in skimage scales poorly with large images. Therefore, the computation is performed per image patch. The implementation of local maxima extraction allows us to specify a minimal distance between maxima within our image patch.

[2] `https://meshoptimizer.org/gltf/` (Accessed: 13.07.2026)
[3] `https://rasterio.readthedocs.io/en/latest/index.html` (Accessed: 28.07.2026)
[4] `https://geopandas.org/` (Accessed: 28.07.2026)
[5] `https://scikit-image.org/` (Accessed: 28.07.2026)

By tuning this distance, we reduce the cases where two local maxima belonging to one tree are counted as two separate trees. Note that this distance might not be guaranteed between maxima from different patches, leading to some trees being present twice. Additionally, all potential tree locations are required to be above a certain height, with the others being filtered out to avoid shrubbery being represented as a tree.

There are still some issues to deal with after computing the local maxima. Some areas of swissSurface3D are mislabeled as shown in Figure 3.2, leading to false positive tree locations. To prevent these problems, we only allow tree positions that are within tree polygons ”Wald” and ”Wald offen” from swissTLM3D, as done in swisstopo 3D. By restricting trees to the forest polygons, we remove many false positives at the cost of also discarding all trees outside of forest regions. These are reintroduced by using the marker for ”TLM EINZELBAUM”. In contrast to swisstopo 3D, we do not use the height data of the tree marker but sample the height directly from Vegetation height to remain consistent with the other trees.



(a) *Vegetation height model NFI* ***(b)*** *swissTLM3D (Forest polygons)*

Figure 3.2.: Misclassifications in Vegetation height data: *The left image shows an example of misclassifications in swissSurface3D that could lead to false positive tree positions. The right image depicts the forest polygons, which can be used to filter out the false positives.*

With the steps mentioned above, we get the x and y positions of a tree, as well as its height h. To get the z position of the tree, swisstopo 3D uses swissALTI3D, sampled at the (x,y) position. For us, the goal is not necessarily that the z position perfectly matches the real-world z position, but that it matches the rendered terrain height. Otherwise, trees might be stuck in the ground or float above it. Floating is the visually more noticeable of the two outcomes and is therefore more important to fix.

An approach to avoid this outcome is to update z at runtime (see Section 3.5.8). Another method is to compute the rendered terrain height at the tree location using the highest terrain LOD. The highest terrain LOD should be rendered when up close, and this is the situation where floating would be the most noticeable. Thus, assuming that the loaded terrain LOD does not lag behind the desired terrain LOD, this method should avoid visible floating artifacts. For the terrain

height computation, we use the height pyramid created by Terrender and look up the height at the nearest pixel given the x and y position. Although nearest-neighbor sampling does not perfectly match the rendered terrain height, as shown in Figure 3.3, it is accurate enough for our use case.

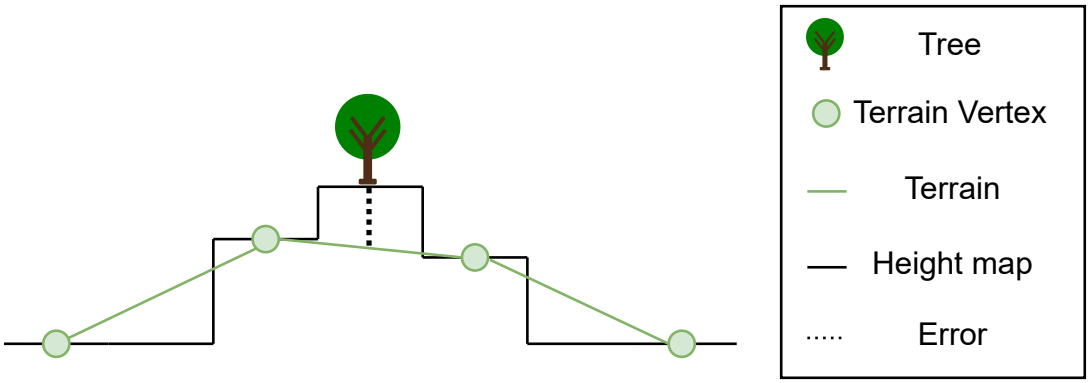


Figure 3.3.: Offline z computation: *The image shows the errors that occur when using offline computed z. The terrain samples the height map using nearest-neighbor sampling at its vertices and linearly interpolates the resulting heights between vertices. In contrast, the offline z computation uses nearest-neighbor sampling of the height map. This difference results in trees being positioned slightly above or below the terrain surface.*

3.2.2. Optional Data

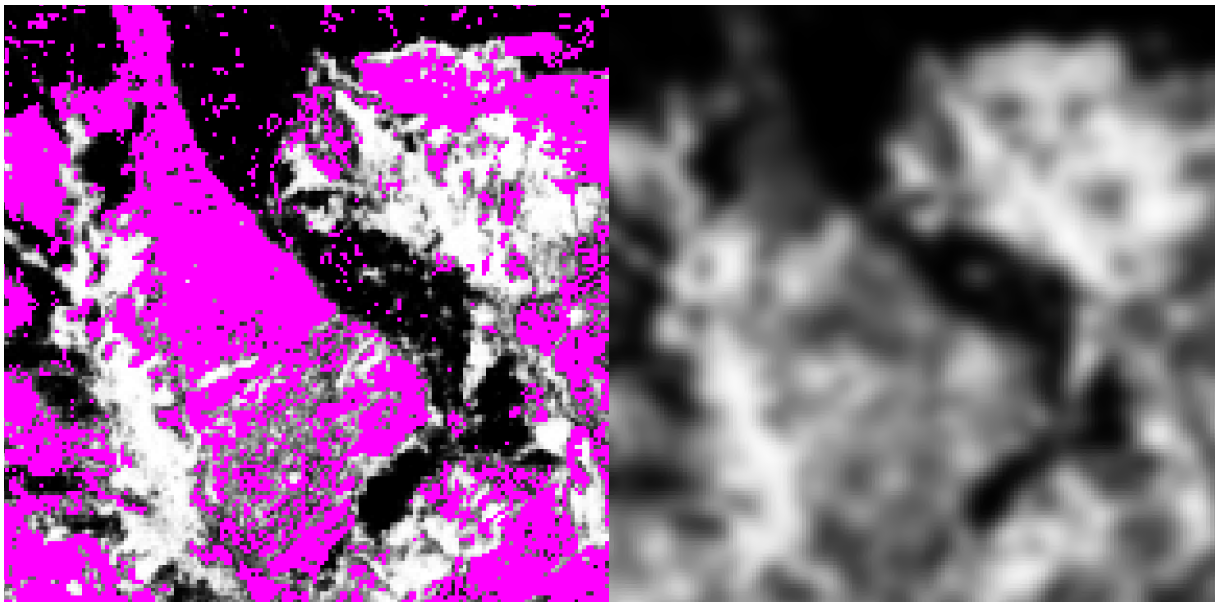
Besides the tree position and height, the WebGPU implementation also supports tree species s, either deciduous or coniferous, as well as a per tree color c, which is used to tint the tree leaves. The tree species is decided using the Forest Mix rate NFI image, where each pixel represents the ratio between deciduous and coniferous trees.

There are cases where an extracted tree is in a pixel without any valid value. A fixed or height-based fallback value for invalid pixels would not represent the underlying tree species distribution. Instead, invalid pixels should be influenced by valid nearby pixels. A naive approach would be to simply blur the image, but this leaves the question of what to set the invalid values to before blurring, e.g., falling back on fixed or height-based heuristics. Instead, the preprocessor first sets all invalid pixels to the value of their closest valid neighbor and then applies a small blur to the result (see Figure 3.4). Without the blur, only the closest valid pixel would have any impact, rather than the broader neighborhood of valid pixels.

The species could be decided using a threshold of 0.5 for each species. However, this decision would not create a mixed forest when close to 0.5, instead always taking the slightly more likely species. Instead, the preprocessor samples the newly created image at the tree position to obtain the ratio between 0 and 1. The ratio is used to decide the tree species by sampling a Bernoulli distribution with the probability corresponding to the sampled ratio.

For the per tree color, we use the color pyramid created by Terrender from swissIMAGE data. The per leaf color should capture the average leaf color and not be influenced by some local lighting effects present in the image. Local lighting effects might be a small highlight or some

leaves being shadowed. To achieve capturing the average leaf color, the preprocessor first blurs the image to remove the high frequency details before sampling.



(a) Original vegetation mix rate **(b)** Processed vegetation mix rate

Figure 3.4.: Processing of Vegetation mix rate: *The original vegetation mix rate is shown on the left, with invalid values shown in pink. The right image shows the result after processing, with invalid values removed.*

3.2.3. Quadtree Building

On the frontend, we want to be able to use a spatial acceleration structure to speed up culling and rendering as previously mentioned. While it is possible to construct such a data structure at runtime, the geometry and position of our trees are static. Thus, we can avoid the runtime cost and create the data structure as another preprocessing step.

We use a quadtree as our data structure due to the z position being given by a height map. This should be a reasonable acceleration structure, as the height map is 2.5 dimensional, i.e., each x,y position maps to exactly one z. Other spatial data structures could be used, such as a BVH. A spatial grid would not benefit from hierarchical culling, as we would need to always iterate over all grid cells and compute culling for each. Tree-based data structures such as BVH and quadtrees allow the frontend to stop once a parent node is culled. A BVH could be used instead of a quadtree, and different splitting heuristics could be interesting further research, as the commonly used surface area heuristic is designed to minimize ray intersection cost and may not suit our use case.

Only leaf nodes contain extracted trees in our quadtree. Each quadtree node stores an axis-aligned bounding box (AABB). We do not explicitly keep track of children indices, as they can be inferred due to the rigidity of a quadtree. Node x, y on level l has children $[(2x, 2y), (2x + 1, 2y), (2x, 2y+1), (2x+1, 2y+1)]$ on level $l+1$ if they exist. If the data is used by the WebGL implementation, the leaf node also stores an average tree position and height.

Algorithm 2 Quadtree creation

```
function CREATE QUADTREE(extent, trees, depth)
    if depth > max depth || size(trees) < min size then
        compute average tree postion, height                ▷ only for WebGL
        AABB ← bounding box(trees, padding)
        return leaf node
    else
        split extents ← split extent in 4
        split trees ← contained in(split extents, trees)
        AABB ← empty box
        for each (e, t) ∈ (split extents, split trees) do
            child ← CREATE QUADTREE(e, t, depth + 1)
            extend AABB by child AABB
        end for
        return node
    end if
end function

CREATE QUADTREE(map size, trees, 0)
```

The quadtree is built top-down with the current extent being divided into four and the trees split between the 4 respective children depending on which quadrant they lie in. The splitting is stopped when either a maximum depth or a minimum number of trees in a node is reached, creating a leaf node. The AABB is computed in a bottom-up manner, with each AABB being a tight fit of the AABBs of its children. Special attention needs to be paid to the AABB in leaf nodes. While the z bound can be computed from the tree's z position p_z and height h, we cannot directly use p_{xy} to compute the x and y extent of the AABB, as the tree models have a non-zero width. To account for this, the x and y bounds of the AABB are padded out with an upper estimate of our tree width. Due to the padding, AABBs on the same level might overlap, trees, however, belong uniquely to one leaf node. See Algorithm 2 for the pseudocode for creating the quadtree.

The frontend might not want to load or render all trees that are present in a leaf node, especially if it is further away. To enable the change in density, we want to split the tree data of leaf nodes into multiple density levels, with each level containing fewer trees than the level before. Each tree is present in only one of the levels. This problem is quite similar to subsampling of point clouds, which is explored in [Schütz et al., 2020]. We support two methods introduced in said paper. The first being uniform subsampling, i.e., choosing completely randomly. The second method being grid-based. The grid-based method first splits the space uniformly into grid cells and then chooses a few trees per cell. Compared to uniform subsampling, the grid-based approach has the benefit of better spatial distribution. Unless specifically mentioned, Forender uses the grid-based downsampling approach.


```
{
    "0": {
        "0_0": {
            "bb": [-0.75, 16000.75, 2383.75, 16379.25, -0.2, 2715.89]
```

```
        }
    },
    ...
    "5": {
        "0_1": {
            "bb": [-0.75, 500.75, 2820.75, 3259.25, 1852.53,
               2179.59],
            "av": [251.75, 2998.38, 2011.77, 18.19],
            "data": {
                "0": [0, 13932, 500],
                "1": [0, 14432, 952],
                "2": [0, 15384, 1896],
                "3": [0, 17280, 3820]
            }
        },
        ...
    }
}
```

Code 3.1: WebGL quadtree example: *An example of a quadtree* `json` *file used by Forender WebGL. The* `bb` *field is the AABB with lower and upper bounds along the x,y and z axes. The first three values of* `av` *contain the average position and the fourth contains the average height. The* `av` *field is not present in quadtree files for WebGPU. The* `data` *field contains the file index, offset and size for each density level.*

The backend storage format of the quadtree is inspired by [Schütz et al., 2020]. We store the computed quadtree as a `json` file. The per tree data is stored in binary, split across multiple files. The split is done to enable the frontend to request data as required without needing to load everything at once. In the `json` file, we store nodes given their level and index in the quadtree. Each node stores its AABB. The leaf nodes also save where and in which file the data is stored per density level, i.e., a file index, an offset, and a size (see Code 3.1). If we want to use the data for WebGL, the leaf nodes additionally store their average tree position and height.

The per tree data is stored as follows. The first 12 bytes, i.e., 3 floats, correspond to the position, the next 4 byte to the height as a float. The optional data is stored as an additional 4 bytes, with the first byte being used for the type and the remaining three being used for color (1 byte per channel). Thus per tree, we need 16 bytes, or 20 bytes if we store the optional data as well.

3.3. Server Backend

The backend serves the website, the 3D models, the quadtree `json` file, and the binary files to the frontend. The WebGL implementation uses a Python HTTP server, while the WebGPU version uses Caddy[6] to serve the content. This decision was made because each backend is already being used in the corresponding Terrender implementation. Caddy is compiled and supports concurrency, unlike Python's HTTP server. Thus, Caddy is likely to be faster at serving

[6] `https://caddyserver.com/` (Accessed: 16.07.2026)

content. This was not benchmarked due to the majority of the served data being terrain data, which is not the focus of the thesis. In the WebGPU implementation, the website is being served by the development server using webpack-dev-server[7] for faster iteration speed. As stated in the description of webpack-dev-server, it should only be used for development, not production, and should be replaced with either a Python server or Caddy in the future. Both backends use their response headers to ensure that the document is cross-origin isolated. Cross-origin isolation is important, as it enables access to accurate time measurement. If not enabled, timer accuracy is reduced to avoid certain cyber attacks such as [Oren et al., 2015].

3.4. Frontend Overview

Forender has two separate frontends with the same overall purpose but different implementations and features. This section provides a brief overview of the two implementations and their supported features as shown in Table 3.1. Each frontend is implemented using a different graphics API. Forender WebGL uses WebGL 2.0, while Forender WebGPU uses WebGPU.

Implementation		
Name	Forender WebGL	Forender WebGPU
Language	JavaScript & TypeScript	Rust (WASM)
Graphics API	WebGL 2.0	WebGPU
Culling		
Frustum culling	Hierarchical	Hierarchical & per instance
Distance culling	Hierarchical	Hierarchical & per instance
Occlusion culling	X	Per instance
Density		
Density streaming	Per leaf node	Per leaf node
Density selection	Per leaf node	Per instance
Density blending	✓	✓
Level of Detail		
LOD selection	Per leaf node	Per instance
LOD metric	Average AABB screen size	Instance AABB screen size
LOD blending	✓	✓
Tree data		
Asynchronous streaming	✓	✓
Tree z position	Offline or runtime	Offline
Optional visual features		
Multiple tree species	X	✓
Leaf tinting	X	✓
SSAO	X	✓

Table 3.1.: Frontend Overview: *Comparison of the supported features and implementations methods of the Forender WebGL and WebGPU frontend.*

[7] `https://github.com/webpack/webpack-dev-server` (Accessed: 16.07.2026)

Both are built on top of Terrender [Croci et al., 2022], an efficient, web-based terrain renderer. It uses a more advanced rendering algorithm, RASTeR [Bösch et al., 2009], when compared to other web-based renderers such as Cesium JS [Cesium GS, 2026], which suffers from seams and discontinuities between LODs. The LOD is decided by combining a distance, a culling and a geometry-based metric. Terrain tiles, fetched from the backend, are decoded on Web Workers to avoid stalling the render thread. Forender uses Terrender for terrain rendering and streaming of color and height texture tiles.

The two implementations share the same overall rendering approach but differ in the granularity at which several operations are performed. Culling is tested hierarchically in both implementations, with Forender WebGPU additionally performing per instance checks. Forender WebGL performs density and LOD selection per leaf node, while Forender WebGPU performs them per instance. Density streaming and blending are implemented identically.

The frontends further differ in their supported features. Runtime z position updates are only available in Forender WebGL, while Forender WebGPU provides additional visual features: multiple tree species, per-tree leaf tinting, and SSAO. The following sections describe each frontend in more detail.

3.5. WebGL Frontend

Forender hooks into the update and render loop of Terrender. The render loop is executed only when new data becomes available or the camera is moved. All terrain streaming and rendering is handled by Terrender, so this section focuses on how the frontend renders large-scale forests in real-time on the Web. An overview of the WebGL implementation is given in Figure 3.5.

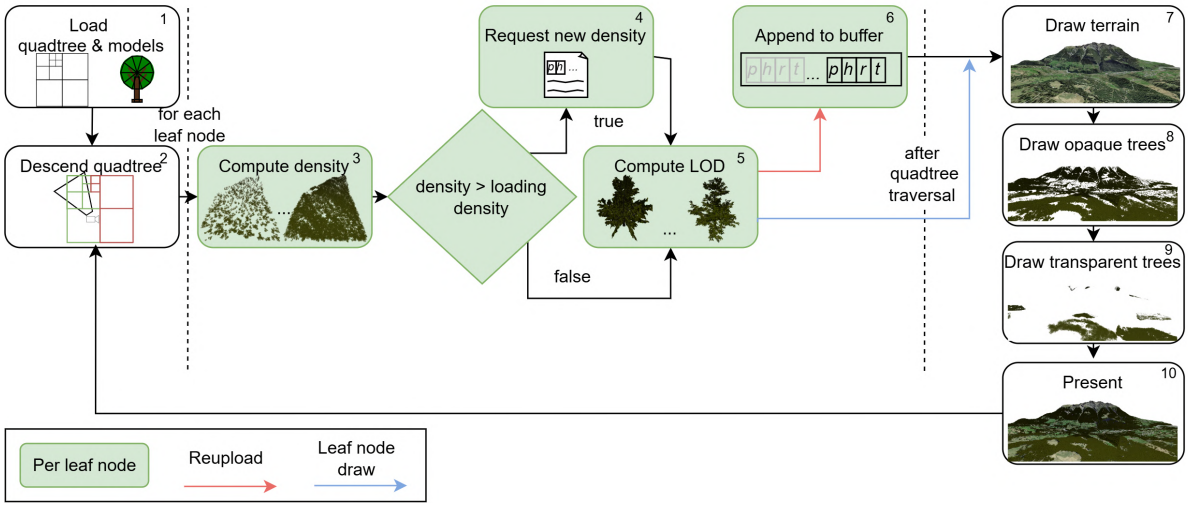


Figure 3.5.: WebGL frontend: *Diagram showing an overview of the WebGL frontend and its two implementations, Reupload and Leaf Node Draw. Reupload fills the per LOD instance buffer each frame on the CPU before uploading them to the GPU. Leaf Node Draw avoids per frame uploads by issuing draw calls per leaf node instead of per model.*

Forender WebGL is largely written in TypeScript, and the rendering uses WebGL 2 via the

twgl[8] library. WebGL 2 was released in 2017 and is supported on all modern browsers[9]. The specification is based on OpenGL ES 3.0 and thus lacks modern concepts such as indirect draw calls and compute shaders. There are two implementations of Forender WebGL, referred to as Reupload and Leaf Node Draw. Both are workarounds for the limitations introduced by WebGL and differ in how instance data is prepared and draw calls submitted to the GPU, with each offering a different trade-off. This motivates the implementation of the WebGPU frontend, which uses compute shaders and indirect draw calls exposed through WebGPU and is discussed in Section 3.6.

To render vegetation, we first descend the quadtree from the root node in a depth-first order. If a node is completely outside the frustum or outside the vegetation specific render distance, we stop descending further from this node. For each leaf node we encounter, we determine the desired density at which the node should be rendered based on the current view. If that density is higher than the currently loading density, we start an asynchronous task to fetch the data from either the cache or the backend. We only ever render the density up to which we finished loading, avoiding rendering unfinished data.

The WebGL implementation also decides the rendered LOD per leaf node. Similarly to culling, this calculation is too expensive to compute per tree and is thus computed per leaf node. The level of detail is decided based on the screen space size of the average tree's bounding box.

The next steps differ depending on the implementation. The Reupload implementation fills instance buffers per frame on the CPU and uploads them after quadtree traversal. The leaf node implementation instead issues draw calls per leaf node using a subsection of a large instance buffer. The leaf node implementation thus has a higher number of draw calls but does not need to upload data every frame.

After drawing the terrain, we draw the trees using instanced draw calls. Opaque trees are drawn first, with transparent trees next. This ordering is required to correctly blend transparent objects with what is behind them.

3.5.1. Startup

On startup, the frontend fetches the quadtree `json` file and 3D models asynchronously (Figure 3.5 step 1). Once a model file has been loaded, we process it into a format suitable for our application. We perform the parsing of the `glb` file using the library gltf-transform[10]. Each `glb` model file can contain multiple materials, each with one or more meshes using it. The meshes describe the 3D data of our model. The vertices store a position, one texture coordinate, a normal and a tangent vector. The position, normal and tangent are stored as `vec3`. The texture coordinate as `vec2`. Each of those vertex attributes is stored in a separate vertex buffer with the mesh having one index buffer. The models are normalized to a height of one world space unit, allowing a single model to represent multiple tree heights. The correct height is then achieved by uniformly scaling the model by the tree height h in the vertex shader.

The material describes how to render and shade the meshes. For an overview of the different

[8] `https://twgljs.org/` (Accessed: 20.07.2026)
[9] `https://caniuse.com/webgl` (Accessed: 17.08.2026)
[10] `https://gltf-transform.dev/` (Accessed: 20.07.2026)

material properties see Table 3.2. How they are used during rendering will be explained in Section 3.5.6. `glb` files store binary data, such as textures, directly in the same file. To upload them to the GPU, we need to decode the binary files from their original format into raw pixel data. The original format is either `jpeg` or `png` and the decoding is done via the browser.

Material properties	
Albedo texture	Texture containing albedo color
Albedo factor	Multiplicative factor applied to sampled albedo color
Alpha mode	Mode on how to use alpha channel. Can be either Opaque or Masked
Alpha cutoff	When to mask in Masked Alpha mode
Metallic roughness texture	Texture containing metallic and roughness
Metallic roughness factor	Multiplicative factor applied to sampled metallic and roughness respectively
Normal texture	Texture containing normal in TBN space
Double sided	Whether material is double sided or if backfaces are culled

Table 3.2.: Overview of material properties: *The listed properties are used in both the WebGL and WebGPU implementation. Although additional parameters are supported in `glTF` / `glb`, they are not used by either frontend. For example alpha mode blend is not supported.*

Once the `json` quadtree file has finished loading, we create the quadtree datastructure. In the frontend, all nodes store the AABB, while the leaf nodes additionally store the average tree. In the Leaf Node Draw implementation, we also allocate the large instance buffer. From that buffer, each leaf node reserves a contiguous subsection the size of the leaf node's instance data at full density.

Once we have finished loading the model for each level of detail and converting them into our representation, we compute the combined bounding box encompassing all our LOD models. We need this computation because the bounding boxes of the same model can differ slightly across LOD levels. When we finished the combined bounding box computation and loaded the entire quadtree, we compute the bounding box of the average tree for all leaf nodes as follows:

$$AvgBB = (min \cdot h + p, max \cdot h + p) \tag{3.1}$$

given the average tree position p, the average tree height h and the combined bounding box (min, max). The resulting bounding box is used for LOD calculations during rendering as described in Section 3.5.4.

3.5.2. Culling

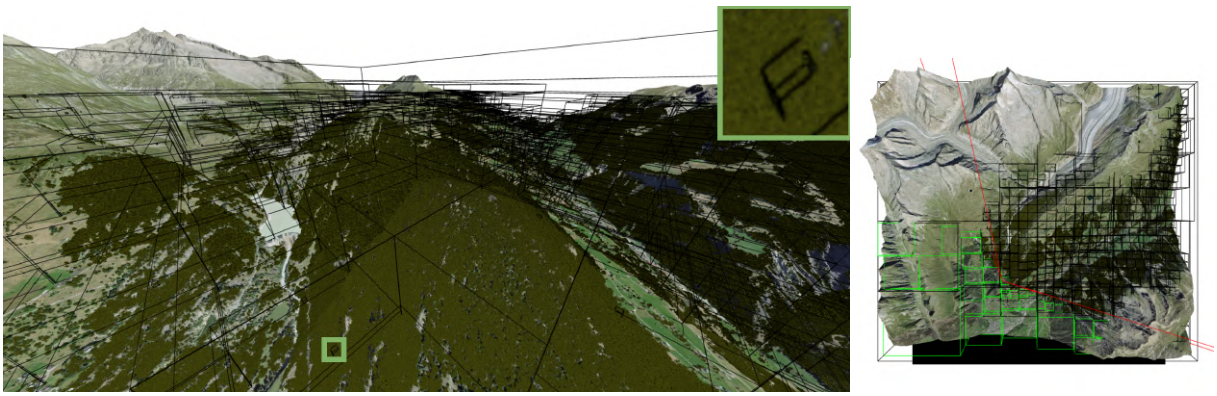
Once startup and terrain loading are complete, we can begin rendering. As mentioned previously, a new frame is initiated by Terrender when either new terrain data finishes uploading or we move the camera. At the beginning of the frame, we perform hierarchical culling using our quadtree similar to [Clark, 1976], i.e., we traverse the visible quadtree nodes (Figure 3.5 step 2).

We start our process from the root node and descend depth-first as long as the current quadtree node is visible. We first perform frustum culling on the node. There are many different methods for frustum culling with different trade-offs. An overview is given in [Akenine-Möller et al., 2018]. We use the following method mentioned in [Akenine-Möller et al., 2018] to test an AABB against a frustum. An axis-aligned bounding box is outside the camera frustum if there

exists one frustum plane where all points lie outside the plane's half-space. We perform this check in the normalized device coordinates (NDC). Using the view and projection matrix, we transform all 8 AABB vertices into clip space and from there into NDC by applying perspective divide. In WebGL, everything within the camera frustum lies inside $[-1, 1] \times [-1, 1] \times [-1, 1]$ in NDC. The pseudocode for the frustum check is shown in Algorithm 3 with an in-engine visualization in Figure 3.6.

Algorithm 3 Frustum check

```
function FRUSTUM CHECK(AABB, view projection matrix)
    NDC points ← transform(AABB, view projection matrix)
    for each dim ∈ {x, y, z} do
        all less ← true
        all greater ← true
        for each p ∈ NDC points do
            all less ← all less && p_dim < −1
            all greater ← all greater && p_dim > 1
        end for
        if all less || all greater then
            return true
        end if
    end for
    return false
end function
```



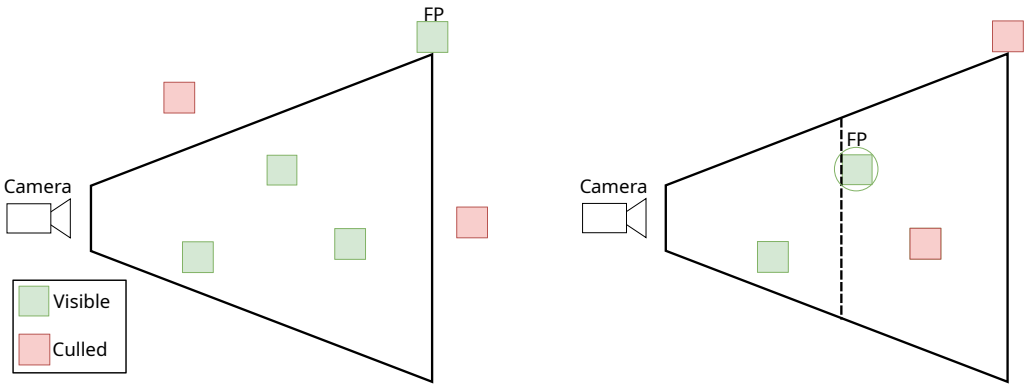
(a) *Camera view* ***(b)*** *External view*

Figure 3.6.: Quadtree frustum culling: *The left image shows the current camera view along with a visualization of the quadtree. The zoomed in square shows the extent of a bounding box of an average tree. The right image shows an external view of the scene, with the green AABB marking where the descent is stopped once a node's AABB is completely outside the camera frustum. Larger boxes indicate that the descent was stopped higher in the quadtree. Note that the black region on the bottom image is due to Terrender WebGL expecting square world sizes.*

This algorithm can lead to false positives, i.e., nodes that are not culled even though they are completely outside the frustum (see Figure 3.7a). Although there are other methods that do not

have false positives, such as using the separating axis theorem, they have a higher cost.

After frustum culling, we perform distance culling. We stop rendering trees earlier than we stop rendering the terrain. This is sensible as the trees are already represented in the terrain color textures and there should not be much of a visual difference. We test if the AABB is fully outside the tree render distance. This is checked by projecting the center of the box into NDC and comparing if the linearized projected depth is further away than the tree render distance plus the AABB circumradius (see Figure 3.7b).



(a) Frustum culling *(b) Distance culling*

Figure 3.7.: Culling visualization: *Visualization of the visibility checks, frustum culling on the left, followed by distance culling on the right. Both checks can suffer from false positives (FP). Frustum culling produces false positives when no view space axis has all vertices outside its half-space. An example is shown in the upper right corner. In distance culling false positives occur when the circumradius extends into the vegetation render distance even though the AABB itself is completely outside it.*

After passing both culling checks, non-leaf nodes descend to their children. In the case of a leaf node, we continue with the next steps, such as computing the density.

3.5.3. Density

To further reduce the number of trees that are rendered, Forender reduces the density of small leaf nodes. Having reached a visible leaf node during the descent, we first determine the density at which we want to render the leaf node (see Figure 3.5 step 3). For this decision, we use the screen size of the node's AABB. We use this metric due to our assumption that reducing the density of a forest patch by a constant factor has less visual impact on smaller patches, either small in world size or due to distance. Due to the varying size of the leaf nodes, this is not equivalent to using distance as a metric.

Computing the pixel accurate screen space size of the bounding box is too expensive. Instead, we estimate it by computing a bounding box of the projected AABB in NDC. The projected AABB has already been computed for culling and can thus be reused.

This value is further reduced based on viewing angle. We reduce density earlier when viewed from above. The strength of this reduction is controlled by view factor, a value between zero and

one, with lower values corresponding to a stronger reduction. The reasoning behind the view-based reduction is that when looking from a top-down view, trees occlude terrain directly below them. Said terrain should have a similar color, as it also contains trees. However, when looking from the side, the trees may occlude either terrain not directly below them or the atmosphere. Both have significantly different colors than the shaded trees, thus, the trees should be kept dense for longer in that case.

To get from the computed value to a discrete density level, we define a list of thresholds between levels and find the index of the largest threshold that is still smaller than the value. The pseudocode of the algorithm is shown in Algorithm 4.

Algorithm 4 Density computation

function DENSITY(NDC points, AABB center, view factor, density thresholds)
  dx $\leftarrow$ `max`(NDC points$_x$) - `min`(NDC points$_x$)
  dy $\leftarrow$ `max`(NDC points$_y$) - `min`(NDC points$_y$)
  view scale $\leftarrow$ 1 - `dot`(world up, `normalize`(camera position - AABB center))
  dv $\leftarrow$ dx $*$ dy $*$ `lerp`(view factor, 1, view scale)
  return `find index`(density thresholds, dv)
end function

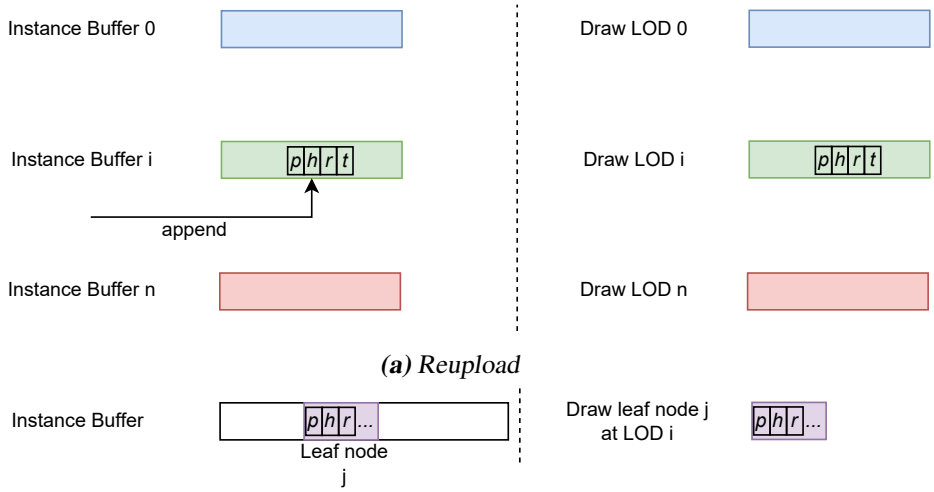
Given our computed density value, we check if everything we want to render has already been loaded or is currently being loaded. If this is not the case, we start an asynchronous task to stream this data in (Figure 3.5 step 4). We `fetch` a new binary file using the file index corresponding to the requested density level. To avoid multiple fetches to the same file, we cache the `Promise` of the `ArrayBuffer` response. Caching the `Promise` instead of the data avoids a second fetch to the same file even if the file is still loading. This implementation makes use of the fact that, in JavaScript, we can `await` the same `Promise` multiple times.

Once the async fetch from either the cache or the backend has finished, we extract the subset of the file corresponding the offset and size of the requested density. With this subset, we update our local CPU instance buffer. In addition to the data stored in the binary file, position p and height h, we also store a random rotation in our CPU buffer. We use this to rotate our model around the up axis in the vertex shader to increase visual variety in the forest. The Leaf Node Draw implementation additionally uploads the new data to the suballocation of the large instance buffer. We render the leaf node at either the desired or the currently loaded density, whichever is smaller.

3.5.4. Level of Detail

After computing the density that we will use to render the leaf node, we decide which level of detail to use (Figure 3.5 step 5). For that decision, we use the screen space size of the average tree bounding box. That bounding box was computed during startup as described in Section 3.5.1. The calculation is similar to Algorithm 4 but without a view-based factor. The final discrete value is computed using a different list of thresholds. Using the screen space size instead of a different metric, such as distance, enables the implementation to use higher LOD earlier in regions with larger trees, at the cost of a more computationally expensive metric.

3.5.5. Instanced Rendering



(a) Reupload

(b) Leaf Node Draw

Figure 3.8.: Instanced drawing implementations: *Comparison of the two WebGL implementations. Reupload fills the per LOD instance buffer each frame and issues instanced draw calls for each material and LOD. Leaf Node Draw uses one big instance buffer and issues draw calls per leaf node and material.*

Once we have finished traversing the visible quadtree, we draw the terrain (Figure 3.5 step 7) as explained in [Croci et al., 2022] and afterwards the trees (Figure 3.5 steps 8 & 9). We make use of instanced rendering to reduce the total draw call overhead. Instead of issuing draw calls per tree, we batch as many trees into as few draw calls as possible. Instanced rendering enables us to draw multiple copies of the same mesh with instance attributes that vary between drawn tree copies. Our instance buffers contain the following attributes: position p, height h, and rotation r in radians. The Reupload implementation also stores transparency in the instance buffer (see Section 3.5.7).

The batching is limited by different materials requiring different configuration states and by different LODs using different meshes and thus requiring separate draw calls. Batching could be improved by using the `WEBGL_multi_draw` extension, which allows rendering multiple mesh primitives in a single draw call. However, Firefox currently does not support this extension[11].

The instance buffers need to be tightly packed without any invalid instance data. Invalid instances would still be rendered, as WebGL cannot discard invalid instances on the GPU due to lack of compute or geometry shader support. The above restrictions lead to two different implementations that both support instanced rendering with multiple LODs in WebGL.

The Reupload implementation uses two instance buffers per level of detail. One for opaque trees and one for transparent trees. These instance buffers are filled on the CPU during quadtree traversal (Figure 3.5 step 6) and uploaded to the GPU afterwards. The buffers are created with usage `DYNAMIC_DRAW` for faster uploading. For each LOD, we issue one draw call per material present in that LOD, as shown in Figure 3.8a.

[11] `https://developer.mozilla.org/en-US/docs/Web/API/WEBGL_multi_draw` (Accessed: 27.07.2026)

The Leaf Node Draw implementation instead uses one large instance buffer with the usage `DYNAMIC_DRAW`. Each leaf node has a subsection that it uploads its instance data into once the data has been streamed in. Each leaf node uses either one LOD or two when blending between them. The frontend issues one draw call per material of the LOD with the corresponding subsection of the instance buffer bound (see Figure 3.8b). Transparency is constant within one draw call and is thus passed as a uniform, not an instance attribute. The implementations trade different sources of overhead to work around WebGL's limitations. Reupload has a reduced number of draw calls at the cost of uploading instance data each frame. Leaf Node Draw avoids these per frame upload but issues more draw calls.

3.5.6. Shading

Shading is implemented based on the Disney BRDF [Burley, 2012] with support for the parameters base color / albedo, metallic and roughness. We use a fixed IOR of 1.5. The albedo represents the surface color of the material, while roughness controls how rough the material looks. The metallic parameter interpolates between a dielectric and metallic surface model. Fully metallic materials do not have a diffuse component and look dark due to not supporting image-based lighting or global illumination techniques.

The normals used for shading are sampled from a normal map. As the normal map is in tangent space, the shader needs to either transform light and view direction to tangent space or the normal to world space. For the transformation, we compute the TBN frame from the vertex normals and tangents. Our shading model does not support proper translucency for leaves. As a cheap proxy for this effect, we flip the normals in the leaf material if we look at the leaf from behind.

Foliage is geometrically complex. To represent that complexity, artists often use alpha masked textures instead of actual geometry. Alpha masked materials discard fragments that have an alpha value smaller than the alpha cutoff. The opaque alpha mode does not discard and outputs opaque fragments. The blend alpha mode blends fragments with their background given their alpha. The different modes are visualized in Figure 3.9.

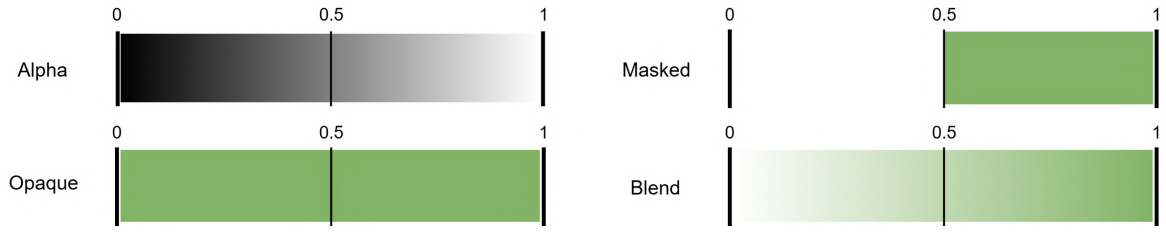


Figure 3.9.: Alpha modes: *Image comparing the effect of different alpha modes when rendering a green object on top of a white background. The alpha of the object varies linearly from 0 to 1 going left to right. For the alpha mode masked, the alpha cutoff is 0.5.*

Alpha mode blend is used for transparent materials. As none of our tree models use materials with alpha mode blend we do not support it as a material property. Note that the whole tree might be transparent as will be explained in Section 3.5.7. Our implementation uses MSAA for anti-aliasing with 4 samples.

3.5.7. Blending

With the aforementioned steps, we can already visualize forest scenes. However, as we discussed in Section 2.2.2, changes in LOD can be quite visually distracting. Additionally, due to deciding LOD per leaf node, we have visible seams at the boundary between leaf nodes where the LOD changes. In order to reduce these effects, we blend between LODs as in [Giegl and Wimmer, 2007]. When close to a threshold, the method first blends in the new LOD while keeping the old one opaque. Once both are fully opaque, the old LOD is blended out as shown in Figure 3.10. This approach avoids making objects behind them visible, which would be the case if both were blended at the same time.

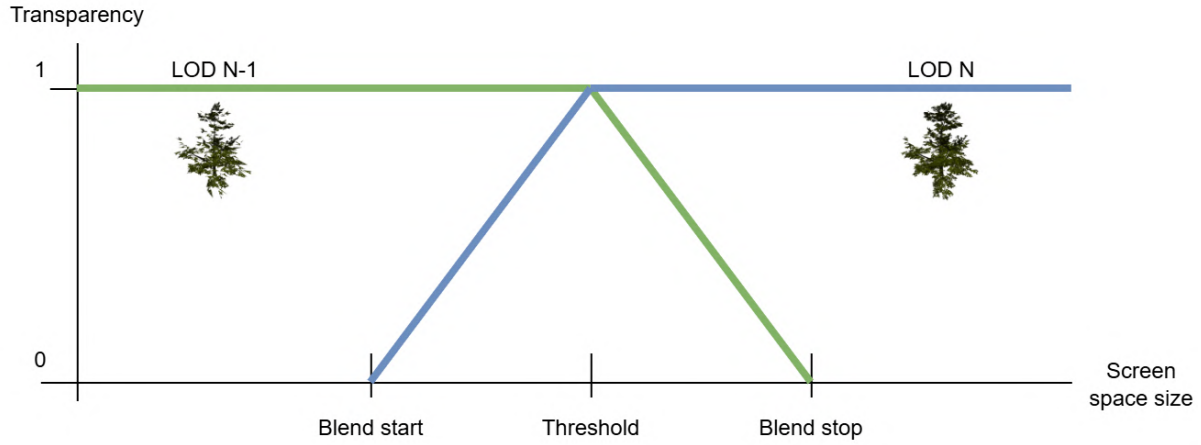


Figure 3.10.: LOD Blending: *Illustrates blending as explained in [Giegl and Wimmer, 2007]. First we blend in the new LOD. Once the new LOD is fully opaque, the old LOD is blended out. This approach avoids making the background visible if both were be blended simulatenously.*

Another source of visual popping between frames occurs when we change density. To avoid this artifact, we blend in a new density once the threshold is crossed. This transparency is only applied to the new density and is not used while the loaded density is lagging behind the desired density.

Both of these blending methods are based on transparency. One commonly used definition of transparency is given by Porter and Duff [Porter and Duff, 1984]. It states that the final color of a pixel C_f given a background surface 0 and a foreground surface 1 is as follows:

$$C_f = C_1 + (1 - \alpha_1)C_0 \tag{3.2}$$

with C_i being the surface color of i premultiplied by its alpha α_i. This corresponds to the OVER operator introduced in [Porter and Duff, 1984]. This operator is not commutative and thus requires sorting of geometry for correct results. However, sorting large amounts of geometry each frame tends to be expensive.

Instead, the result of this operation can be approximated by methods that do not rely on sorting. A simple implementation would be screen door transparency, which discards a certain percentage of pixels given its transparency. The high-frequency patterns introduced by screen

door transparency degrade the visual quality. Screen door transparency is used in Cesium for Unreal[12] for LOD transitions, but Cesium relies on temporal anti-aliasing (TAA) to reduce high-frequency artifacts introduced by the method. As TAA is not already implemented in Terrender, we instead use a different order-independent transparency solution: *Weight Blended Order-Independent Transparency* [McGuire and Bavoil, 2013]. It approximates the operator over n surfaces as follows:

$$C_f = \frac{\sum_{i=1}^{n} C_i \cdot w(z_i, \alpha_i)}{\sum_{i=1}^{n} \alpha_i \cdot w(z_i, \alpha_i)} (1 - \prod_{i=1}^{n} (1 - \alpha_i)) + C_0 \prod_{i=1}^{n} (1 - \alpha_i) \tag{3.3}$$

$$w(z, \alpha) = \alpha \cdot clamp(\frac{10}{10^{-5} + (z \cdot 100)^2 + (z \cdot 2.5)^6}, 10^{-2}, 3 \cdot 10^3) \tag{3.4}$$

where w is a non-zero, monotonically decreasing function given the linear depth of the surface fragment. The w used with its corresponding constants is suggested in [McGuire and Bavoil, 2013]. In Equation (3.4) linear z is assumed to be normalized and in the range $[0, 1]$. The numerator (Equation (3.3) blue) and denominator (Equation (3.3) green) are stored in a single `RGBA16f` texture. The RGB channels store to the numerator, while the alpha channel stores the denominator. The product (Equation (3.3) red) is stored in a `R16f` texture. We perform the accumulation in a separate render pass, using the two textures as render attachments. Each attachment uses a different hardware blending configuration. We resolve the transparent accumulation textures as a post-processing step, combining the transparent trees with the opaque background C_0.

3.5.8. Z Position Updates

Although our binary tree data already contains a z position, this static position might not match the rendered terrain surface. This discrepancy arises either from error introduced during offline height sampling or from terrain loading lagging behind. To reduce this difference, Forender WebGL can optionally update the z position of the trees to match the terrain height.

Terrender WebGL exposes a function, `getHeightValue`, which allows the currently rendered terrain height to be sampled. It bilinearly samples the terrain height tile currently being used. Note that even the sampled height values might slightly differ from the actual terrain height shown, due to also not considering the terrain mesh resolution. Forender WebGL uses this function to update the tree's z position.

The z update is implemented as an asynchronous function executed when the terrain texture updates. The asynchronous function calls `yield` after a fixed number of z updates. Forender WebGL uses `yield` to avoid using too much frame time and blocking other functions, as the function is executed on the main thread. Ideally, the function would be executed on a

[12] `https://cesium.com/blog/2022/10/20/smoother-lod-transitions-in-cesium-for-unreal/` (Accessed: 27.07.2026)

separate thread. However, Web Workers do not support sharing all memory with the main thread, which would make this change difficult to integrate with the existing codebase. Due to being asynchronous, the update might lag behind the terrain height being shown.

3.6. WebGPU Frontend

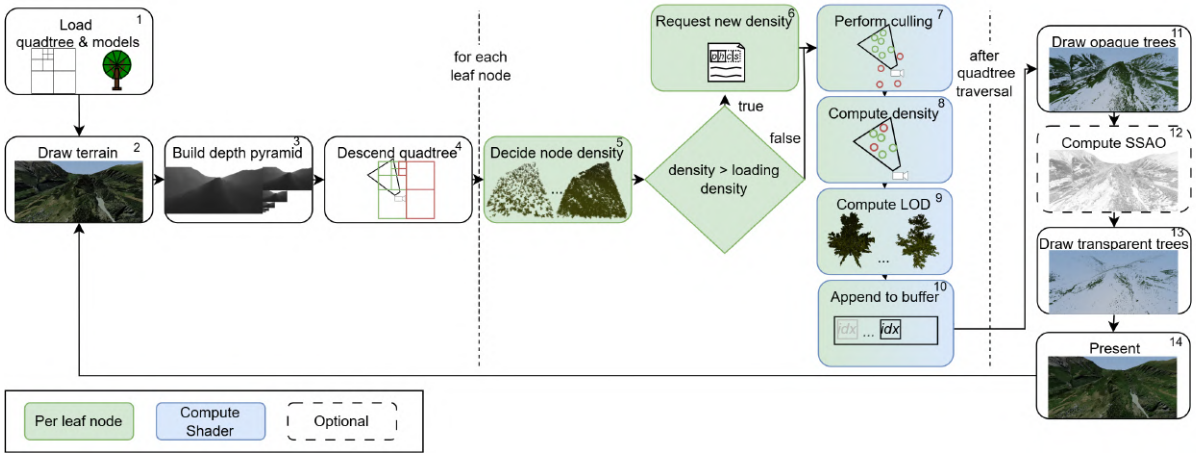


Figure 3.11.: WebGPU frontend: *Diagram showing an overview of the WebGPU frontend. Terrain is rendered before descending the visible nodes of the quadtree. The terrain depth is used to build a depth pyramid for occlusion culling. In contrast to WebGL, we additionally perform culling and density selection per tree instance using compute shaders. The LOD selection is also computed on a per instance basis in a compute shader. The optional SSAO step can be used to improve the shading of the scene. The SSAO parameters are exaggerated to emphasize the effect.*

As mentioned previously, the WebGL specification is based on the OpenGL ES 3.0 specification released in 2012[13]. The old OpenGL ES 3.0 specification does not match the underlying hardware design of modern GPUs and thus imposes some unnecessary limitations. These limitations require our implementation to use workarounds that have different trade-offs as discussed in Section 3.5. If compute shaders and storage buffers were available in WebGL, then these workarounds would not be necessary.

WebGPU was released in 2023 as a new graphics API for the Web[14]. The API exposes some modern features found in the desktop graphics APIs, such as compute shaders. By now, WebGPU is widely supported, although some browsers do not enable it by default. One notable browser that still lacks support is Firefox on Linux, where WebGPU support is expected to ship in 2026[15].

Forender WebGPU is similar to its WebGL counterpart but makes use of compute shaders and GPU-driven rendering, i.e., generating rendering commands on the GPU. Compute shader

[13] `https://www.khronos.org/news/press/khronos-releases-opengl-es-3.0-specification` (Accessed: 14.08.2026)

[14] `https://developer.chrome.com/blog/webgpu-release/` (Accessed: 14.08.2026)

[15] `https://github.com/gpuweb/gpuweb/wiki/Implementation-Status` (Accessed: 30.07.2026)

support enables Forender to perform GPU occlusion culling and other additional per instance culling. Finer granularity in culling leads to fewer instances being rendered. The increased performance enables adding new features to the Forender WebGPU frontend such as multiple tree species, greater color variation between trees due to leaf tinting and *Screen space ambient occlusion* (SSAO), for greater visual fidelity.

An overview of the WebGPU frontend is shown in Figure 3.11. After loading our data from the backend, we start our frame by rendering the terrain and building a depth pyramid from it. The depth pyramid will be used for occlusion culling. As is the case in the WebGL implementation, we descend the visible quadtree, compute the per leaf node density, and stream in additional density data if necessary. Afterwards, we invoke a compute shader per leaf node. Due to running on the GPU, Forender is now able to perform some per instance computations that were too expensive on the CPU. The compute shader performs additional, per instance culling and density reduction. After culling, Forender computes the LOD per instance in the compute shader. The tree instance is then appended to an instance index buffer. Once quadtree traversal is finished, we render the opaque trees followed by the transparent ones. We optionally compute an SSAO texture from the terrain and opaque trees.

Forender WebGPU is written in Rust and compiled to WebAssembly (WASM) using wasm-pack[16]. For rendering, Forender WebGPU uses the WGPU library[17] with the WebGPU backend. Forender WebGPU uses Terrender WebGPU for terrain rendering. Terrender WebGPU supports all features of its WebGL counterpart with some additional features such as tone mapping and atmosphere rendering.

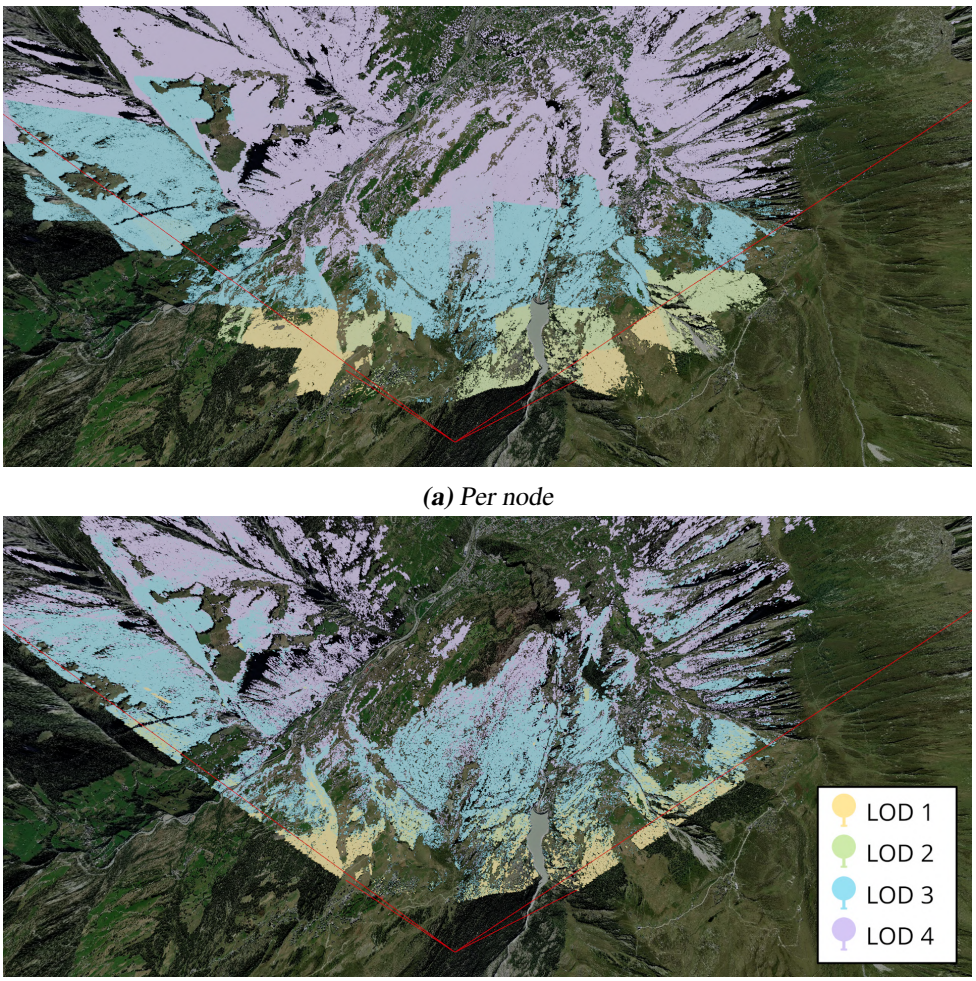
3.6.1. Density

Density streaming is handled the same as in the WebGL implementation. Note that in the WebGPU implementation, the density we compute here is only used to progressively stream in our data. Reduction in density is now handled in the compute shader, discussed in Section 3.6.2. The implementation of streaming new density data requires special attention due to Rust's ownership semantics and how `async` functions work in Rust. In JavaScript, an async function that has been called will eventually be executed even without calling `await`. In contrast, Rust only begins execution once `await` is called on the `Future`. JavaScript's eager execution behavior is replicated through `spawn_local` in the wasm_bindgen_futures crate[18]. In Rust, we cannot `await` the same `Future` multiple times, which is behavior we rely on for caching binary file fetches. Instead, we use the async_once_cell crate[19] to cache resolved fetches.

In order to avoid multiple mutable references existing at the same time when streaming in data, the implementation uses type `Rc<RefCell<T>>` for a mutable reference of type `T`. This implementation is safe and will not panic at runtime because we do not have overlapping mutable accesses.

[16] `https://wasm-bindgen.github.io/wasm-pack/` (Accessed: 30.07.2026)
[17] `https://wgpu.rs/` (Accessed: 30.07.2026)
[18] `https://docs.rs/wasm-bindgen-futures/latest/wasm_bindgen_futures/index.html` (Accessed: 31.07.2026)
[19] `https://docs.rs/async-once-cell/latest/async_once_cell/` (Accessed: 31.07.2026)

3.6.2. GPU-Driven Rendering



(a) *Per node*

(b) *Per instance*

Figure 3.12.: Per instance decisions: *An in-engine visualization comparing the per node and per instance decisions for visibility and LOD selection. The per instance variant also performs occlusion culling which is not implemented per node. The camera frustum is shown in red. Note that this image uses modified LOD thresholds to make the differences more visible.*

GPU-driven rendering allows Forender WebGPU to perform our visibility checks and LOD computations per instance in addition to hierarchical culling. Per instance visibility checks increase the granularity of culling, and per instance LOD reduces visible artifacts due to LOD transitions. The difference between these approaches is shown in Figure 3.12. Executing the checks on the massively parallel GPU makes the per instance checks feasible.

GPU-driven rendering is implemented as follows. After density computation, we dispatch one compute shader per leaf node covering all currently loaded instances (see Figure 3.11 steps 7-10). Similarly to the node draw implementation in WebGL, all instance data is stored in one large buffer, with each leaf node having a reserved subsection. Each thread corresponds to a single tree instance with instance index = thread index + leaf node offset, with the leaf node offset being the index of the leaf's subregion. The instance data buffer stores the position

p, height h, and rotation r. Optionally, the instance data also stores the leaf color c and tree species s.

We first determine if the tree is visible and then compute the LOD. The first visibility check is frustum culling using the combined AABB rotated around the up vector by r and uniformly scaled by h. The combined bounding box is the AABB that encompasses all tree models across all LODs. We use the same criterion as described in Section 3.5.2 to check the transformed AABB against the camera frustum.

Next, the threads perform distance culling on the transformed instance bounding box. The check is implemented the same as the per node check.

The next step is density reduction and is implemented as follows. Each tree stops rendering once its screen space size falls below a random cutoff. The cutoff differs between instances, ranging from 3 to 10 pixels, and is computed based on the random tree rotation. We use a random range instead of a fixed cutoff to gradually reduce density. To avoid popping, we begin fading trees out once their screen space size gets close to their cutoff. This method of density reduction is similar to uniform subsampling, as it also has poor spatial distribution when compared to grid-based subsampling (see Section 3.2.3). However, the method keeps rendering larger trees longer than their smaller counterparts, which was not taken into account by the other methods.

The last visibility check is occlusion culling, which is described separately in section Section 3.6.3. If a tree passes all visibility checks, we compute the LOD to be used. This uses the screen space metric on the transformed bounding box and performs blending between LODs based on [Giegl and Wimmer, 2007].

The implementation uses a layer of indirection in order to avoid unnecessarily copying instance data. After LOD selection, we fill up an instance index buffer, whose entries point into the instance data buffer. When issuing a draw call from the CPU using `draw_indexed`, we pass `instance_count` as an argument. As this varies from frame to frame and is stored on the GPU, this would require a read back to the CPU. Indirect rendering avoids this cost by storing the arguments to the draw call in a GPU buffer.

Combining the above steps, the complete pipeline is shown in Figure 3.13 and works as follows. At the beginning of our frame, we reset the indirect argument buffer to an `instance_count` of zero. Next, we fetch our instance data given our instance index. Then we perform our visibility checks and compute the LOD. Let us assume the computed LOD is i. Afterwards, we atomically increment the `instance_count` argument in the indirect argument buffer. The buffer is indexed according to LOD i, tree species s, and whether the tree to be drawn is opaque $o \in \{true, false\}$. Finally, we append our instance index to either the opaque or transparent instance index buffer. The region used is given by the LOD i and species s. Within the region, we write our instance index into the position of the `instance_count` before incrementing. The transparent instance index buffer regions store the index as well as the corresponding transparency.

One model may contain multiple meshes, each requiring a separate indirect argument. We update only the first mesh's `instance_count`. Once we finish adding instances, the remaining meshes copy the `instance_count` from the first mesh.

When rendering, we access the instance index buffer using `@builtin(instance_index)`.

The instance data is then accessed using the retrieved index. In order to minimize pipeline switches, the draw calls are sorted by material. In contrast to the WebGL implementation, we use a separate shader for alpha masked materials instead of using a runtime check.

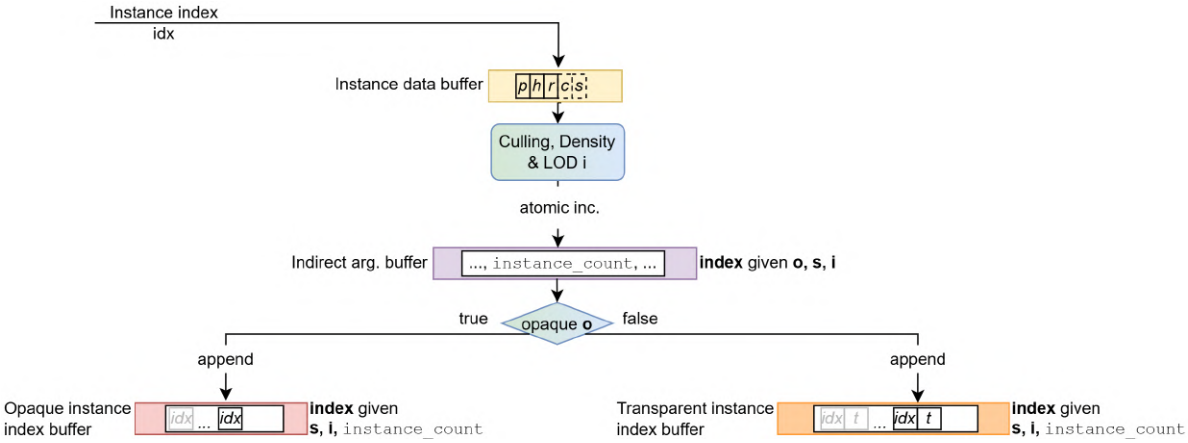


Figure 3.13.: GPU-driven rendering: *Diagram illustrating GPU-driven rendering and buffer usage. Instance data holds per tree data, which is accessed in the vertex shader using the instance index buffer. Each model (LOD i, species s) has two fixed regions in this buffer, one for opaque and one for transparent instances. In contrast, the entries in indirect argument buffer are stored per mesh. If an instance passes all visibility checks, it atomically increments the corresponding* `instance_count` *and appends its index into the instance index buffer.*

Together, these steps enable finer culling and LOD decisions, avoiding the workarounds necessary in WebGL.

3.6.3. Occlusion Culling

GPU-driven rendering enables Forender WebGPU to implement GPU-based occlusion culling. Occlusion culling further reduces the number of rendered trees by removing tree instances that are inside the frustum but hidden completely behind other objects. Although occlusion culling is technically possible in WebGL, such an implementation would rely on occlusion queries, which are difficult to use correctly and efficiently. Additionally, this implementation would require a roundtrip from the GPU to the CPU. Thus, Forender WebGL does not support occlusion culling.

We assume that terrain is the major occluder in these scenes. Trees are porous, i.e., contain many gaps, making them bad occluders. Trees are therefore not considered as occluders. The terrain is rendered at the beginning of a frame (see Figure 3.11 step 2). Afterwards, we generate a depth pyramid from the rendered terrain depth (Figure 3.11 step 3). Each new layer of the pyramid has half the resolution of the previous layer and is calculated using maximum for downsampling. Using the maximum operation, each depth value is a conservative estimate of the distance to the furthest visible fragment that corresponds to that pixel. As MSAA is used for anti-aliasing, the implementation has to first resolve the MSAA texture. While other APIs expose depth resolve within a render pass, WebGPU only allows color resolves. Thus, the depth resolve is done explicitly when creating the first layer of the depth pyramid by taking the maximum of all MSAA depth samples.

The per instance occlusion check then uses the created depth pyramid as follows. First, we compute the mip level λ to sample given the extent of the projected transformed AABB as described in Equation (3.5):

$$\lambda = \min(\lceil \log_2(\max(l, 1)) \rceil, n - 1) \tag{3.5}$$

where l is the maximum length of the 2D extent and n is the number of mip levels. The formula guarantees that all 4 corners of the projected AABB map to a 2x2 region in the mip level. Thus, we only need to sample the depth pyramid 4 times at the pixel coordinates of the corners of the extent and compare these values with the minimal depth of the projected AABB. This comparison checks for occlusion by the terrain as shown in Figure 3.14.

This test allows Forender to perform occlusion culling on the GPU in addition to the previously discussed visibility tests, further reducing the number of rendered trees without requiring a CPU roundtrip.

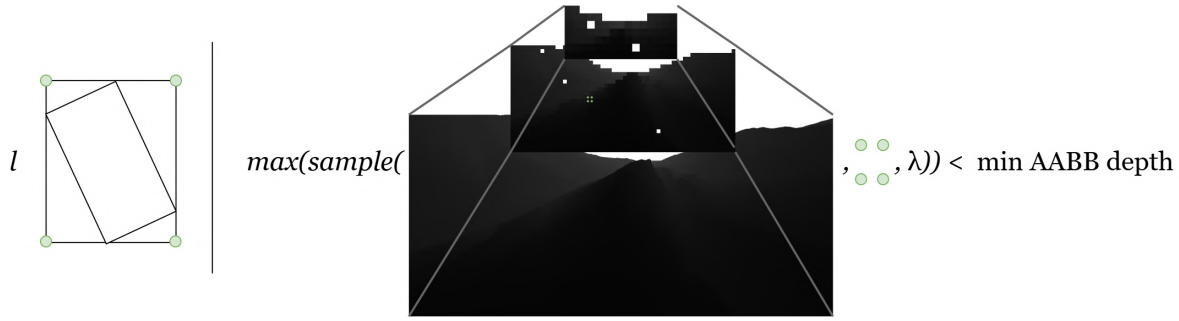


Figure 3.14.: Occlusion culling test: *The occlusion test is performed by comparing the pixels corresponding to the four corners of the projected AABB extent at pyramid depth λ with the minimum depth of the projected AABB. The white holes in the terrain are due to floating-point precision issues.*

3.6.4. Leaf Tinting

Forender has low visual variation between trees, even when using two tree species. There is also a notable difference in color between the 3D model and the underlying terrain, which makes the density reduction and distance culling more noticeable. Both of these issues can be improved in Forender WebGPU by optionally tinting tree leaves to the tree color c present in the terrain orthoimages.

Only materials with masked alpha mode are tinted in order to avoid changing the bark color. The textures used in the alpha masked materials can still contain non-leaf objects such as branches, thus requiring some sort of masking. Ideally, this mask would be created by an artist, but for Forender, we compute it at runtime. The runtime method is used to support arbitrary tree models without manual preprocessing steps.

We assume that the albedo textures of the alpha masked materials consist mostly of leaves, all having a similar color. Thus, the mask can be seen as removing outliers in the color space.

Forender WebGPU only tints fragments whose color distance from the mean is within k standard deviations, with k empirically chosen.

These distances are computed in the Oklab color space using the Euclidean distance. Oklab is used because it is perceptually linear[20], unlike RGB, i.e., equal Euclidean distances in Oklab correspond to equal perceived color distances to the human eye. Both the albedo color mean and standard deviation are computed as a preprocessing step.

Tinting is performed by linearly interpolating between the sampled albedo and the tint color c. Tinting uses a fixed weight and is only applied within the computed mask.

3.6.5. Alpha to Coverage

Forender makes heavy use of alpha masked materials, which can suffer from noise even with MSAA. As mentioned in Section 3.5.6, Forender WebGL discards fragments below a certain alpha cutoff. MSAA on its own does not anti-alias edges that arise from discarding and not actual geometry. These edges are not anti-aliased because MSAA only executes the fragment shader once per triangle per pixel and not per sample. Forender WebGPU instead makes use of *Alpha to Coverage* (A2C), which maps the alpha value of the fragment to the coverage mask, limiting how many samples can contribute to the final color. If alpha is below the alpha cutoff value, we set the alpha to zero, which causes no samples to be written. The masked edge will be anti-aliased as the implementation outputs the sampled alpha value if above the cutoff.

Both discard and A2C suffer from the issue that alpha masked materials disappear when seen from further away. This phenomenon arises due to the way we compute mipmaps. When computing the next layer, we downsample by averaging. The ratio between pixels passing the alpha test and the total number of pixels goes down across mip levels due to averaging. The decrease in the passing pixel ratio causes the material to disappear at a distance, as shown in Figure A.6. Forender WebGL reduces the alpha cutoff as a workaround, leading to fewer discarded pixels up close than intended by the artist. Forender WebGPU instead compensates for the loss by scaling the alpha value depending on the mip level as explained by Ben Golus[21]. Note that A2C is only implemented in Forender WebGPU, not due to technical or performance limitations but due to time constraints.

3.6.6. SSAO

Forender WebGPU has a simple shading model that lacks both dynamic shadows and global illumination. It can be hard to judge the depth, and all areas of the tree look visually similar to each other. Full global illumination is outside the scope of this thesis. Instead, ambient occlusion is used to darken areas less exposed to ambient light. Ambient occlusion darkens, for example, areas of the tree further within the canopy, as well as the ground beneath the trees. Per model ambient occlusion could be baked offline. However, scaling this to the complete scene

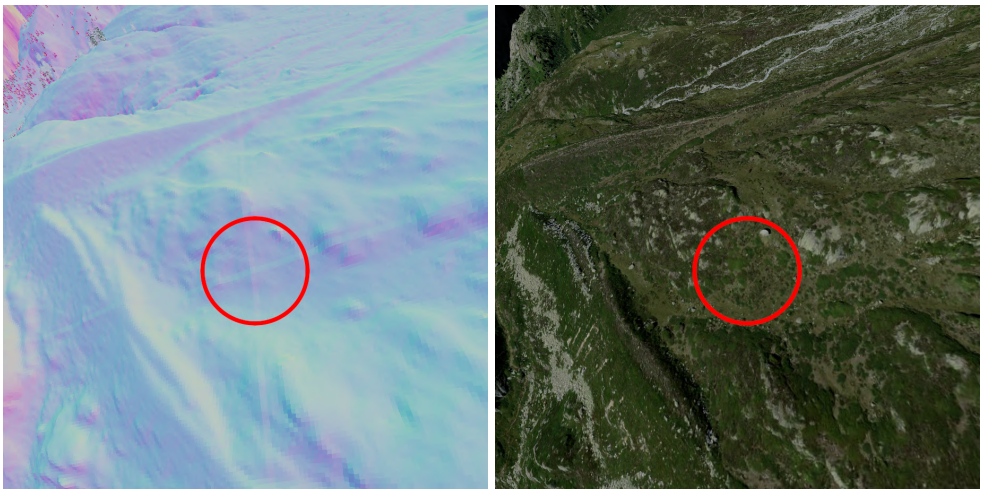
[20] `https://www.w3.org/Graphics/Color/Workshop/slides/talk/lilley` (Accessed: 05.08.2026)

[21] `https://bgolus.medium.com/anti-aliased-alpha-test-the-esoteric-alpha-to-coverage-8b177335ae4f` (Accessed: 04.08.2026)

across all terrain LODs would be very expensive. Instead, Forender WebGPU relies on *Screen space ambient occlusion* (SSAO), a dynamic but efficient runtime approach that computes ambient occlusion in screen space.

Forender implements SSAO (Figure 3.11 step 12) based on the description from John Chapman[22]. The SSAO texture is computed after the opaque trees have been rendered. Chapman's method generates samples in a hemisphere oriented along the normal and compares the scene depth with the sample's depth to estimate how occluded a pixel is. The samples are randomly rotated around the normal to avoid banding.

When rendering the terrain and the opaque trees, the fragment shader writes into a color and normal texture attachment. The 3D models output their normal directly. However, the terrain does not have a normal map. Instead, Forender estimates the normals using central difference. Note that forward and backward difference would introduce a bias into the estimated normal due to their asymmetric nature. The normals are estimated every frame, as different tiles might be streamed in at any moment, and storing the normals explicitly would slow down terrain streaming and updating. Due to the tile-based nature of the terrain, there are small discontinuities in the estimated normals between tiles, as shown in Figure 3.15a. The discontinuities are not visible in the final shaded output due to the error being small and the SSAO texture being blurred, as explained next (see Figure 3.15b).



(a) *Normal* ***(b)*** *Shaded*

Figure 3.15.: Terrain normal discontinuity: *On the left we visualize the terrain normals estimated using central difference. Due to the tile-based nature of the terrain, there are discontinuities at the tile borders. These are not visible in the shaded output with SSAO applied, as the error compared to the ground-truth normals is small and the SSAO texture is blurred before being applied. The low resolution of the estimated normals is due to the height being sampled using nearest-neighbor filtering.*

Due to the randomness of the rotation, the resulting SSAO texture is noisy as shown in Figure 3.16. Forender WebGPU first blurs the SSAO texture (see Figure 3.16b) before applying it to the final color C_f after resolving transparency. Transparent objects do not contribute to

[22] `https://john-chapman-graphics.blogspot.com/2013/01/ssao-tutorial.html` (Accessed: 05.08.2026)

the SSAO texture. However, if we were to only apply SSAO to opaque objects, there would be noticeable brightness changes during LOD transitions. Thus, the method darkens both opaque and transparent objects. During LOD transitions, each transparent LOD has an opaque model of the same tree, but of a different LOD, behind it (see Section 3.5.7). The SSAO computed on the opaque geometry therefore closely approximates how ambient occlusion would appear on the transparent LOD. However, when the transparency in the model is due to a reduction in density, this is not the case. As density reduction only applies to trees that occupy a small screen space area, there is little perceptual impact from the incorrect SSAO contribution.



(a) Noisy SSAO *(b) Blurred SSAO*

Figure 3.16.: SSAO noise: *SSAO noise arises due to the random rotation of the samples. The randomness uses a 2x2 noise texture. SSAO is blurred using a box filter before being applied.*

4

Results

This chapter focuses on the experimental evaluation of Forender. The main goal of this chapter is to assess that Forender can achieve real-time performance on large scenes while still having a realistic visual appearance.

We will first discuss the setup used for our evaluation in Section 4.1, specifying the hard- and software stack used, as well as which scenes our tests are performed on. Next, we evaluate multiple quantitative experiments in Section 4.2, with a major focus on the runtime performance of Forender. We compare multiple implementations and configurations and verify the efficiency of the optimizations implemented. Particular emphasis is placed on the differences between WebGL and WebGPU implementations, as WebGPU avoids expensive workarounds and enables new GPU-based optimizations. Finally, in Section 4.3, we evaluate the visual quality of Forender with a focus on the increased diversity and complexity provided by the optional features in Forender WebGPU.

4.1. Evaluation Setup

All experiments were conducted on a laptop running Windows 11 (version 10.0.26200). It has an Intel i7-11800H CPU and two GPUs: an *Intel(R) UHD Graphics* integrated GPU and a *NVIDIA GeForce RTX 3060 laptop GPU* with 6 GB VRAM. The IGPU driver used is on version 30.0.101.2079 and the NVIDIA driver is on version 32.0.15.9282. Unless specifically mentioned, all experiments use the NVIDIA GPU. The laptop has 16 GB DDR4 RAM. Both the frontend and the backend run on the aforementioned laptop.

Forender WebGL is written in TypeScript and is compiled using tsc version 5.9.3 targeting ES-Next. Forender WebGPU uses rustc version 1.94.1 with `opt-level=3` to produce its WASM modules. The experiments were executed on the Chrome Browser version 150.0.7871.187

with V-Sync enabled. The JavaScript is interpreted using the JavaScript Engine V8 version 15.1.206.16.

For completeness, Forender was also tested to run on an Apple M3 laptop, however, the performance was not rigorously benchmarked on this device. During testing, performance was worse, dropping to 20 FPS in the Valais scene on Chrome. This was notably slower than on the Windows laptop that was used during development. The difference can be plausibly explained by the performance of the respective GPUs. The M3 10-Core GPU[1] scores considerably lower on public benchmarks than the RTX 3060 Laptop GPU[2] used in the Windows system. Rigorous cross-platform benchmarking is left as future work.

Forender is evaluated on two different scenes. The first scene is located in Valais and represents the smaller of the two evaluation areas. It covers a region of 16 x 17 km and contains 667'996 trees extracted by the preprocessor. The second scene is located in Ticino, covering an area of 85 x 65 km, containing 17'962'967 extracted trees. Note that in Ticino, the rendered area and the corresponding tree extraction region are slightly smaller than the mentioned area, due to it not being completely within Switzerland. The extent of both scenes compared to Switzerland as a whole is shown in Figure 4.1.

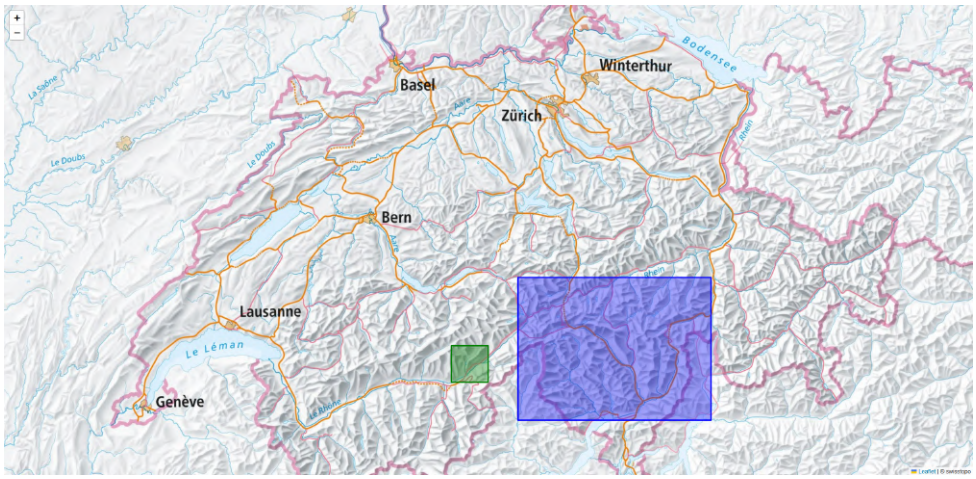


Figure 4.1.: Extent of evaluation scenes: *Image comparing the two scenes used during evaluation relative to Switzerland as a whole. The Valais scene is shown in green, with extent of the Ticino scene shown in blue. Only the region within Switzerland is used for rendering and tree extraction. The image was produced using the swisstopo API.*

Forender exposes many different parameters that can be tuned. Some parameters, such as max quadtree depth, only affect performance, while others, such as the LOD thresholds, represent a trade-off between visual quality and runtime performance. Forender uses meshes with five LODs (0-4) and has four density levels, where each level halves the number of trees. Trees are rendered up to a distance of 16 km, but density reduction thins forests before the cutoff. Forender WebGPU does not render trees smaller than 3 pixels, so individual tree geometry stops rendering well before this distance. As trees stop being rendered, the visuals of the forests and

[1] `https://www.notebookcheck.net/Apple-M3-10-Core-GPU-Benchmarks-and-Specs.765636.0.html` (Accessed: 21.08.2026)

[2] `https://www.notebookcheck.net/NVIDIA-GeForce-RTX-3060-Mobile-GPU-Benchmarks-and-Specs.497453.0.html` (Accessed: 21.08.2026)

trees are instead created by the terrain's color textures. The parameters, apart from the quadtree depth, were determined empirically through manual tuning and not selected using experiments. The selected values used are shown in Table 4.1.

Scene	Valais	Ticino
Preprocessor		
Min tree height	5 m	
Min tree distance	5 m	
Min quadtree tree count	256	2056
Max quadtree depth$^{(\text{WebGL})}$	5	7
Max quadtree depth$^{(\text{WebGPU})}$	4	
Downsampling ratio	0.5	
Downsampling method	Grid-based	
Max binary file size	3906.25 KB	
Color blur sigma	5	
Species blur sigma	7	
Frontend		
Max Terrain LOD	3	6
Terrain error threshold	0.01	100
LOD 0 threshold	0.1	
LOD 1 threshold	0.01	
LOD 2 Threshold	0.005	
LOD 3 threshold	0.0005	
Density 0 Threshold	0.005	
Density 1 threshold	0.003	
Density 2 Threshold	0.001	
Density 3 threshold	0.0001	
Tree render distance	16 km	
View factor	0.5	
SSAO sample count	16	

Table 4.1.: Forender parameters: *Parameters used for preprocessing and rendering in the Forender evaluations.*

4.2. Quantitative

The quantitative experiments are used to evaluate Forender's real-time performance claims as well as its initial startup time. The runtime performance is measured in frame time, i.e., how much time passes between two consecutive frames. Frames per second (FPS) measurements are not used directly because FPS is the inverse of frame time. This can make performance differences difficult to interpret, as equal changes in frame time do not correspond to equal changes in FPS. The fastest possible frame time is 16.6 ms due to having V-Sync enabled. The interactive performance target we aim for is between 30 and 60 FPS, which corresponds to frame times between 16.6 and 33.3 ms. Ideally, for an interactive experience, both the mean frame time and the 99th percentile lie within these bounds. A low mean frame time can still feel

unresponsive if there are severe frame spikes, hidden by averaging. The 99th percentile better captures these severe frame spikes.

Measurements are collected with the camera following one of two predefined trajectories per scene. One trajectory follows close to the ground, and another flies high above the treetops. They each represent different challenges, as the low-altitude trajectory requires greater geometric detail, while being further from the tree means more visible tree instances at lower detail. Each trajectory lasts 60 seconds, and each experiment is repeated three times. The same trajectories are used across all quantitative experiments to ensure comparability.

Frame time measurements are started on the first rendered frame, i.e., after startup (see Section 3.5.1). For each of the three measurements, the trajectory is executed once, with no prior run-through or warm-up. During benchmarking, only the local backend server and browser are actively running, though some unavoidable system processes may have remained active in the background.

Frame times are visualized using violin plots because frame time distributions often contain long tails. Unlike just reporting mean times, shown as a white dot in the plots, violin plots show the full distribution of the frame times. This is important because occasional outliers can significantly affect the user experience in real-time software.

Across the experiments, we compare multiple implementations with different configurations. The evaluated configurations are summarized below:

- **Reupload**: The Reupload implementation in WebGL, which fills the instance buffers every frame.
- **Leaf Node Draw**: The Leaf Node Draw implementation in WebGL. The implementation issues draw calls per leaf node instead of per LOD and uses one large instance buffer, with each leaf node having a reserved subsection.
- **Reupload (Z update)**: The Reupload implementation with runtime z position updates as described in Section 3.5.8.
- **Leaf Node Draw (Z update)**: The Leaf Node Draw implementation with runtime z position updates as described in Section 3.5.8.
- **Forender WebGL**: The faster of the two WebGL implementations without height updates. It is used as the WebGL baseline for comparison with Forender WebGPU.
- **Forender WebGPU**: The implementation in WebGPU using GPU-driven rendering and occlusion culling. It uses all optional features, such as SSAO, leaf tinting, and multiple tree species. All optimizations described in the implementation chapter are performed, and the quadtree depth was tuned for better performance.
- **Depth {i}**: Same as Forender WebGPU with a quadtree depth of i.
- **Frustum culling**: Same implementation as Forender WebGPU with all visual features enabled. The only optimization performed is frustum culling, both hierarchically on the quadtree as well as per instance using compute shaders. Leaf nodes always load their tree data at full density, and the density is not reduced per instance.

- **Frustum + Distance culling**: Based on Frustum culling with all visual features enabled. In addition to frustum culling, this implementation also performs distance culling, both hierarchically and per instance.
- **Frustum + Distance culling + Density Reduction**: Based on Frustum + Distance culling with all visual features enabled. Leaf nodes load their tree data based on density computations, and the density is reduced per instance in compute shaders as described in Section 3.6.1.
- **Frustum + Distance culling + Density Reduction + Occlusion culling**: Based on Frustum + Distance culling + Density reduction with all visual features enabled. Additionally, this implementation performs occlusion culling per instance using compute shaders. Terrain is the only occluder used. Same implementation as Forender WebGPU.

4.2.1. WebGL Implementations

We first compare the two WebGL implementations we introduced in Section 3.5.5 without z position updates to evaluate which workaround performs better. The frame time plots are shown in Figure 4.2. In both Valais trajectories, there is little difference between implementations. Both have a mean frame time between 18 and 21 ms. The tails in Valais Near do not differ substantially in length. However, in Valais Far, the Leaf Node Draw implementation has a notably shorter tail.

Performance differences are more pronounced in the Ticino scene. In Ticino Near, the major difference between the implementations is that Leaf Node Draw has a shorter tail. Their mean frame time is similar, with Reupload having a mean frame time of 50.6 ms when compared to 48.9 ms of Leaf Node Draw. In Ticino Far, Leaf Node Draw is both faster on average, with mean frame times of 47.3 ms (Leaf Node Draw) and 70.5 ms (Reupload), as well as having a shorter and thinner tail. Leaf Node Draw has less extreme frame spikes, with the maximal frame time dropping from 499.8 to 349.9 ms.

The performance results are likely due to Leaf Node Draw avoiding uploading large amounts of data. CPU-GPU memory transfer bandwidth is limited and is also used for uploading new terrain texture data. Leaf Node Draw only uploads data to the GPU when new density data is streamed in. This theory is supported by the greater difference in the larger scene, Ticino. The amount of data to be uploaded, i.e., visible trees, increases more due to the increased area than the amount of draw calls, i.e., visible leaf nodes. By trading the increase in draw calls for fewer data uploads, Leaf Node Draw reduces frame spikes and produces a smoother experience. Thus, Leaf Node Draw is used as the WebGL baseline for later comparison with WebGPU.

4.2.2. WebGL Z Updates

In this experiment, we evaluate the cost of performing z position updates at runtime, as described in Section 3.5.8. We compare both WebGL implementations, Reupload and Leaf Node Draw, with and without Z position updates.

In both Valais trajectories, implementations with and without Z updates have a similar mean

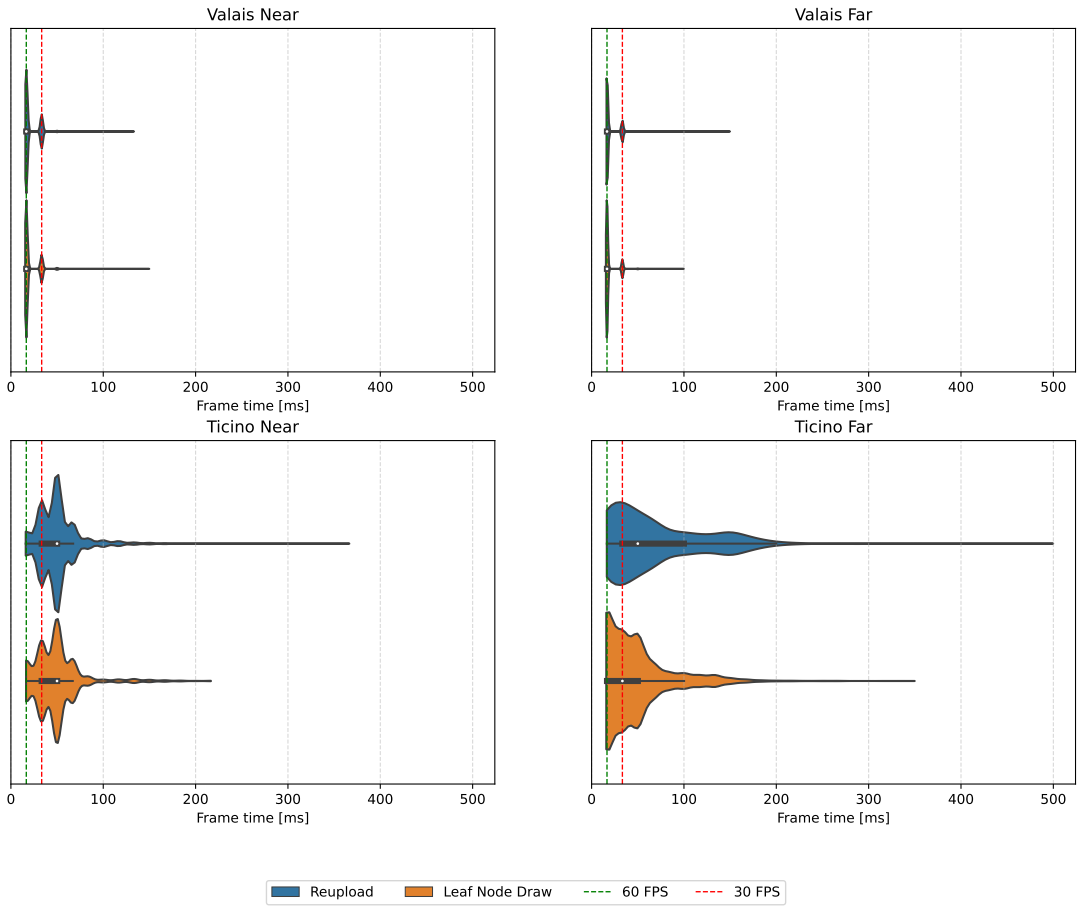


Figure 4.2.: Comparing WebGL implementations: *Violin plots of the frame times of the two WebGL implementations Reupload and Leaf Node Draw, with Leaf Node Draw performing better.*

frame time, with more frame times concentrated around 16.6 ms (60 FPS) in the implementations without updates, as shown in Figure 4.3. Both trajectories in the Ticino scene incur a major distribution shift upwards when using Z updates, thus drastically slowing down. In both trajectories, the effect is slightly less in Leaf Node Draw (Z update). Unexpectedly, in Ticino Far, the tails of both implementations with Z update, are notably shorter than without Z updates.

Runtime Z updates are shown to greatly increase frame times, making their current implementation unsuitable for larger scenes. To compute the currently visible height, `getHeightValue` descends the loaded terrain quadtree and samples from the texture for each tree. The cost of that function is too high to be computed on the large number of trees even asynchronously. The cost is further exaggerated by updating all trees, not just the important nearby trees. Although implementations with Z updates have shorter maximal frame times in some trajectories, this does not outweigh their performance overhead on the mean frame time. The cause of the shorter tails in Ticino Far is unclear. One possible explanation is that Z updates might block other async tasks such as density streaming. This could result in fewer trees being loaded, reducing extreme frame spikes.

The current implementation of Z position updates is therefore too expensive for larger scenes. Further optimizations, such as Web Workers or moving computation to the GPU, could make

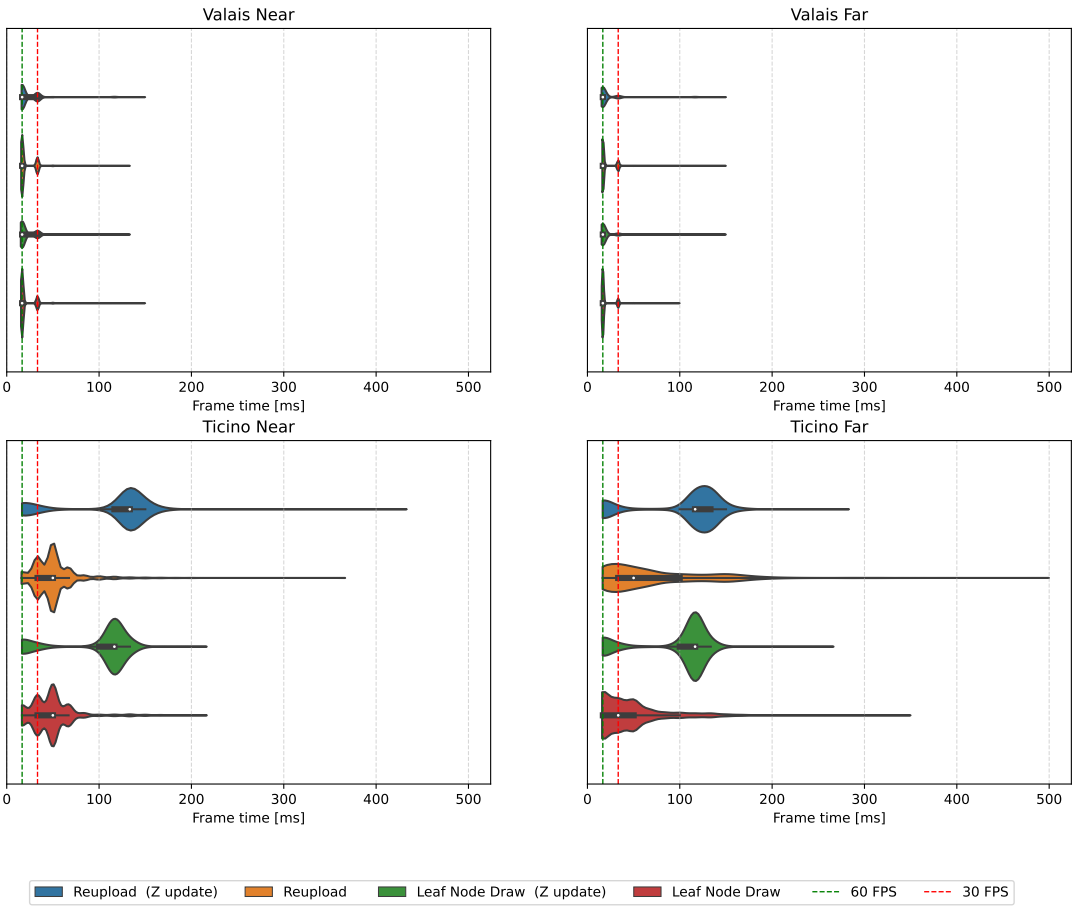


Figure 4.3.: Cost of runtime z position update: *Violin plots of frame times evaluating the cost of z position updates. Although there is only a slight impact on the performance in the Valais scene, on the larger Ticino scene, z updates incur a substantial runtime overhead.*

them viable and reduce the error present in offline z computation.

4.2.3. Forender WebGL and Forender WebGPU

In this section, we compare the two Forender implementations: Forender WebGL and Forender WebGPU. Forender WebGL uses the Leaf Node Draw implementation, as it was shown to be more performant in Section 4.2.1. Both implementations do not perform any z updates. Forender WebGPU avoids the workarounds required in WebGL to perform instanced rendering across multiple LODs by using a GPU-driven rendering pipeline. Both Forender implementations use all optimizations introduced in their respective sections, with Forender WebGPU having all its optional features enabled. Note that this experiment does not attempt to compare the performance of the two APIs but the two implementations that use the different features exposed by their respective API. When comparing the same implementation based on Reupload across both APIs, no statistically significant performance difference was observed.

The difference between the implementations impacts the performance across all scenes, with Forender WebGPU performing better in all of them as shown in Figure 4.4. In Valais, Forender

WebGL had a notable number of frames with frame times around 33.3 ms (30 FPS), whereas Forender WebGPU has nearly all frame times concentrated at 16.6 ms (60 FPS). The performance difference is even more notable in the Ticino scene. Forender WebGPU frame time has a 99th percentile of at most 32.6 ms occurring in the Ticino Far trajectory. In contrast, Forender WebGL has many frame times that are slower than 33.3 ms. Forender WebGPU has smaller frame spikes with frame times between 58 and 74 ms, while Forender WebGL has spikes varying between 100 and 350 ms.

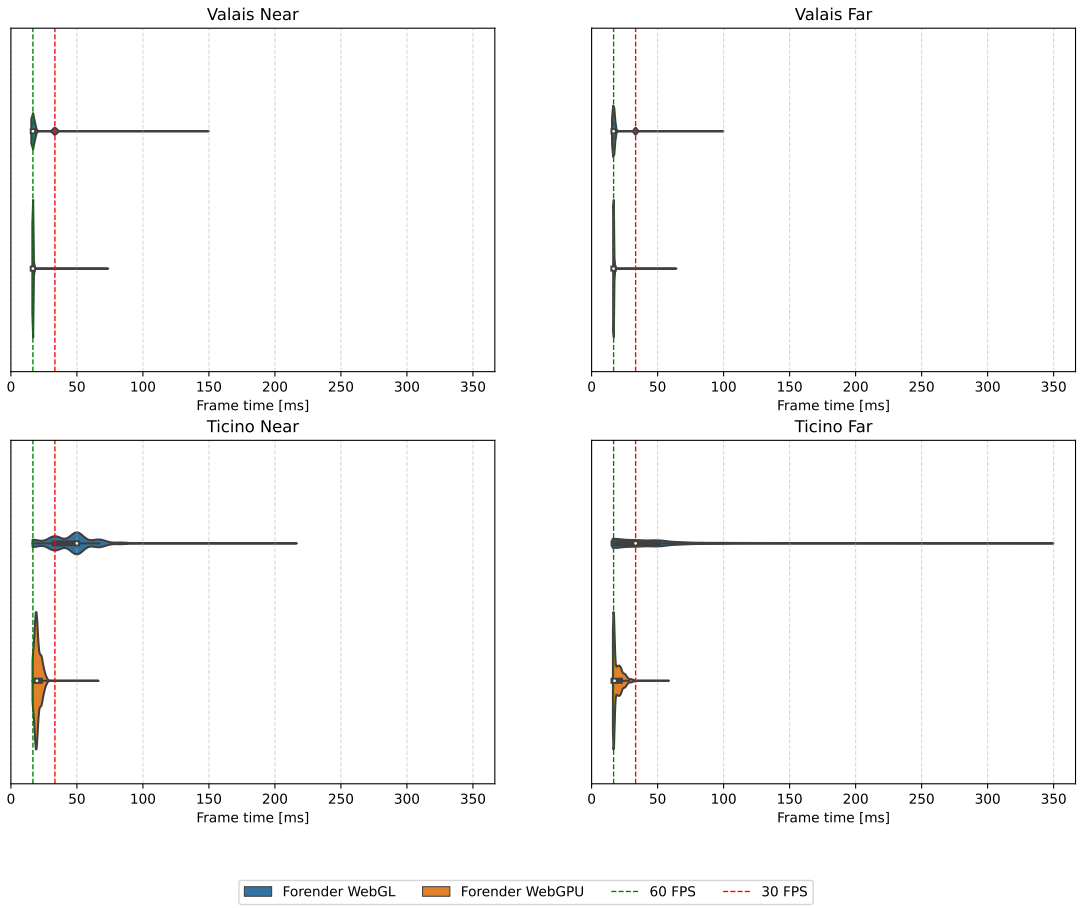


Figure 4.4.: Comparing Forender WebGL & Forender WebGPU: *Violin plots of frame times of Forender WebGL and Forender WebGPU. Forender WebGPU achieves notably lower and more consistent frame times. This result is despite Forender WebGPU using its additional optional visual features, such as SSAO.*

By making use of compute shaders, Forender WebGPU performs both culling and LOD selection at a finer granularity than Forender WebGL, thus rendering fewer trees unnecessarily. The impact of the individual optimizations is evaluated separately in Section 4.2.5.

Forender WebGPU offers substantially improved performance compared to Forender WebGL. It predominantly achieves more than 30 FPS, with the mean FPS being closer to 60 FPS than 30. Due to the major reduction in peak frame times, Forender WebGPU offers a more consistent frame time, demonstrating the benefit of the new optimizations supported by GPU-driven rendering. This experiment showcases the benefit of an optimized GPU-driven rendering approach for large-scale vegetation visualization. This result is achieved despite Forender WebGPU hav-

ing its optional visual features enabled (SSAO, multiple tree species, leaf tinting), which incur additional cost.

4.2.4. WebGPU Quadtree Depth

The next experiment is on quantifying the impact of the quadtree depth on Forender WebGPU's performance. All implementations use all optimizations and optional visual features, the only difference being the quadtree depth. Unlike Forender WebGL, changing the quadtree depth in Forender WebGPU only affects performance and not the rendered appearance, thus simplifying the task of empirically tuning the quadtree depth. The experiment is used to justify the default quadtree depth of four for Forender WebGPU.

As seen in Figure 4.5, in both Valais trajectories, all depths perform similarly, both in mean frame time and overall distribution of frame times. In the Valais Near trajectory, Depth 4 and 5 show a slightly larger portion of frame times around 16.6 ms than other implementations. They also have the shortest maximum frame times in Valais Near. In Valais Far, Depth 4 performs better than Depth 5 with the shortest tail and with the distribution strongly concentrated around 16.6 ms. The results are more drastic in Ticino. Depth 4 still has the best distribution, with more frame times close to 16.6 ms than any other variant. Although some other depths exhibit shorter maximum frame times, this metric is inherently sensitive to outliers. The 99th percentile is more robust and is the lowest at Depth 4 with 27.8 ms in Ticino Near and 32.6 ms in Ticino Far.

The quadtree depth is a trade-off between the granularity of hierarchical culling and the cost of traversal. If the quadtree is too deep, the traversal overhead on the CPU outweighs the benefit of more fine-grained culling. The results indicate that Depth 4 provides the best performance among the tested depths for these scenes. Ticino Far exhibits a greater frame time variance than Ticino Near across all depths. This might be due to an increased amount of data streaming, as the trajectory covers a larger area and thus triggers more terrain and vegetation data streaming. However, the streaming costs are not isolated, and thus this potential explanation cannot be confirmed.

The experiment shows that quadtree depth four is an empirically sound value for our scenes, with it having both the lowest mean and 99th percentile frame times. The results demonstrate that empirically evaluating parameters can be useful as long as they do not influence visual quality. Parameters such as min quadtree tree count and max binary file size could be tuned using similar experiments. However, many other parameters that have both an impact on visual quality and performance cannot easily be empirically tuned and still rely on tuning by hand.

4.2.5. WebGPU Optimizations

Next, we perform an ablation study on the different optimizations we introduced into Forender WebGPU. Starting with Frustum culling, we progressively add the remaining optimizations to evaluate their incremental impact on performance. Each optimization, apart from occlusion culling, is performed both on the quadtree on the CPU side and per instance on the GPU. Running the trajectories without any optimizations was infeasible for the larger Ticino scene.

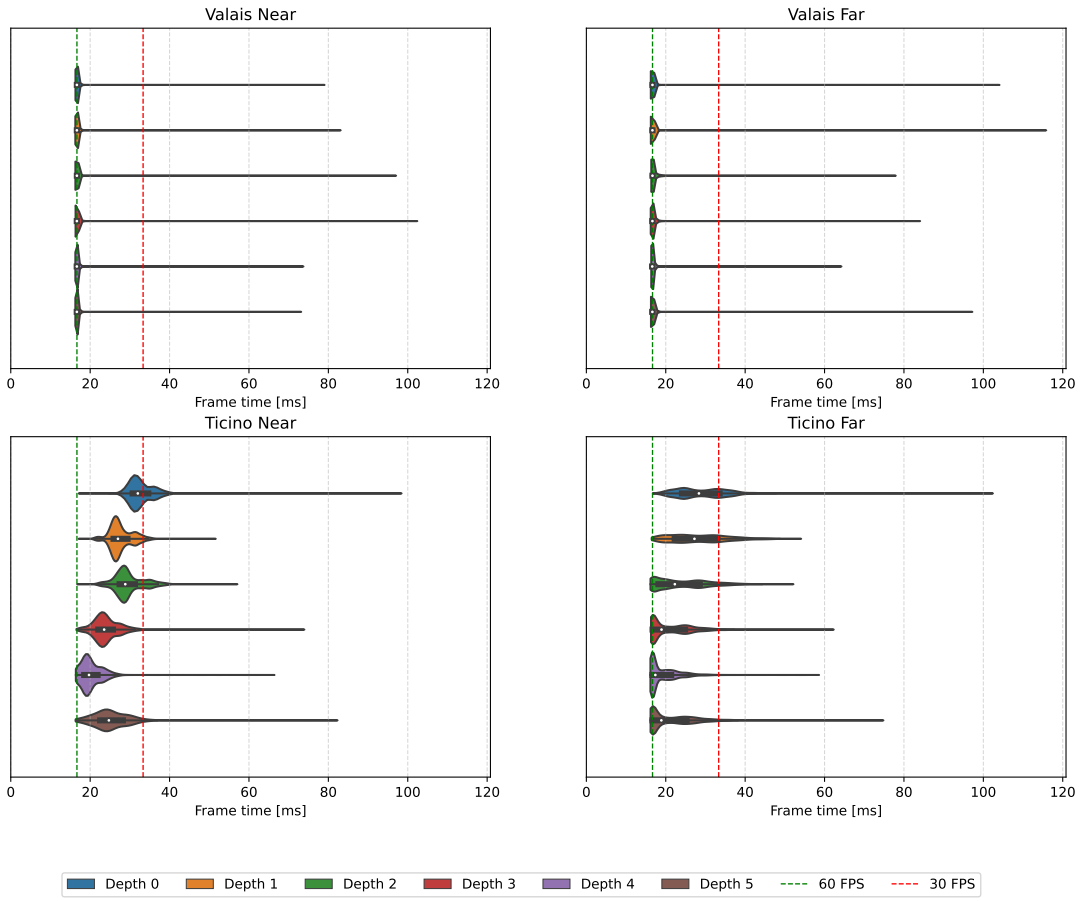


Figure 4.5.: Empirical evaluation of quadtree depth: *Violin plots showing frame times across various quadtree depths. Depth 4 performs the best out of the evaluated depths in our scenes.*

In the Valais scene, the tail of the frame time distribution shrinks with each additional optimization as shown in Figure 4.6. Without any optimizations, the trajectories have a mean frame time of 256.1 ms in Valais Near and 271.6 ms in Valais Far. In the Ticino scene, each optimization shifts the frame time distribution lower. Distance culling only has a small benefit when compared to the impact of Density reduction. Adding Occlusion culling also has a notable impact, with it being slightly greater in Ticino Near, further shifting the distribution. The maximum frame times are also gradually decreased when adding optimizations, though to a lesser extent than the reduction in mean frame time.

The small impact of distance culling could be explained by the relatively small number of trees removed, which are likely already rendered using low-detail LODs. Density culling is a lot more aggressive, reducing the tree count further, which might explain the performance difference. One possible explanation for the difference in impact of Occlusion culling between Ticino Near and Far is that the Near trajectory runs inside a valley, where terrain occludes numerous trees. Another explanation is that in Ticino Far Frustum + Distance culling + Density reduction is already quite performant, and further improvements are limited by the V-Sync rate.

The ablation study has shown that a great benefit comes from frustum culling, density reduction, and occlusion culling, with distance culling having a lesser impact. Each of the optimizations provides more benefit than what they cost in overhead and are therefore sensible to be used by

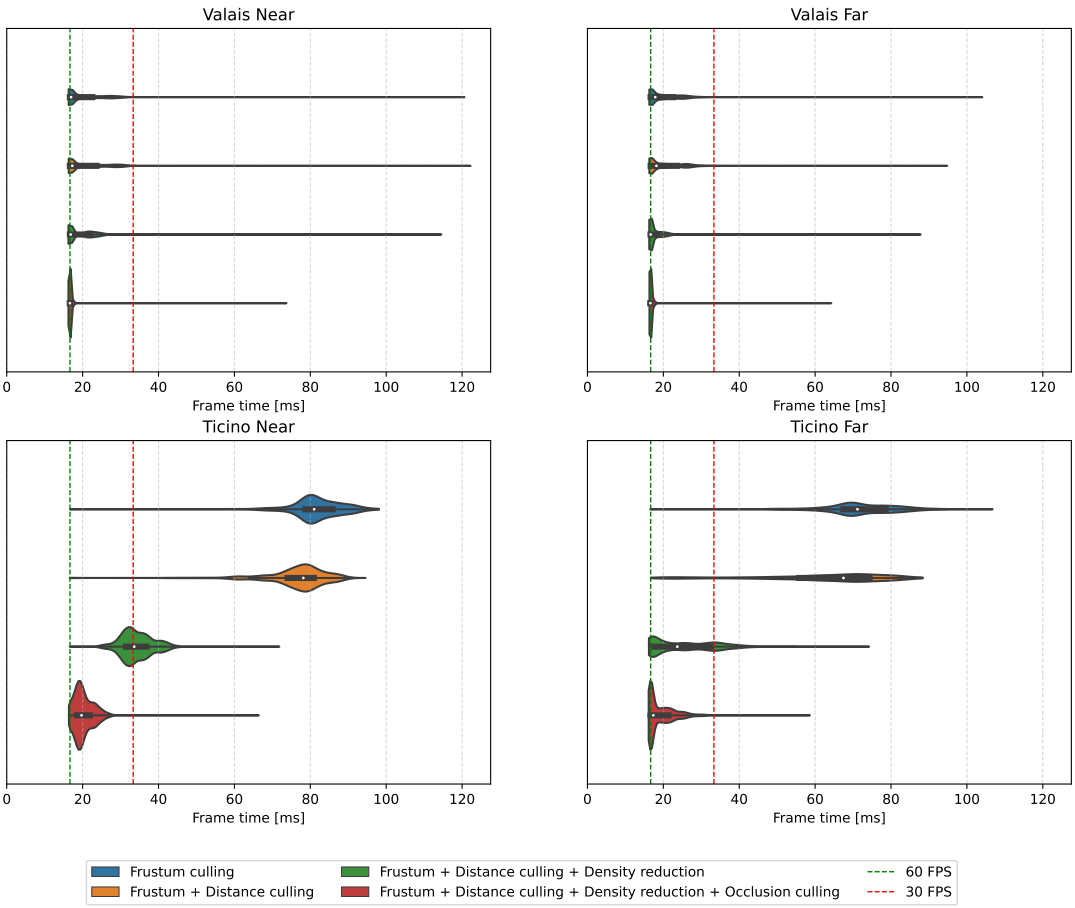


Figure 4.6.: Ablation study on optimizations: *Violin plots of the frame time evaluating the incremental impact of different optimizations. Each further improves the frame time distribution, indicating their performance benefit outweighs their additional cost.*

Forender.

4.2.6. WebGPU Startup Times

In this experiment, we evaluate the startup times in Forender WebGPU, i.e., the time between selecting a scene and displaying the first visible frame. We compare Forender WebGPU's startup time with Terrender WebGPU, i.e., only loading Terrain. In contrast Forender WebGPU also loads the quadtree, 3D models, and the tree instance data. Ten 3D models are loaded, corresponding to two species with five LODs each.

Note that the terrain and tree instance data are loaded before the first visible frame based on the initial camera position, so there could be differences in the work performed between trajectories and scenes. However, in Figure 4.7, the startup times are similar across all scenes and trajectories. Forender WebGPU is around five seconds slower than Terrender WebGPU at starting up.

Further profiling has shown that the majority of the additional Forender WebGPU startup time

is spent on the 3D model loading, more specifically the texture decoding. This is performed in the main thread and explains the additional five seconds.

The texture formats used in the models are png and jpeg. Neither is optimal for sending the texture or for being decoded into a GPU friendly format and should ideally be replaced. Similarly to terrain texture decoding, this work should also be performed on a separate Web Worker and not on the main thread. Although initial startup is not yet optimized, the measured startup times are sufficiently short for the current use case, making these optimizations lower priority.

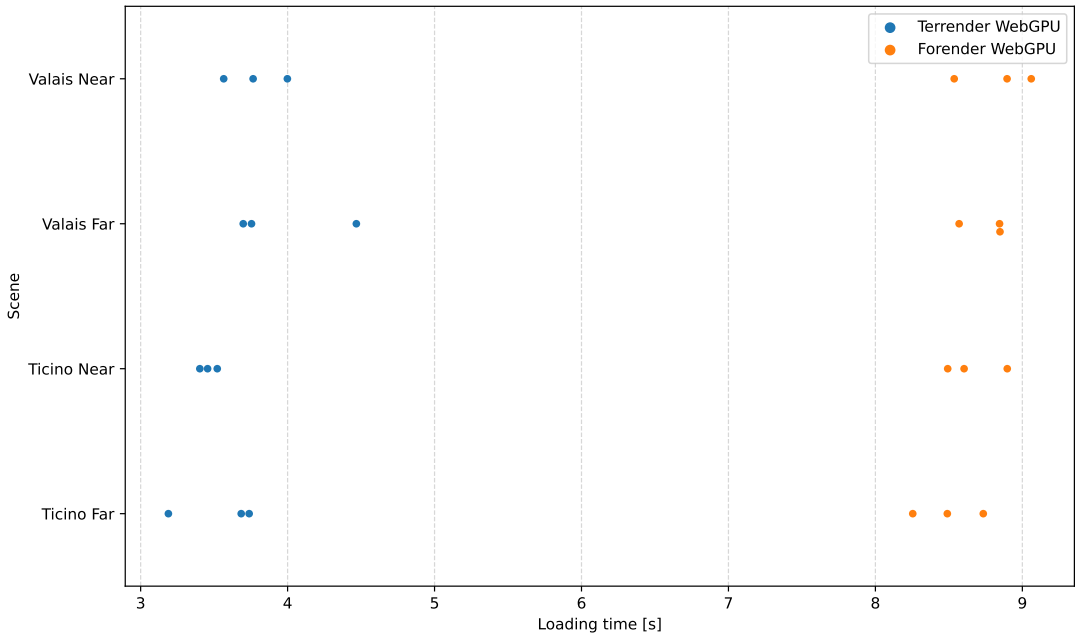


Figure 4.7.: Startup times in Forender WebGPU: *The image compares the startup times of Forender WebGPU and Terrender WebGPU. Forender WebGPU adds approximately five seconds, mostly spent on texture decoding.*

4.2.7. WebGPU Long-Term Stability

All previous frame time experiments were evaluated on 1-minute trajectories. This might not represent the performance a user experiences during a longer usage session, as it might degrade over time. To evaluate long-term stability, we introduce a separate trajectory that takes 30 minutes, and we evaluate the frame time along this trajectory. The trajectory is in the Ticino scene, and it repeats the same path eleven times to make potential performance degradation more pronounced and easier to observe.

In Figure 4.8 we show the smoothed frame time during the long trajectory. The frame time is consistent across the eleven repetitions, with a temporary disturbance causing higher variance between five and ten minutes. The reason for that is unclear, with a possible explanation being interference from unrelated browser or OS background tasks.

This experiment indicates that the performance remains consistent even during long usage sessions of Forender WebGPU, with no observable degradation.

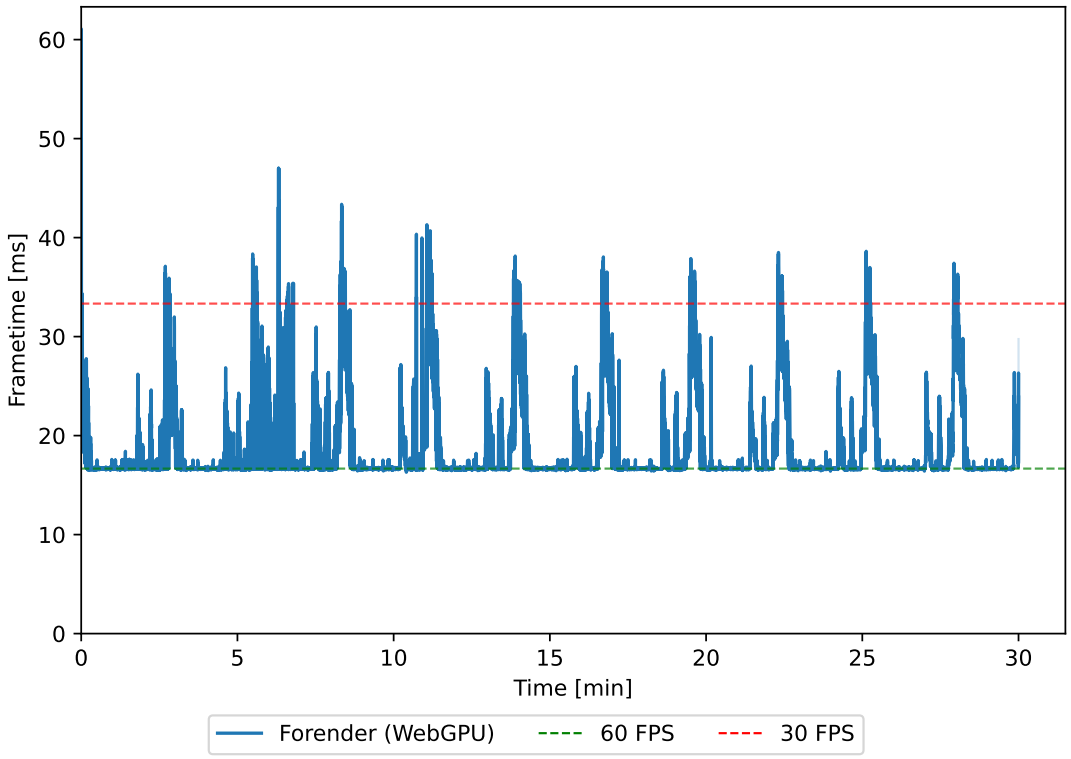


Figure 4.8.: Long-term trajectory: *Trajectory showing performance over a 30 minute trajectory. We visualize the smoothed frame time over the course of the trajectory. No performance degradation is observed.*

4.2.8. iGPU Performance

The last quantitative experiment focuses on how well Forender performs on a weaker, integrated graphics card such as *Intel(R) UHD Graphics*. Such cards are more broadly available than the *NVIDIA GeForce RTX 3060 laptop GPU* used in the previous experiments. We compare the two GPUs across three different Forender implementations: Forender WebGPU, Reupload, and Leaf Node Draw. The experiment evaluates whether the more advanced graphics features used by Forender WebGPU result in worse performance on integrated GPUs compared with the WebGL implementations.

In general, Forender WebGPU still performs the best even on an integrated GPU as shown in Table 4.2, with it having the lowest mean frame time in all but one trajectory, Valais Far. Note that even while being the best out of the implementations compared, Forender WebGPU still has a mean frame time between 140 and 160 ms when running on UHD. These frame times correspond to an FPS between 6 and 7. Forender WebGPU has the lowest 99th percentile frame time in the Ticino scene. However, this is not the case in Valais, with it having extremely high 99th percentile frame times of over two seconds. Both WebGL implementations have a similar mean frame time on UHD, with the better 99th percentile changing between trajectories, with Leaf Node Draw being better in Valais Far and Reupload being better in the other ones.

The cause of the unusually high 99th percentile frame times in Valais is not fully understood. The same issue probably also contributes to Forender WebGPU not having the lowest mean

Measure	Mean frame time [ms]		99th percentile [ms]	
GPU	RTX 3060	UHD	RTX 3060	UHD
Reupload (WebGL)	20.8	151.6	49.9	**303.8**
Leaf Node Draw (WebGL)	20.0	151.5	50.0	353.3
Forender WebGPU	**16.8**	**139.8**	**20.3**	2192.8

(a) *Valais Near*

Measure	Mean frame time [ms]		99th percentile [ms]	
GPU	RTX 3060	UHD	RTX 3060	UHD
Reupload (WebGL)	19.5	108.0	33.5	349.9
Leaf Node Draw (WebGL)	18.8	**107.4**	33.5	**300.0**
Forender WebGPU	**16.8**	160.5	**18.7**	2413.6

(b) *Valais Far*

Measure	Mean frame time [ms]		99th percentile [ms]	
GPU	RTX 3060	UHD	RTX 3060	UHD
Reupload (WebGL)	50.6	212.0	149.9	948.7
Leaf Node Draw (WebGL)	48.9	210.7	150.1	1132.8
Forender WebGPU	**20.4**	**150.9**	**27.8**	**186.3**

(c) *Ticino Near*

Measure	Mean frame time [ms]		99th percentile [ms]	
GPU	RTX 3060	UHD	RTX 3060	UHD
Reupload (WebGL)	70.5	187.4	233.2	683.0
Leaf Node Draw (WebGL)	47.3	169.2	166.7	932.8
Forender WebGPU	**19.5**	**132.9**	**32.6**	**209.1**

(d) *Ticino Far*

Table 4.2.: iGPU frame time*: Tables showing the mean and 99th percentile frame times for the four trajectories. Forender WebGPU performs better than the WebGL implementations in the larger Ticino scene. However, it exhibits unusually high 99th percentile frame times in the smaller Valais scene. Forender WebGPU is not interactively usable on UHD Graphics in the current configuration.*

frame time in Valais Far. Valais differs from Ticino by having a lower number of trees and higher terrain quality settings, i.e., a lower terrain error threshold. The unusual differences in the 99th percentile frame times might be due to differences between the terrain rendering in Terrender WebGL and Terrender WebGPU.

Forender WebGPU performs better than the WebGL implementations in large scenes, even on an integrated GPU. However, it exhibits unusual behavior on UHD in the smaller Valais scene with extremely long frame time tails. In general, Forender WebGPU is too slow to be used on an integrated GPU and would require additional optimizations or aggressive parameter tuning, potentially degrading the visual quality.

4.3. Qualitative

In this section, we evaluate the visual impact of the different design decisions and features introduced in Forender. The motivations for these features vary, from avoiding artifacts in the case of blending to performance in the case of density to visual fidelity such as SSAO and leaf tinting. The comparisons are mostly based on screenshots captured in the engine. Note that some color differences between Forender WebGL and Forender WebGPU are not due to a difference in the Forender implementations but due to Terrender WebGPU applying tone mapping.

4.3.1. Tree Position Extraction

Although not a focus of this thesis, the quality of tree position extraction still has a significant impact on the accuracy and visual quality of our rendered output.

The tree position extraction process is visualized for a small area in the Valais in Figure 4.9. Many trees appear to be positioned plausibly, and there seem to be few occasions where a single

tree is represented as multiple trees. However, there are multiple regions where trees appear to be present, but none are extracted. This issue is due to the forest polygon not capturing these regions nor being represented as single trees ("TLM_EINZELBAUM"). Another issue present in the tree extraction process is false positives within the forest polygons due to electricity poles or similar structures, shown in Figure 4.10. These structures might be incorrectly added to the vegetation category of swissSurface3D and rasterized into the Vegetation Height Model NFI. Leaf tinting might make them stand out further against the differently colored forest.

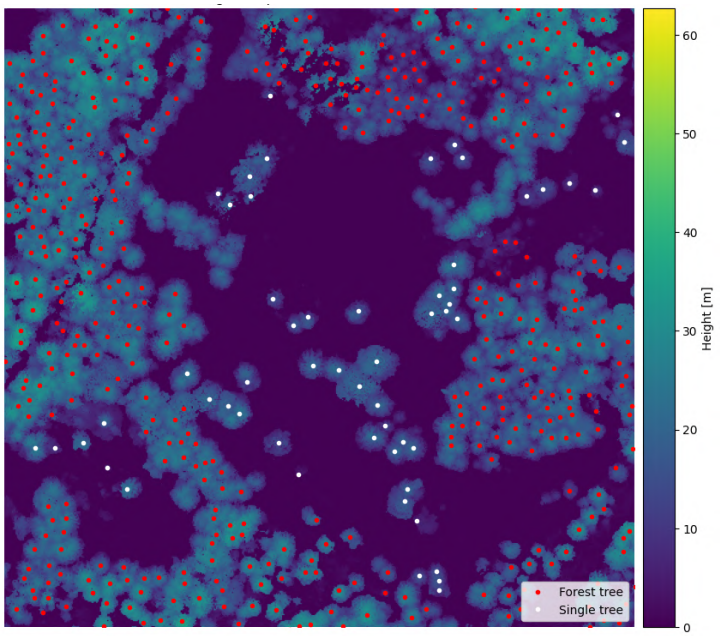


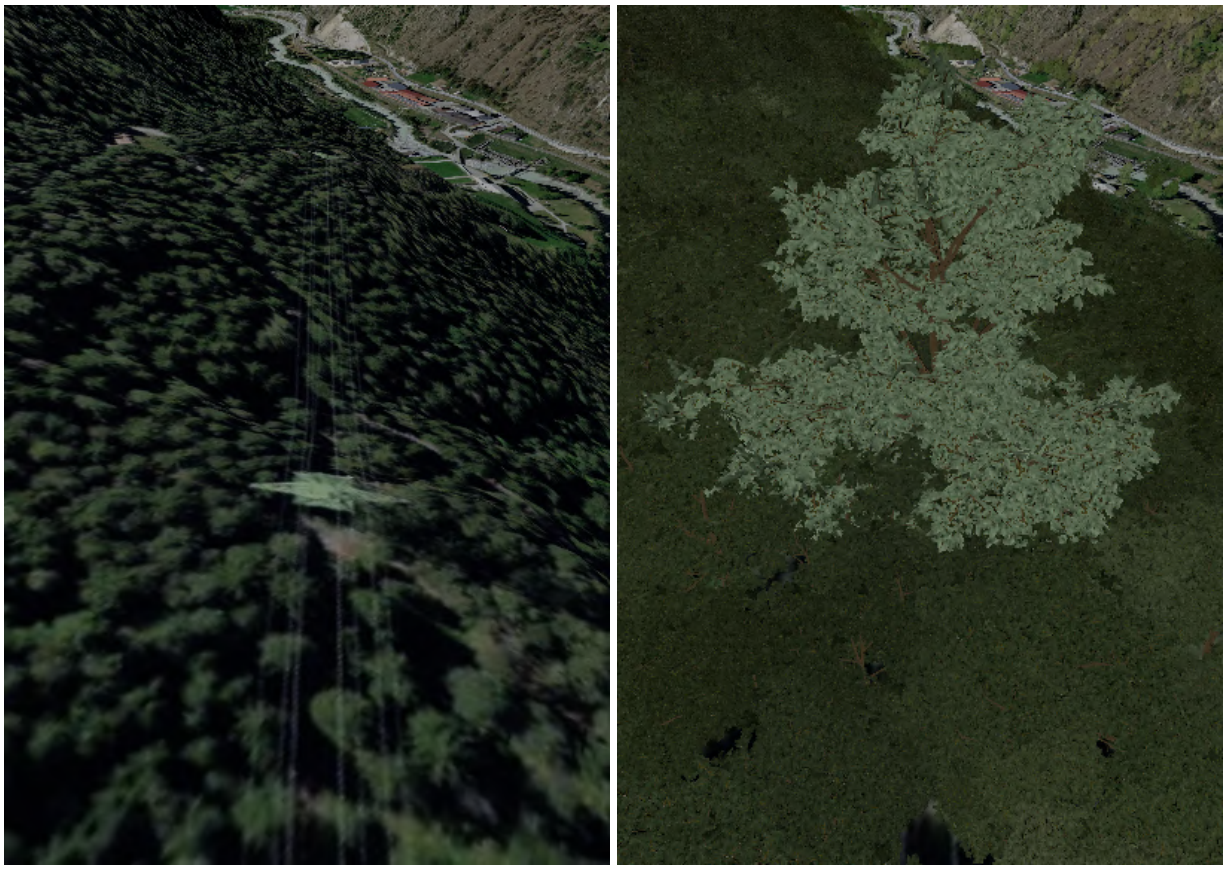
Figure 4.9.: Tree position extraction: *This figure shows the result of tree position extraction using Vegetation height and swissTML3D. Some areas contain false negatives, where trees are present but not detected. The red points show the positions of local maxima after filtering using forest polygons, while the white points represent single trees in swissTML3D.*

Overall, the position extraction system has some limitations. However, position extraction is not the focus of this thesis and is thus not quantitatively evaluated. The extraction step of the preprocessor could easily be replaced by a better system if this becomes a strict requirement.

4.3.2. LOD Decision

Forender WebGL and Forender WebGPU differ in how the LOD is computed, with Forender WebGL computing the LOD per leaf node and Forender WebGPU computing it per instance. In this evaluation, we compare the visual quality of these two approaches. Note that the LOD thresholds have been tuned for the per instance decisions, and the artifacts present in Forender WebGL can be slightly reduced by tuning them differently. However, the reduction of artifacts would likely require a more conservative LOD selection and therefore increase the frame time.

In Figure 4.11 we compare the different approaches for LOD decisions on the same view. Deciding the LOD per leaf node leads to visual artifacts between node boundaries where LOD changes occur. When applying the LOD blending approach discussed in Section 3.5.7, some



(a) Terrain texture *(b) Trees*

Figure 4.10.: False positive tree: *Example of a false positive within a forest polygon. Structures such as electricity poles might incorrectly be represented as trees.*

seams become less visible. Increasing the blending duration might reduce the seams further, but would come at the cost of drawing more trees twice. If the decision is made per instance on compute shaders, there are no seams. When performing LOD selection per instance, trees in the same leaf node can use different LODs, but there is still a visible difference between the low-detail LODs, which is exacerbated by the uniform tree colors. These differences are less noticeable when using leaf tinting.

Instance-based LOD decisions bring not just performance benefits, but also removes distracting seam artifacts present in Forender WebGL. The visual difference between low-detail LODs is reduced when using the optional leaf tinting feature.

4.3.3. Density Downsampling

Next, we discuss the visual impact of density reduction. We first compare the two approaches implemented to split tree instances across density levels discussed in Section 3.2.3: uniform and grid-based downsampling. Afterwards, we also evaluate the visuals of per instance density at varying distances. The per instance density method removes trees based on their screen space size as explained in Section 3.6.1.



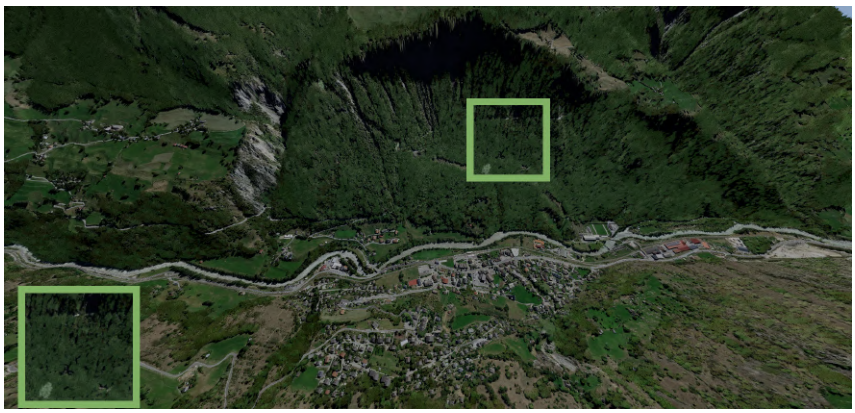
(a) *Per leaf node*



(b) *Per leaf node & blending*

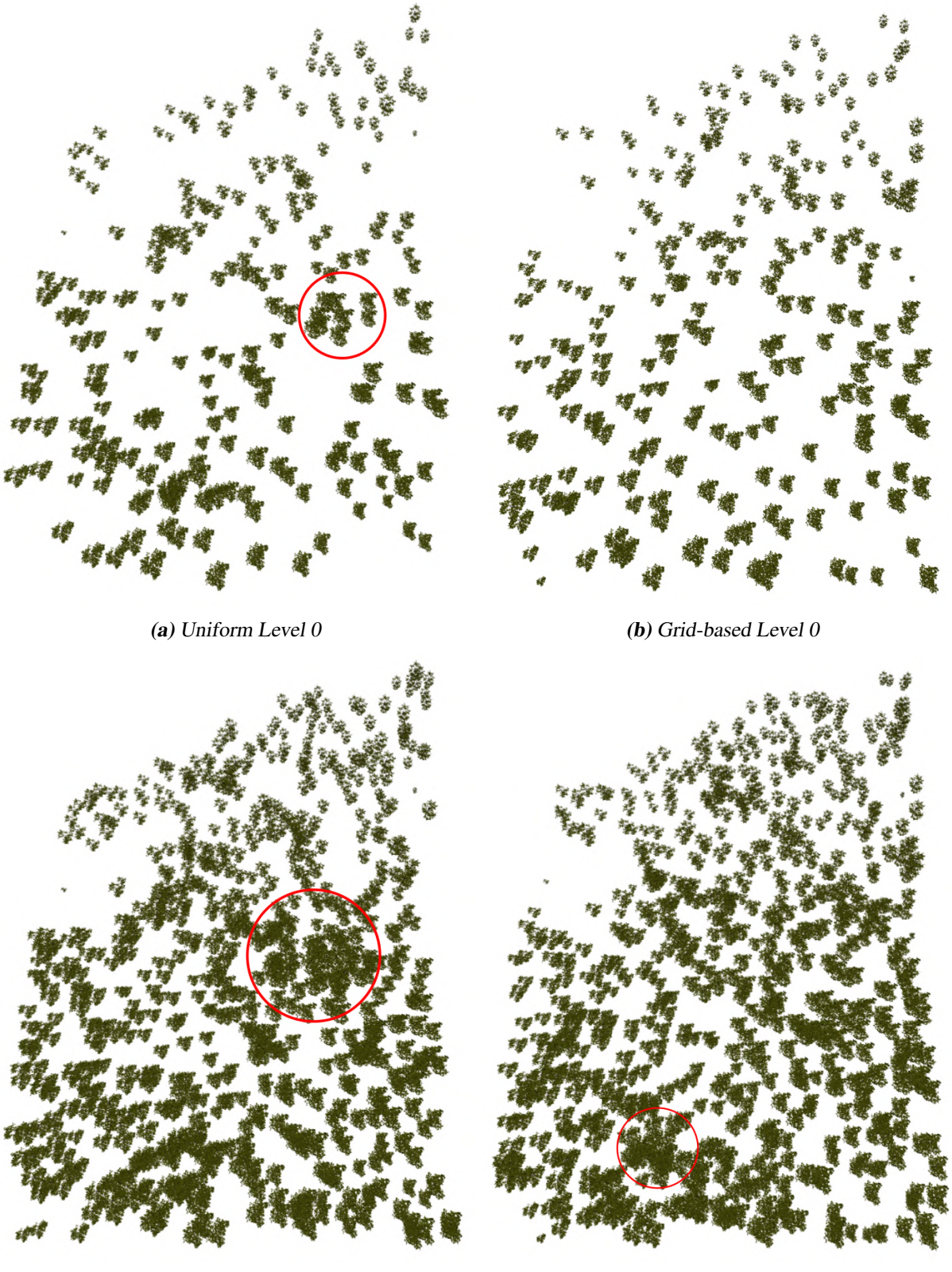


(c) *Per instance*



(d) *Per instance & leaf tinting*

Figure 4.11.: LOD decision visuals: *Different approaches for deciding the LOD and their respective visual artifacts. The artifacts are reduced by per instance LOD and leaf tinting.*



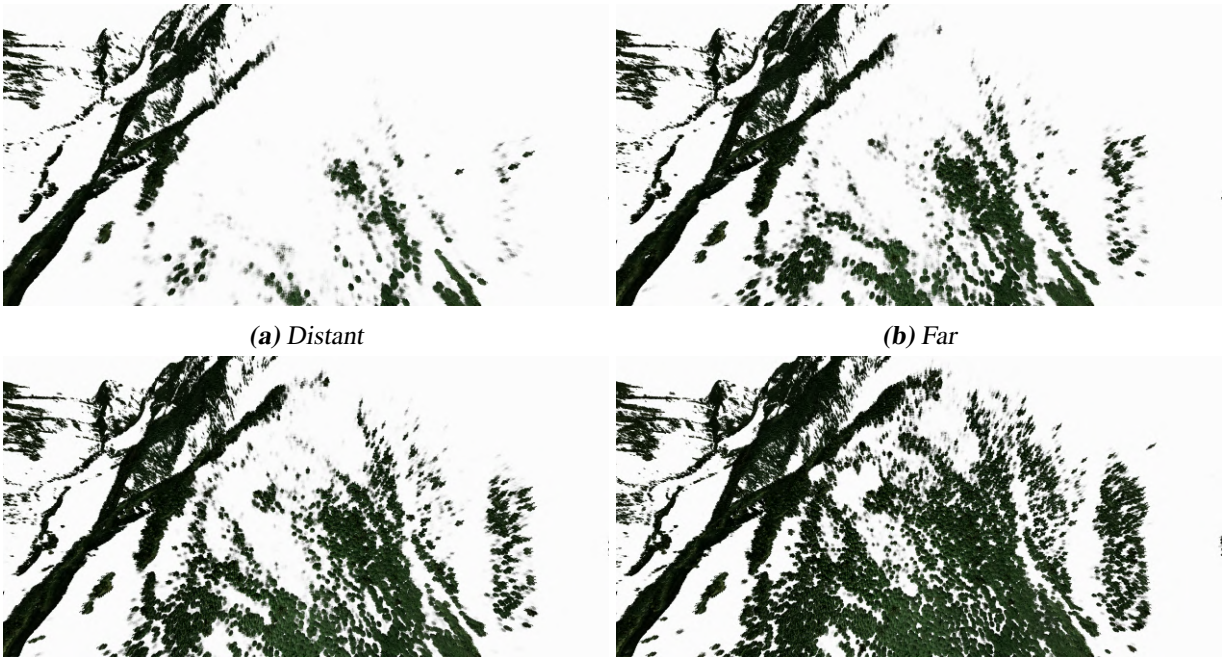
(a) Uniform Level 0

(b) Grid-based Level 0

(c) Uniform Level 1

(d) Grid-based Level 1

Figure 4.12.: Density levels with different methods: *Images comparing the two downsampling methods introduced to create density levels. We compare the two lowest density levels, containing the fewest trees. Uniform downsampling suffers from clustering, as highlighted in red. Due to projection into 2D and the 3D extent of trees, the grid-based method can also exhibit some visual clustering.*



(a) Distant **(b)** Far

(c) Moderately far **(d)** Mid-range

Figure 4.13.: Per instance density at various distances: *Per instance density decisions for a camera at various distances, visualized from a separate fixed camera. Note that even mid-range does not show the maximum available density.*

The lower discrete density levels created for a single leaf node are shown in Figure 4.12. The remaining density levels are shown in Figure A.5. Uniform sampling suffers from lots of clustering and empty regions, which are less present in the grid-based method. These are especially noticeable on the density level with the fewest trees, level zero. The grid-based method can still appear to be clustered from different views even though the grid used is 3D. The visual clustering is partly caused by projecting the 3D models onto the two dimensional screen. Trees that are well separated in 3D can appear cluttered on the screen. Another reason might be that the offline preprocessor treats trees as points and does not take their 3D extent into account when creating density levels.

Figure 4.13 shows an external view of the same region. A separate camera is used to compute the density. The separate camera is moved backwards from the view shown in the image along the view direction, resulting in the different densities shown as the distance increases. The per instance method does not remove trees viewed from the side even at large distances as shown in Figure A.7a. They are especially important as they define the outline of the forest against non-forest terrain or the atmosphere as explained in Section 3.5.3. The discrete density selection has a similar effect, as there the density level selection depends on the viewing angle of the AABB. Per instance density preserves the density of the original data when downsampling. Keeping the relative density results in spatial clustering, which is by design rather than an undesired artifact as in uniform downsampling. The per instance density computation is tuned to be more aggressive than the per leaf node version, as leaf node density can suffer from similar artifacts as discussed in Section 4.3.2 due to per node LOD decisions.

Using grid-based downsampling is preferable over uniform downsampling as the method results

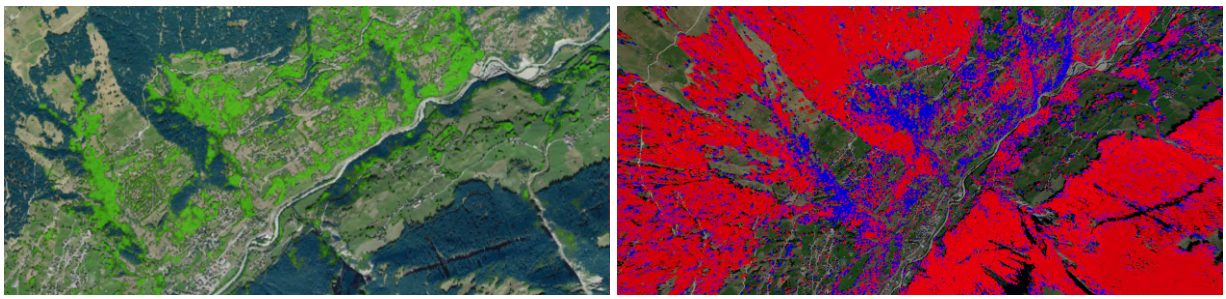
in less clustering. By computing the density per instance at runtime, Forender WebGPU can reduce the density more aggressively without creating visual artifacts. Per instance density keeps dense areas in the original data dense when downsampled. Per instance density thus results in clustering per definition but is not considered an artifact. Whether preserving the density distribution or the spatial extent of sparse forest regions is preferable is unclear. The original downsampling methods are focused on 3D point clouds and not 3D models, where the models have a 3D extent and are also present in the underlying terrain.

4.3.4. Tree species

The next evaluation focuses on the tree species s. The goal is not to achieve nor to test for perfect classification for individual trees but rather to determine whether the large-scale distribution present in the forest mix rate data is preserved in the sampled tree species.

In Figure 4.14, we compare the species c visualized in the engine with the forest mix rate data. The areas of the deciduous and coniferous trees broadly correspond to those in the forest mix rate data. As the values in the forest mix rate data are probabilities, there is some randomness and noise in the species selection. Forender cannot distinguish between the value being close to 0.5 due to uncertainty or due to the forest being mixed. The noise is stronger if the blur radius for species is smaller.

Similarly to position extraction, the species computation is not a large focus of this thesis, but this evaluation showcases that the approach manages to represent general species trends. The approach is only as good as the underlying data and has methods to deal with invalid data. Due to the randomness of the decision, there is some noise in the selected species. As was already discussed for tree position extraction, this method could be replaced or extended to multiple tree species if required.

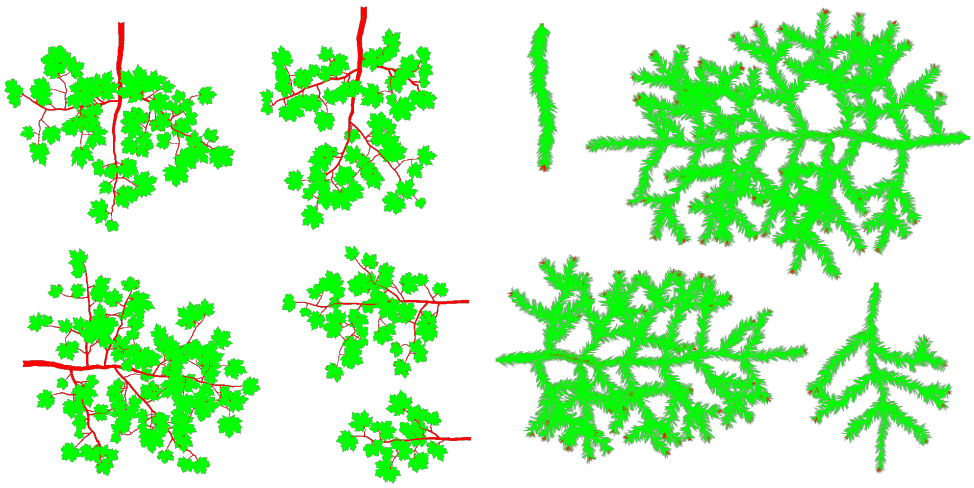


(a) *Forest mix rate* ***(b)*** *Species c*

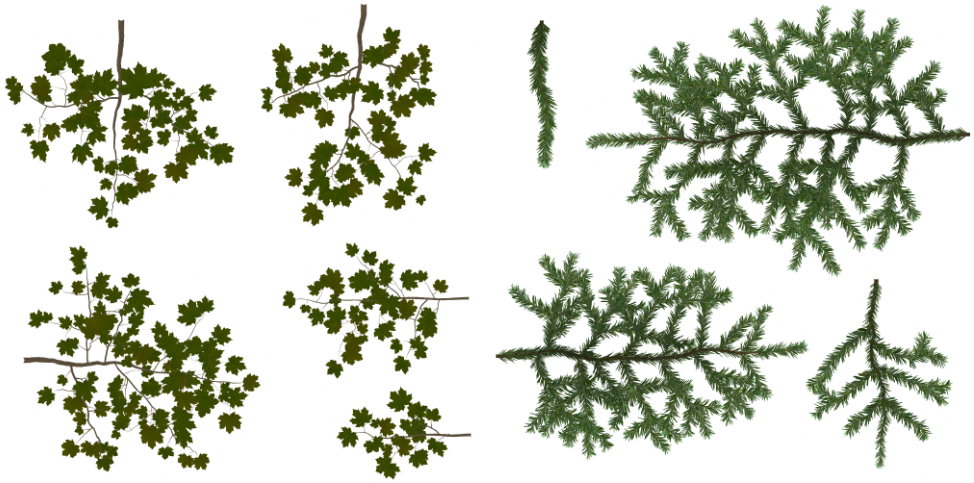
Figure 4.14.: Tree species computation: *The figure on the left shows the forest mix rate visualized by swisstopo with dark green regions meaning trees in a pixel are likely to be coniferous and light green meaning they are likely deciduous. The right figure shows an in-engine visualization of the tree species where red represents fir trees i.e., coniferous and blue represents maple trees i.e., deciduous. The spatial distributions in the two figures match closely. The distortion between the two images is due to 3D projection.*

4.3.5. Leaf Tinting

In this section, we evaluate the quality of the leaf tinting masks computed at runtime, as well as the visual quality of leaf tinting.



(a) *Maple mask* ***(b)*** *Fir mask*



(c) *Maple texture* ***(d)*** *Fir texture*

Figure 4.15.: Leaf tinting masks: *Visualization of the leaf textures and their corresponding spatial masks computed at runtime. The maple mask uses $k = 1.5$ and the fir mask uses $k = 2.2$. The fir mask incorrectly masks out the tips even with large k due to their large color difference.*

The two masks computed for the two species are shown in Figure 4.15. Each mask is computed as the pixels that are within k times the standard deviation from the texture's mean color. k differs between species. This selection works reasonably well for the maple texture and separates the bark from the leaves in Figure 4.15a with only a few small regions of leaves being misclassified as bark. However, the method does not work well in Figure 4.15b where large numbers of tips are misclassified as bark even with a large k. The large factor leads to less bark being correctly marked. The issue is due to the young tips being notably brighter than the rest of the texture. The issues with the fir mask are avoided by tinting the complete fir texture. The

workaround leads to slight color differences between the bark of the alpha masked and the bark of the opaque material but is more tolerable than the tips not being tinted.

The per leaf color is extracted in a preprocessing step from the orthoimages used for terrain tiles. The choice of blur strength has a significant impact on the visual quality of leaf tinting. If the blur is too small, the tinted trees look noisy as shown in Figure 4.16a. If the blur is too large, the trees begin to look like one large, diffuse surface as in Figure 4.16b. Another important parameter for leaf tinting is the tint strength. If the strength is too small, then there is little difference between trees as shown in Figure 4.17a. However, if the tint strength is too large, details present in the albedo textures are lost.

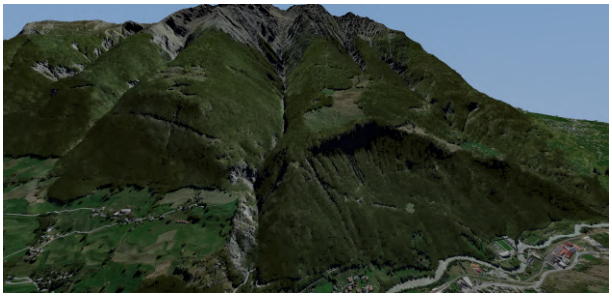


(a) *Insufficient blur* ***(b)*** *Excessive blur*

Figure 4.16.: Leaf color under- and over-blurring: *Image on the left uses an insufficient blur (radius 2), leading to a noisy appearance. The right image uses an excessive blur (radius 15), resulting in an overly diffuse appearance. Both images have a tint strength of one.*



(a) *Too little tint* ***(b)*** *Too much tint*



(c) *Suggested tint*

Figure 4.17.: Various tint strengths from afar: *The image on the top left has a tint strength of 0.2 and has too little impact on appearance. The image on the top right has a strength of 1 which looses the albedo texture details. The image on the bottom shows tint strength 0.6 from afar which increases variation without losing details.*

Overall, leaf tinting can add substantial visual variation between trees and approximate other visual effects that Forender does not explicitly handle, such as static terrain shadows, tree species that are not represented, and variations in leaf color within a single species due to environmental factors.

4.3.6. Alpha to Coverage

Alpha to Coverage (A2C) can be difficult to evaluate directly on images, as its main benefit is a reduction in temporal aliasing during camera movement. However, we can still demonstrate its effect on the anti-aliasing performed on alpha masked edges.

We compare two enlarged images in Figure 4.18, one without Alpha to Coverage and one with. Looking at the border between leaf and sky, the edges are less jagged in the A2C image because some pixels appear semitransparent. Edges that are due to geometry and not alpha masking, such as the brown branch, are already anti-aliased even without A2C.

Alpha to Coverage improves MSAA to also anti-alias edges that are due to alpha masking. If MSAA is already in use, A2C is quick to implement in any engine. Forender WebGL is currently missing this feature due to time constraints, but it could easily be added as well.



(a) *No Alpha to Coverage* ***(b)*** *Alpha to Coverage*

Figure 4.18.: Alpha to Coverage visuals: *Zoomed in screenshots comparing alpha masking with and without Alpha to Coverage. Although geometry edges are anti-aliased in both cases, edges due to alpha masking are only aliased when using Alpha to Coverage.*

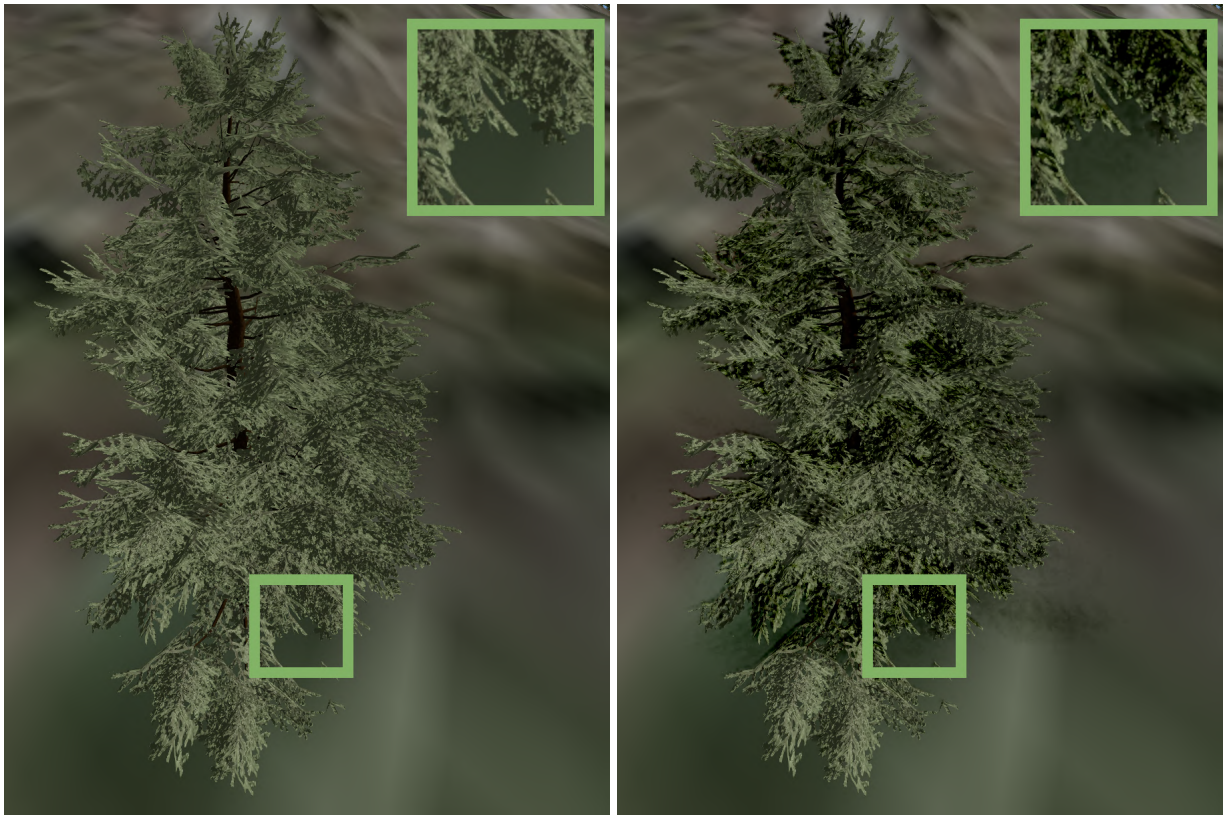
4.3.7. SSAO

The last qualitative evaluation is on the visual impact of *Screen space ambient occlusion* (SSAO) on our rendered forest scenes.

As shown in Figure 4.19, SSAO darkens the ground below a tree and areas further within the tree that would receive less light in the real world. The trees appear to have actual depth due to the darkening, and they seem more connected to the ground. SSAO is sensitive to its parameters. They need to be carefully tuned to avoid darkening the terrain behind a tree or further darkening

the terrain due to other terrain occluding ambient light, as this effect is already represented in the terrain texture.

SSAO improves the visual quality of Forender, specifically by making trees appear less uniformly lit and more connected to the 3D world as long as the parameters are carefully handled.



(a) *No SSAO* ***(b)*** *SSAO*

Figure 4.19.: SSAO visuals: *Comparison of a fir tree with and without SSAO. SSAO increases the perceived visual depth and makes the tree appear more integrated with the surrounding scene. Leaf tinting was enabled.*

5

Conclusion

This thesis aimed to evaluate the feasibility of a real-time web-based forest visualizer with large viewing distances. The results discussed in the previous chapter show that the visualization is feasible, while maintaining both interactive frame rates and high visual fidelity.

Two implementations of Forender were developed, one based on WebGL and the other on WebGPU. Forender WebGL achieves interactive frame rates in smaller scenes but does not scale to larger scenes without a notable reduction in frame rate or visual quality. In contrast, Forender WebGPU scales better and achieves interactive frame rates even on larger scenes and at large viewing distances. Using compute shaders enables Forender WebGPU to perform several decisions with finer granularity, including per tree culling, density, and LOD selection.

The results demonstrate the benefits of GPU-driven rendering. GPU-driven rendering enables more fine-grained decisions, increasing performance and reducing visual artifacts caused by coarse-grained decisions, such as LOD seams. Although no individual optimization achieves the desired performance on its own, the optimizations incrementally improve frame times. Together, they enable interactive frame rates even in demanding scenarios.

The performance gained enables the addition of optional features that increase the visual fidelity of Forender WebGPU, such as SSAO, support for multiple tree species, and leaf tinting.

The implementation demonstrates that rendering techniques used in video games and other desktop applications, such as GPU-driven rendering and occlusion culling, can also be adapted for a web-based forest visualizer.

The GPU-driven pipeline is only possible due to WebGPU exposing new features, such as compute shaders and indirect draw calls, which were not available in WebGL. The benefits achieved through the GPU-driven pipeline demonstrate the advantage of WebGPU's new capabilities for the task of large-scale forest visualization.

5.1. Limitations and Future Work

One limitation that stops Forender from being used on even larger scenes is that data never gets evicted from the instance data buffer. Without evicting, the size of the buffer would exceed the maximum supported buffer size on many GPUs. A cache system similar to the one used for terrain tiles could alleviate the problem.

Currently, trees are uniformly scaled to tree height h. This approach does not match how trees actually grow. The width of a tree does not necessarily grow proportionally to its height. In the future, the extent of a tree could be estimated during preprocessing, or multiple tree models could be used depending on height.

Large scenes in Forender can suffer from z fighting. The current workaround rescales the scene to decrease the distance between camera and geometry and also moves the near plane further away. These workarounds limit how close the camera can draw geometry and reduce the floating-point resolution of the scene. A potential solution for z fighting would be to implement *Reverse Z* in Forender. The effects of reversing the z buffer are visualized by Nathan Reed[1].

Forender WebGL could be further optimized as WebGPU is not currently supported on all major browsers. Beyond tuning the parameters and reducing visual quality, the LOD metric could be simplified. There appears to be little variation in average AABB sizes, making the average tree screen-space size metric primarily dominated by the distance to the camera. Thus, the LOD metric could be replaced with the cheaper distance metric. The frustum culling check could be similarly improved by using a more efficient test. These optimizations would improve the CPU side of Forender but are unlikely to match the performance of Forender WebGPU.

The thesis primarily focused on handling the geometric complexity and large number of trees but not on visual realism arising from complex lighting and materials. In the future, the current SSAO implementation could be replaced with GTAO [Jimenez et al., 2016] to improve the quality of ambient occlusion. Image-based lighting [Maquignaz, 2025, Maquignaz, 2026] could provide realistic environment lighting, while global illumination techniques [Majercik et al., 2021, Osborne and Sannikov, 2024] could account for indirect illumination. Finally, leaf-specific materials [Wang et al., 2005, Habel et al., 2007] could improve the leaf's appearance.

Currently, the trees do not dynamically cast any shadows in Forender. The only shadows are those statically present in the orthoimages. Dynamic shadows could be implemented using Cascaded Shadow Maps [Engel, 2006], once the terrain has been deshadowed [Staats, 2024]. If shadows are implemented without deshadowing, then the static shadows conflict with the dynamic shadows.

Many modern games trend towards using path-traced rendering. Opacity Micro Maps [Gruen et al., 2020] introduces precomputed masks to accelerate raytracing alpha tested materials. Antwerpen [van Antwerpen et al., 2026] introduces a system for raytracing large animated scenes with varying LOD. WebGPU currently does not support raytracing and bindless textures[2], which makes implementing these approaches less viable.

[1] `https://developer.nvidia.com/blog/visualizing-depth-precision/` (Accessed: 18.08.2026)

[2] `https://www.khronos.org/assets/uploads/developers/presentations/WebGL+WebGPU_-_SIGGRAPH_Jul26.pdf` (Accessed: 18.08.2026)

A

Appendix

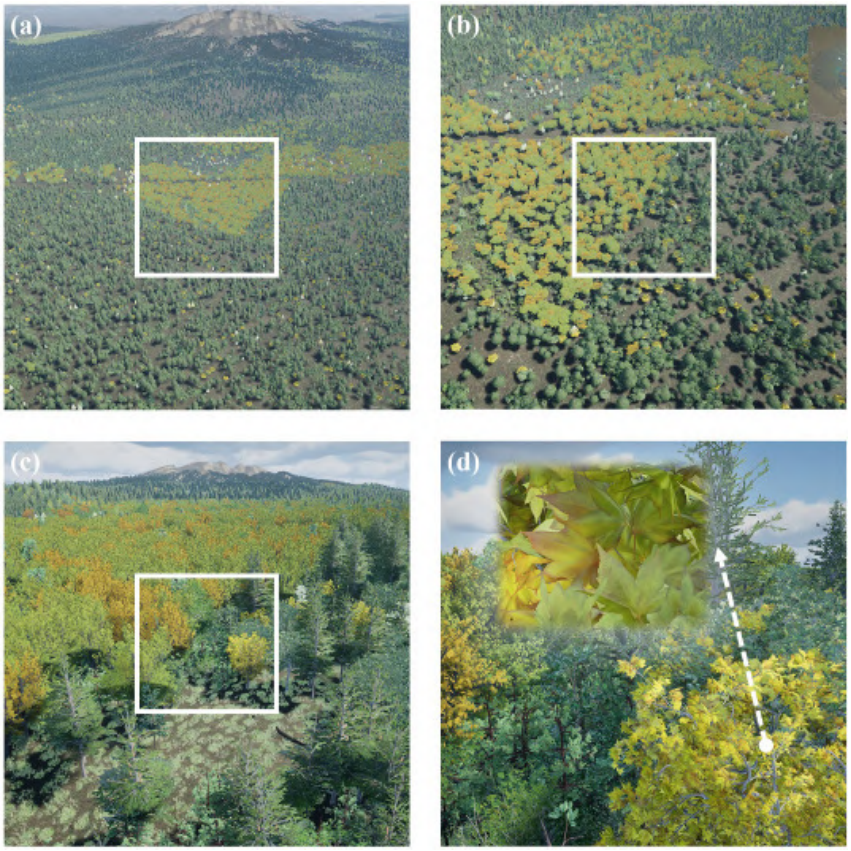
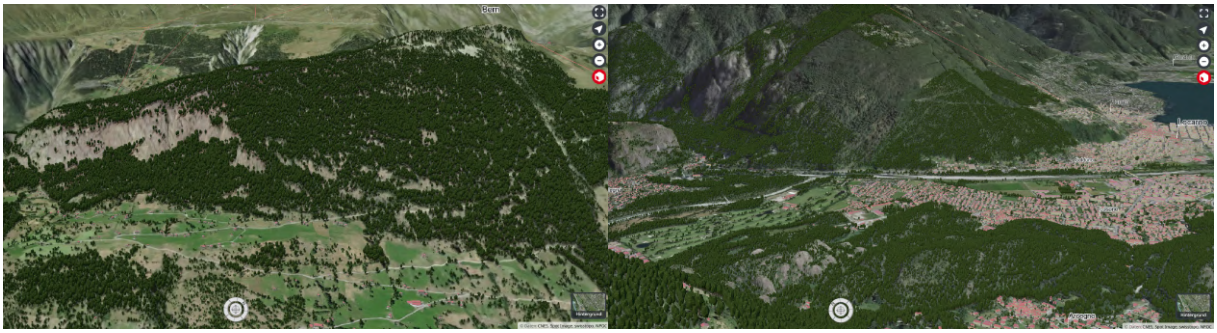


Figure A.1.: FLV visuals: *Images produced in FLV showing the same scene at various distances. The images demonstrate a large viewing distance while maintaining great visual fidelity at close range. However, tiling artifacts are noticeable due to the way trees are placed.*



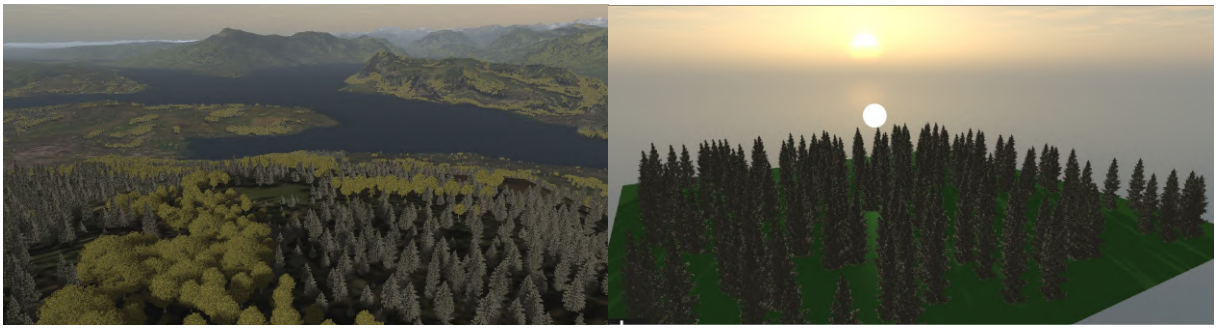
(a) Visual Forester Pro *(b) Stand Designer*

Figure A.2.: Visual Forester visuals: *The left image, produced by Visual Forester Pro, showcases the high render distance of their visualizer, while the image on the right, produced by Stand Designer, shows their weather effects at close range.*



(a) Color Uniformity *(b) Cutoff*

Figure A.3.: swisstopo3D visuals: *Images showcasing forest visualization in swisstopo3D. The left image shows the uniform color of trees when viewed from afar. The right image shows the culling on a per tile basis, with trees incorrectly missing form parts of the hill in the background.*

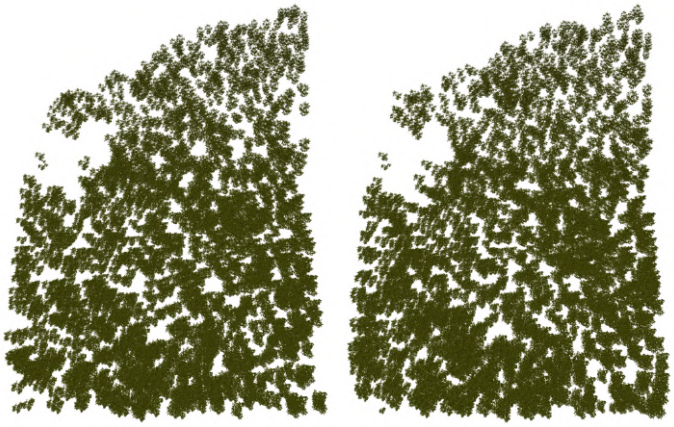


(a) Render distance *(b) Silver lining*

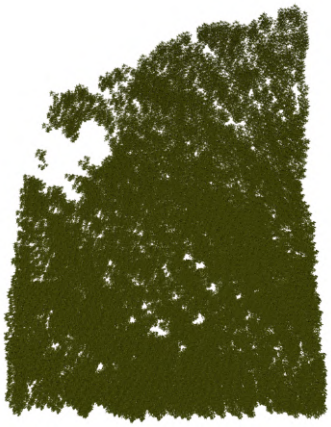
Figure A.4.: Stahl visuals: *The image on the left showcases a large scene with two tree species. The cutoff at the render distance is noticeable from far away, and all trees have the same color. The image on the right shows a silver lining effect.*

LOD	Vertices	Triangles
Maple		
0	13685	11134
1	3071	2278
2	2070	1117
3	1430	673
4	108	36
Fir		
0	10481	11671
1	2267	2287
2	1438	1143
3	998	601
4	59	20

Table A.1.: Geometric complexity of tree LODs: *Vertex and triangle counts of the two tree species across LODs. The LODs were created with similar triangle counts between species as to avoid the species mix rate having a substantial impact on performance.*



(a) *Uniform Level 2* ***(b)*** *Grid-based Level 2*



(c) *Level 3*

Figure A.5.: Remaining density levels with different methods: *Images showcasing the remaining density levels. The clustering becomes less noticeable with higher density. Note that level 3 is the same for both methods because it contains all trees.*

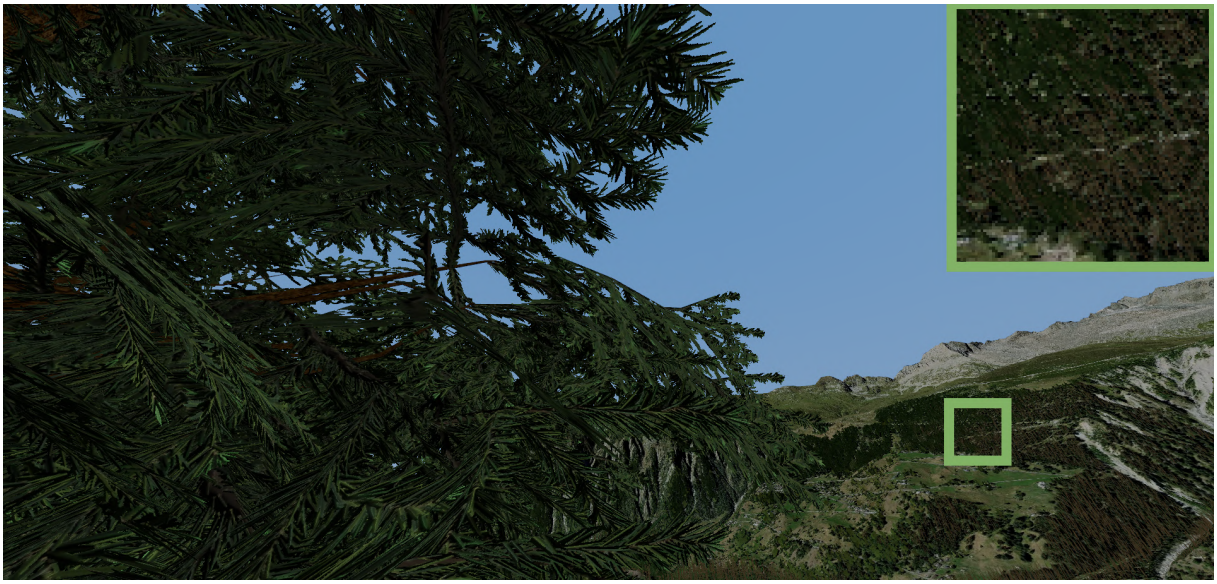
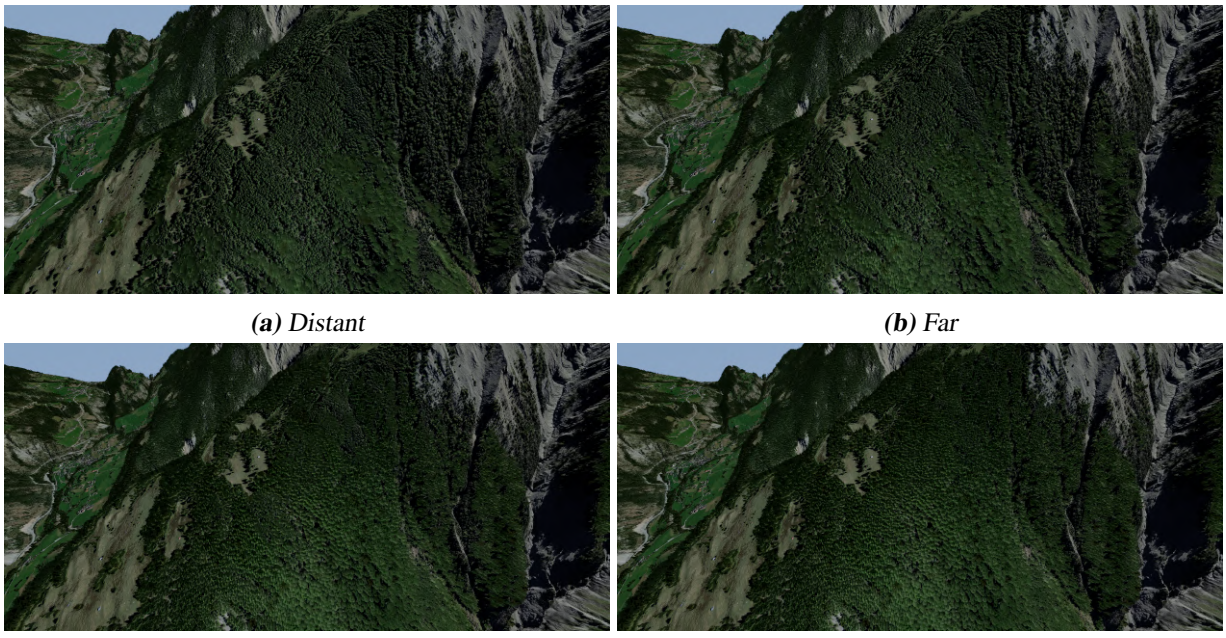


Figure A.6.: Mipmap issues in Alpha Masked material: *Mipmapping albedo textures used for alpha tests can cause foliage to disappear at a distance. To keep the same ratio of pixels passing the alpha test, mipmapping needs to be modified or be compensated for at runtime.*



(a) *Distant* ***(b)*** *Far*

(c) *Moderately far* ***(d)*** *Mid-range*

Figure A.7.: Instance density with Terrain background: *Per instance density decisions for a camera at various distances, visualized from a separate fixed camera with terrain. Note that even mid-range does not show the maximum available density and that this does not use the camera distances as Figure 4.13.*

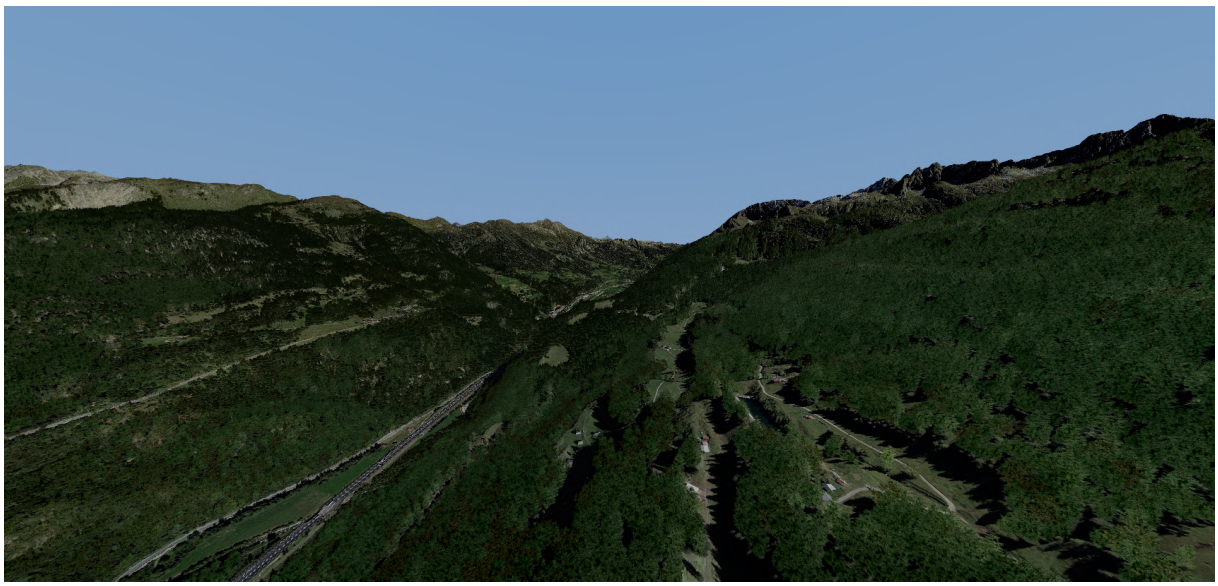


Figure A.8.: Large viewing distance screenshot: *Screenshot showcasing large viewing distances of forests in Forender.*

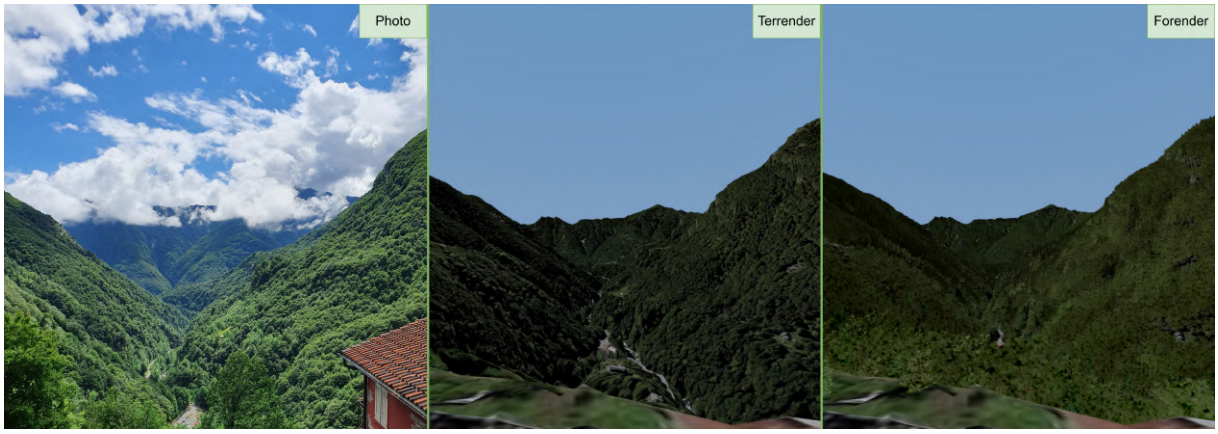


Figure A.9.: Real world comparison: *Additional comparison of a real world photograph with Terrender and Forender. Terrender represents trees only in the terrain texture and thus appears flat, rendering parts of the road obscured by trees in both the photograph and Forender.*

B

Declaration of Usage of AI

AI-based language tools (Grammarly) were used to check grammar throughout the thesis, and LLMs were used to critique the text, though not to write it. LLMs were used to suggest papers for the initial background research but were used in conjunction with tools such as Connected Papers to expand outward from papers recommended by the supervisor. LLMs were used in generating some initial code for the python preprocessor and plotting scripts, which was manually reviewed and replaced in large parts by hand written code later on.

Bibliography

[swi, 2026] (2026). Swisstopo 3d viewer. https://www.swisstopo.admin.ch/en/switzerland-in-3d. Accessed: 29.06.2026.

[Akenine-Möller et al., 2018] Akenine-Möller, T., Haines, E., Hoffman, N., Pesce, A., Iwanicki, M., and Hillaire, S. (2018). *Real-Time Rendering 4th Edition*. A K Peters/CRC Press, Boca Raton, FL, USA.

[Alpine LIS, 2026] Alpine LIS (2026). Visual forester. https://visualforester.com/. Accessed: 07.07.2026.

[Behrendt et al., 2005] Behrendt, S., Colditz, C., Franzke, O., Kopf, J., and Deussen, O. (2005). Realistic real-time rendering of landscapes using billboard clouds. *Computer Graphics Forum*, 24(3):507–516.

[Bösch et al., 2009] Bösch, J., Goswami, P., and Pajarola, R. (2009). Raster: Simple and efficient terrain rendering on the gpu. In *Eurographics*.

[Brucks, 2018] Brucks, R. (2018). Octahedral impostors. https://shaderbits.com/blog/octahedral-impostors. Accessed: 23.02.2026.

[Bruneton and Neyret, 2012] Bruneton, E. and Neyret, F. (2012). Real-time realistic rendering and lighting of forests. *Comput. Graph. Forum*, 31(2pt1):373–382.

[Burley, 2012] Burley, B. (2012). Physically-based shading at disney.

[Cesium GS, 2026] Cesium GS (2026). Cesium js. https://cesium.com/platform/cesiumjs/. Accessed: 07.07.2026.

[Clark, 1976] Clark, J. H. (1976). Hierarchical geometric models for visible surface algorithms. *Commun. ACM*, 19(10):547–554.

[Collin, 2011] Collin, D. (2011). Culling the battlefield: Data oriented design in practice.

GDC 2011. https://www.gdcvault.com/play/1014491/Culling-the-Battlefield-Data-Oriented. Accessed: 23.06.2026.

[Croci et al., 2022] Croci, J. A., Amiraghdam, A., and Pajarola, R. (2022). Terrender: A web-based multi-resolution terrain rendering framework. In *Proceedings of the 27th International Conference on 3D Web Technology*, Web3D '22, New York, NY, USA. Association for Computing Machinery.

[Croci et al., 2025] Croci, J. A., Skorski, C., Rauscher, J., Pajarola, R., and Diehl, A. (2025). Trailview: An interactive 3d hike planning tool. In *Proceedings of the 30th International Conference on 3D Web Technology*, Web3D '25, page 1–10, New York, NY, USA. Association for Computing Machinery.

[Deussen et al., 1998] Deussen, O., Hanrahan, P., Lintermann, B., Měch, R., Pharr, M., and Prusinkiewicz, P. (1998). Realistic modeling and rendering of plant ecosystems. In *Proceedings of the 25th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '98, page 275–286, New York, NY, USA. Association for Computing Machinery.

[Engel, 2006] Engel, W. (2006). Cascaded shadow maps. In Engel, W., editor, $ShaderX^5$, pages 197–206. Charles River Media.

[Epic Games, 2025] Epic Games (2025). Nanite foliage. Unreal Documentation. https://dev.epicgames.com/documentation/unreal-engine/nanite-foliage. Accessed: 29.06.2026.

[ESRI, 2026] ESRI (2026). Introduction to tree management. https://doc.arcgis.com/en/arcgis-solutions/latest/reference/introduction-to-tree-management.htm. Accessed: 07.07.2026.

[Federal office of Topography swisstopo, 2024a] Federal office of Topography swisstopo (2024a). swissALTI3D. https://www.swisstopo.admin.ch/de/hoehenmodell-swissalti3d. Accessed: 13.07.2026.

[Federal office of Topography swisstopo, 2024b] Federal office of Topography swisstopo (2024b). swissIMAGE. https://www.swisstopo.admin.ch/de/orthobilder-swissimage-10-cm. Accessed: 13.07.2026.

[Federal office of Topography swisstopo, 2024c] Federal office of Topography swisstopo (2024c). swissSURFACE3D. https://www.swisstopo.admin.ch/de/hoehenmodell-swisssurface3d. Accessed: 13.07.2026.

[Federal office of Topography swisstopo, 2026] Federal office of Topography swisstopo (2026). swissTLM3D. https://www.swisstopo.admin.ch/de/landschaftsmodell-swisstlm3d. Accessed: 13.07.2026.

[Feeley, 2019] Feeley, S. (2019). Interactive wind and vegetation in 'god of war'. GDC 2019. https://gdcvault.com/play/1026036/Interactive-Wind-and-Vegetation-in. Accessed: 23.02.2026.

[Fuentes, 2022] Fuentes, L. (2022). Designing the terrain system of flight simulator: Representing the earth. GDC 2022, https://gdcvault.com/play/1027581/Advanced-Graphics-Summit-Designing-the. Accessed: 07.07.2027.

[Gasch et al., 2020] Gasch, C., Chover, M., Remolar, I., and Rebollo, C. (2020). Multiresolution foliage rendering. In *Conference of the Spanish Association for Videogames Sciences*.

[Gasch et al., 2018] Gasch, C., Remolar, I., Chover, M., and Rebollo, C. (2018). Viewpoint-driven simplification of plant and tree foliage. *Entropy*, 20.

[Giegl and Wimmer, 2007] Giegl, M. and Wimmer, M. (2007). Unpopping: Solving the image-space blend problem for smooth discrete lod transitions. *Computer Graphics Forum*, 26(1):46–49.

[Grammatikaki et al., 2026] Grammatikaki, A., Eschner, J., Hermosilla, P., Argudo, O., and Waldner, M. (2026). TreeON: Reconstructing 3D Tree Point Clouds from Orthophotos and Heightmaps . *Computer Graphics Forum*.

[Greene, 1995] Greene, N. (1995). *Hierarchical rendering of complex environments*. PhD thesis.

[Greene et al., 1993] Greene, N., Kass, M., and Miller, G. (1993). Hierarchical z-buffer visibility. In *Proceedings of the 20th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '93, page 231–238, New York, NY, USA. Association for Computing Machinery.

[Gruen et al., 2020] Gruen, H., Benthin, C., and Woop, S. (2020). Sub-triangle opacity masks for faster ray tracing of transparent objects. *Proc. ACM Comput. Graph. Interact. Tech.*, 3(2).

[Gumbau et al., 2011] Gumbau, J., Chover, M., Remolar, I., and Rebollo, C. (2011). View-dependent pruning for real-time rendering of trees. *Comput. Graph.*, 35:364–374.

[Haar and Aaltonen, 2015] Haar, U. and Aaltonen, S. (2015). Gpu-driven rendering pipelines. SIGGRAPH Advances in Real-Time Rendering in Games course.

[Habel et al., 2007] Habel, R., Kusternig, A., and Wimmer, M. (2007). Physically based real-time translucency for leaves. In *Proceedings of the 18th Eurographics Conference on Rendering Techniques*, EGSR'07, page 253–263, Goslar, DEU. Eurographics Association.

[Heckbert and Garland, 1997] Heckbert, P. S. and Garland, M. (1997). Survey of polygonal surface simplification algorithms.

[Hu, 2023] Hu, Y. (2023). Broadleaf: Real-time cinematic rendering of large-scale forests. GDC 2023. https://www.gdcvault.com/play/1028974/BroadLeaf-Real-Time-Cinematic-Rendering. Accessed: 23.02.2026.

[Jansson, 2024] Jansson, E. (2024). Gpu-driven rendering with mesh shaders in alan wake 2. Digital Dragons Conference 2024. https://www.youtube.com/watch?v=EtX7WnFhxtQ. Accessed: 23.06.2026.

[Jimenez et al., 2016] Jimenez, J., Wu, X.-C., Pesce, A., Jarabo, A., and Blizzard, A. (2016). Practical real-time strategies for accurate indirect occlusion.

[Kaplanyan, 2010] Kaplanyan, A. (2010). Cryengine 3: Reaching the speed of light. SIGGRAPH Advances in Real-Time Rendering in 3D Graphics and Games course.

[Karis et al., 2021] Karis, B., Stubbe, R., and Wihlidal, G. (2021). Nanite a deep dive. SIGGRAPH Advances in Real-Time Rendering in 3D Graphics and Games course.

[Kerbl et al., 2023] Kerbl, B., Kopanas, G., Leimkuehler, T., and Drettakis, G. (2023). 3d gaussian splatting for real-time radiance field rendering. *ACM Trans. Graph.*, 42(4).

[Khan et al., 2022] Khan, D., Plopski, A., Fujimoto, Y., Kanbara, M., Jabeen, G., Zhang, Y. J., Zhang, X., and Kato, H. (2022). Surface remeshing: A systematic literature review of methods and research directions. *IEEE Transactions on Visualization and Computer Graphics*, 28(3):1680–1713.

[Kratt et al., 2014] Kratt, J., Coconu, L., Dapper, T., Schliep, J. W., Paar, P., and Deussen, O. (2014). Adaptive billboard clouds for botanical tree models. In Hayek, W., editor, *Peer reviewed proceedings of Digital Landscape Architecture 2014 at ETH Zurich*, pages 274–282, Berlin. Wichmann.

[LOLIPOP, 2024] LOLIPOP (2024). Fir trees pack. https://sketchfab.com/3d-models/realistic-fir-trees-pack-lods-gameready-f58e8b6d733e4b0586e5b7db847b89e7. Accessed: 13.07.2026.

[LOLIPOP, 2025] LOLIPOP (2025). Maple trees pack. https://sketchfab.com/3d-models/maple-trees-pack-lowpoly-game-ready-lods-b5d2833c258f4054a01ee2b4ef85adf0. Accessed: 13.07.2026.

[Majercik et al., 2021] Majercik, Z., Marrs, A., Spjut, J., and McGuire, M. (2021). Scaling probe-based real-time dynamic global illumination for production.

[Maquignaz, 2025] Maquignaz, I. J. (2025). Towards physically-based sky-modeling for image based lighting.

[Maquignaz, 2026] Maquignaz, I. J. (2026). Full dynamic range sky-modelling for image based lighting.

[Marty and Ginzler, 2025] Marty, M. and Ginzler, C. (2025). Vegetation Height Model LiDAR NFI. https://www.envidat.ch/dataset/vegetation-height-model-lidar-nfi. Accessed: 13.07.2026.

[McGuire and Bavoil, 2013] McGuire, M. and Bavoil, L. (2013). Weighted blended order-independent transparency. *Journal of Computer Graphics Techniques (JCGT)*, 2(2):122–141.

[Meyer et al., 2001] Meyer, A., Neyret, F., and Poulin, P. (2001). Interactive rendering of trees with shading and shadows. In *Proceedings of the 12th Eurographics Conference on Rendering*, EGWR'01, page 183–196, Goslar, DEU. Eurographics Association.

[Mildenhall et al., 2021] Mildenhall, B., Srinivasan, P. P., Tancik, M., Barron, J. T., Ramamoorthi, R., and Ng, R. (2021). Nerf: representing scenes as neural radiance fields for view synthesis. *Commun. ACM*, 65(1):99–106.

[Mittring, 2007] Mittring, M. (2007). Finding next gen: Cryengine 2. SIGGRAPH Advances in Real-Time Rendering in 3D Graphics and Games course.

[Niese et al., 2022] Niese, T., Pirk, S., Albrecht, M., Benes, B., and Deussen, O. (2022). Procedural urban forestry. *ACM Trans. Graph.*, 41(2).

[Ojo et al., 2024] Ojo, T., La, T., Morton, A., and Stavness, I. (2024). Splanting: 3d plant

capture with gaussian splatting. In *SIGGRAPH Asia 2024 Technical Communications*, SA '24, New York, NY, USA. Association for Computing Machinery.

[Oren et al., 2015] Oren, Y., Kemerlis, V. P., Sethumadhavan, S., and Keromytis, A. D. (2015). The spy in the sandbox – practical cache attacks in javascript.

[Osborne and Sannikov, 2024] Osborne, C. M. J. and Sannikov, A. (2024). Radiance cascades: a novel high-resolution formal solution for multidimensional non-lte radiative transfer. *RAS Techniques and Instruments*, 4.

[Pałubicki et al., 2022] Pałubicki, W., Makowski, M., Gajda, W., Hädrich, T., Michels, D. L., and Pirk, S. (2022). Ecoclimates: climate-response modeling of vegetation. *ACM Trans. Graph.*, 41(4).

[Porter and Duff, 1984] Porter, T. and Duff, T. (1984). Compositing digital images. In *Proceedings of the 11th Annual Conference on Computer Graphics and Interactive Techniques*, SIGGRAPH '84, page 253–259, New York, NY, USA. Association for Computing Machinery.

[Rammer et al., 2026] Rammer, W., Guérin, E., Marais, P., Peytavie, A., Kapp, K., Galin, E., Seidl, R., and Gain, J. (2026). Ecoviz: a tool for visual analysis and photorealistic rendering of forest landscape model simulations. *Ecography*, 2026(4):e08198.

[Remolar et al., 2002] Remolar, I., Chover, M., Belmonte, O., Ribelles, J., and Rebollo, C. (2002). Geometric simplification of foliage. In *Eurographics*.

[Remolar et al., 2003] Remolar, I., Chover, M., Ribelles, J., and Belmonte, O. (2003). View-dependent multiresolution model for foliage. In *International Conference in Central Europe on Computer Graphics and Visualization*.

[Saeed et al., 2024] Saeed, T., Hussain, E., Ullah, S., Iqbal, J., Atif, S., and Yousaf, M. (2024). Performance evaluation of individual tree detection and segmentation algorithms using als data in chir pine (pinus roxburghii) forest. *Remote Sensing Applications: Society and Environment*, 34:101178.

[Sanders, 2018] Sanders, G. (2018). Between tech and art: The vegetation of 'horizon zero dawn'. GDC 2018. https://gdcvault.com/play/1025530/Between-Tech-and-Art-The. Accessed: 07.07.2026.

[Schaufler and Stuerzlinger, 1996] Schaufler, G. and Stuerzlinger, W. (1996). A three dimensional image cache for virtual reality. *Comput. Graph. Forum*, 15:227–236.

[Schütz et al., 2020] Schütz, M., Ohrhallinger, S., and Wimmer, M. (2020). Fast out-of-core octree generation for massive point clouds. *Computer Graphics Forum*, 39(7):155–167.

[Staats, 2024] Staats, M. (2024). Alpine terrain relighting. Master's thesis, Research Unit of Computer Graphics, Institute of Visual Computing and Human-Centered Technology, Faculty of Informatics, TU Wien, Favoritenstrasse 9-11/E193-02, A-1040 Vienna, Austria.

[Stahl, 2021] Stahl, J. (2021). Realistic real-time lighting in large scale vegetation rendering. Master's thesis, VMML UZH Zurich.

[Stora Enso Metsä, 2026] Stora Enso Metsä (2026). Virtual forest.

https://www.metsabiotalous.fi/en/virtuaalimetsa/. Accessed: 07.07.2026.

[van Antwerpen et al., 2026] van Antwerpen, D. G., Gautron, P., Biddulph, E., Kubisch, C., Schmid, J., and Stich, M. (2026). Real-time path tracing of massive dynamic foliage. *Proc. ACM Comput. Graph. Interact. Tech.*, 9(4).

[Wang et al., 2017] Wang, H., Chen, X., Polys, N., and Sforza, P. (2017). A web3d forest geovisualization and user interface evaluation. In *Proceedings of the 22nd International Conference on 3D Web Technology*, Web3D '17, New York, NY, USA. Association for Computing Machinery.

[Wang et al., 2005] Wang, L., Wang, W., Dorsey, J., Yang, X., Guo, B., and Shum, H.-Y. (2005). Real-time rendering of plant leaves. *ACM Trans. Graph.*, 24(3):712–719.

[Waser et al., 2025] Waser, L., Ginzler, C., Psomas, A., Rüetschi, M., and Rehush, N. (2025). Forest Type NFI. https://www.lfi.ch/de/karten/waldmischungsgrad. Accessed: 13.07.2026.

[Whatley, 2005] Whatley, D. (2005). Toward photorealism in virtual botany. *Gpu gems 2: programming techniques for high-performance graphics and general-purpose computation*, pages 7–45.

[Yang et al., 2025] Yang, Y., Shinoda, R., Santo, H., and Okura, F. (2025). Gaussianplant: Structure-aligned gaussian splatting for 3d reconstruction of plants.

[Yu et al., 2022] Yu, H., Wu, M. M., and He, H. S. (2022). Developing platform of 3-d visualization of forest landscape. *Environ. Model. Softw.*, 157(C).